



# Coding For Placement Series

## Python Track

CDP 401 to 418

---

### STUDENT HANDOUT

**BIZOTIC**

*Empowering students towards their first job!*

# TABLE OF CONTENTS

| #        | Block                                                                         | Page |
|----------|-------------------------------------------------------------------------------|------|
| <b>1</b> | <b>CDP 401 - Python Fundamentals: Environment, Data Types, Operators, I/O</b> |      |
|          | Key Concepts                                                                  | 08   |
|          | In-Class Programs                                                             |      |
|          | Program 1 - Electricity Bill Calculator ( <b>Easy</b> )                       | 10   |
|          | Program 2 - Employee Salary Breakdown ( <b>Medium</b> )                       | 11   |
|          | Program 3 - Grocery Store Price Calculator ( <b>Medium</b> )                  | 12   |
|          | Program 4 - Fuel Cost Optimizer ( <b>Hard</b> )                               | 13   |
|          | Program 5 - Temperature Unit Converter ( <b>Easy</b> )                        | 14   |
|          | Challenge Programs (Homework)                                                 |      |
|          | Challenge 1 - Shopkeeper's Discount Calculator ( <b>Medium</b> )              | 15   |
|          | Challenge 2 - Distance Between Two Points ( <b>Medium</b> )                   | 15   |
|          | Challenge 3 - BMI Calculator with Health Advisory ( <b>Medium</b> )           | 16   |
| <b>2</b> | <b>CDP 402 - Conditionals</b>                                                 |      |
|          | Key Concepts                                                                  | 17   |
|          | In-Class Programs                                                             |      |
|          | Program 1 - Student Grade Classifier ( <b>Easy</b> )                          | 19   |
|          | Program 2 - ATM Withdrawal System ( <b>Medium</b> )                           | 19   |
|          | Program 3 - Movie Ticket Pricing Engine ( <b>Medium</b> )                     | 20   |
|          | Program 4 - Loan Eligibility Checker ( <b>Hard</b> )                          | 21   |
|          | Program 5 - Vowel or Consonant Checker ( <b>Easy</b> )                        | 22   |
|          | Challenge Programs (Homework)                                                 |      |
|          | Challenge 1 - Shipping Cost Calculator ( <b>Medium</b> )                      | 23   |
|          | Challenge 2 - Income Tax Calculator ( <b>Hard</b> )                           | 24   |
|          | Challenge 3 - Triangle Classifier ( <b>Medium</b> )                           | 24   |
| <b>3</b> | <b>CDP 403 - Loops + Patterns + Number Problems</b>                           |      |
|          | Key Concepts                                                                  | 26   |
|          | In-Class Programs                                                             |      |
|          | Program 1 - Warehouse Stock Counter ( <b>Easy</b> )                           | 28   |
|          | Program 2 - PIN Validation System ( <b>Medium</b> )                           | 28   |
|          | Program 3 - Staircase Pattern Printer ( <b>Medium</b> )                       | 29   |
|          | Program 4 - Armstrong Number Range Finder ( <b>Hard</b> )                     | 30   |
|          | Program 5 - Concatenation of Array (LeetCode 1929, <b>Easy</b> )              | 30   |

|          |                                                                                                        |    |
|----------|--------------------------------------------------------------------------------------------------------|----|
|          | Challenge Programs (Homework)                                                                          |    |
|          | Challenge 1 - Digit Sum Reducer ( <b>Medium</b> )                                                      | 31 |
|          | Challenge 2 - Diamond Pattern ( <b>Medium</b> )                                                        | 31 |
|          | Challenge 3 - Happy Number (LeetCode 202, <b>Medium</b> )                                              | 32 |
| <b>4</b> | <b>CDP 404 - Functions</b>                                                                             |    |
|          | Key Concepts                                                                                           | 33 |
|          | In-Class Programs                                                                                      |    |
|          | Program 1 - Unit Converter Toolkit ( <b>Easy</b> )                                                     | 35 |
|          | Program 2 - Student Report Card Generator ( <b>Medium</b> )                                            | 35 |
|          | Program 3 - Prime Number Toolkit ( <b>Medium</b> )                                                     | 36 |
|          | Program 4 - Password Strength Analyzer ( <b>Hard</b> )                                                 | 37 |
|          | Program 5 - Concatenation of Array + Build Array from Permutation (LeetCode 1929/1920, <b>Medium</b> ) | 38 |
|          | Challenge Programs (Homework)                                                                          |    |
|          | Challenge 1 - GCD and LCM Calculator ( <b>Medium</b> )                                                 | 38 |
|          | Challenge 2 - Statistical Calculator ( <b>Medium</b> )                                                 | 39 |
|          | Challenge 3 - Power of Two (LeetCode 231, <b>Medium</b> )                                              | 39 |
| <b>5</b> | <b>CDP 405 - Lists (1D)</b>                                                                            |    |
|          | Key Concepts                                                                                           | 40 |
|          | In-Class Programs                                                                                      |    |
|          | Program 1 - Inventory Stock Tracker ( <b>Easy</b> )                                                    | 42 |
|          | Program 2 - Student Marks Filter ( <b>Medium</b> )                                                     | 42 |
|          | Program 3 - Vote Counter ( <b>Medium</b> )                                                             | 43 |
|          | Program 4 - Best Time to Buy and Sell Stock (LeetCode 121, <b>Hard</b> )                               | 44 |
|          | Program 5 - Running Sum of Array ( <b>Medium</b> )                                                     | 44 |
|          | Challenge Programs (Homework)                                                                          |    |
|          | Challenge 1 - Remove Duplicates from List ( <b>Medium</b> )                                            | 45 |
|          | Challenge 2 - Move Zeroes (LeetCode 283, <b>Medium</b> )                                               | 45 |
|          | Challenge 3 - Single Number (LeetCode 136, <b>Medium</b> )                                             | 45 |
| <b>6</b> | <b>CDP 406 - Lists (2D) and Tuples</b>                                                                 |    |
|          | Key Concepts                                                                                           | 47 |
|          | In-Class Programs                                                                                      |    |
|          | Program 1 - Classroom Seating Map ( <b>Easy</b> )                                                      | 49 |
|          | Program 2 - Exam Score Analyzer ( <b>Medium</b> )                                                      | 49 |
|          | Program 3 - Matrix Transpose ( <b>Medium</b> )                                                         | 50 |

|          |                                                                                                |    |
|----------|------------------------------------------------------------------------------------------------|----|
|          | Program 4 - Rotate Image (LeetCode 48, <b>Hard</b> )                                           | 50 |
|          | Program 5 - Diagonal Sum Calculator ( <b>Medium</b> )                                          | 51 |
|          | Challenge Programs (Homework)                                                                  |    |
|          | Challenge 1 - Boundary Traversal ( <b>Medium</b> )                                             | 51 |
|          | Challenge 2 - Row with Maximum Ones ( <b>Medium</b> )                                          | 52 |
|          | Challenge 3 - Spiral Matrix (LeetCode 54, <b>Medium</b> )                                      | 52 |
| <b>7</b> | <b>CDP 407 - Strings (Priority Block)</b>                                                      |    |
|          | Key Concepts                                                                                   | 53 |
|          | In-Class Programs                                                                              |    |
|          | Program 1 - Username Validator ( <b>Easy</b> )                                                 | 55 |
|          | Program 2 - Valid Palindrome (LeetCode 125, <b>Medium</b> )                                    | 56 |
|          | Program 3 - Word Reversal Engine ( <b>Medium</b> )                                             | 56 |
|          | Program 4 - Longest Common Prefix (LeetCode 14, <b>Hard</b> )                                  | 57 |
|          | Program 5 - Anagram Checker ( <b>Medium</b> )                                                  | 57 |
|          | Challenge Programs (Homework)                                                                  |    |
|          | Challenge 1 - Caesar Cipher Encoder/Decoder ( <b>Medium</b> )                                  | 58 |
|          | Challenge 2 - Find the Index of the First Occurrence in a String (LeetCode 28, <b>Medium</b> ) | 58 |
|          | Challenge 3 - Roman to Integer (LeetCode 13, <b>Medium</b> )                                   | 59 |
| <b>8</b> | <b>CDP 408 - Recursion</b>                                                                     |    |
|          | Key Concepts                                                                                   | 60 |
|          | In-Class Programs                                                                              |    |
|          | Program 1 - Power Calculator ( <b>Easy</b> )                                                   | 62 |
|          | Program 2 - List Sum and Max Finder ( <b>Medium</b> )                                          | 62 |
|          | Program 3 - String Reversal and Palindrome Checker ( <b>Medium</b> )                           | 63 |
|          | Program 4 - Fibonacci Number (LeetCode 509, <b>Hard</b> )                                      | 63 |
|          | Program 5 - Power of Two (LeetCode 231, <b>Medium</b> )                                        | 64 |
|          | Challenge Programs (Homework)                                                                  |    |
|          | Challenge 1 - Tower of Hanoi ( <b>Medium</b> )                                                 | 64 |
|          | Challenge 2 - Pow(x, n) (LeetCode 50, <b>Medium</b> )                                          | 65 |
|          | Challenge 3 - Reverse String (LeetCode 344, <b>Medium</b> )                                    | 65 |
| <b>9</b> | <b>CDP 409 - Searching and Sorting</b>                                                         |    |
|          | Key Concepts                                                                                   | 66 |
|          | In-Class Programs                                                                              |    |
|          | Program 1 - Library Book Finder ( <b>Easy</b> )                                                | 68 |

|           |                                                                                          |    |
|-----------|------------------------------------------------------------------------------------------|----|
|           | Program 2 - Student Rank Sorter (Medium)                                                 | 68 |
|           | Program 3 - Merge Sort Implementation (Medium)                                           | 69 |
|           | Program 4 - Find First and Last Position of Element in Sorted Array (LeetCode 34, Hard)  | 69 |
|           | Program 5 - Sort Array By Parity (LeetCode 905, Medium)                                  | 70 |
|           | Challenge Programs (Homework)                                                            |    |
|           | Challenge 1 - Sort Colors (LeetCode 75, Medium)                                          | 70 |
|           | Challenge 2 - Search Insert Position (LeetCode 35, Medium)                               | 70 |
|           | Challenge 3 - Merge Intervals (LeetCode 56, Hard)                                        | 71 |
| <b>10</b> | <b>CDP 410 - Dictionaries and Sets</b>                                                   |    |
|           | Key Concepts                                                                             | 72 |
|           | In-Class Programs                                                                        |    |
|           | Program 1 - Word Frequency Counter (Easy)                                                | 74 |
|           | Program 2 - Two Sum (LeetCode 1, Medium)                                                 | 75 |
|           | Program 3 - Student Grade Analyzer with Dict (Medium)                                    | 75 |
|           | Program 4 - Group Anagrams (LeetCode 49, Hard)                                           | 76 |
|           | Program 5 - Duplicate Finder with Set (Medium)                                           | 77 |
|           | Challenge Programs (Homework)                                                            |    |
|           | Challenge 1 - Valid Anagram (LeetCode 242, Medium)                                       | 77 |
|           | Challenge 2 - Contains Duplicate (LeetCode 217, Medium)                                  | 78 |
|           | Challenge 3 - Intersection of Two Arrays II (LeetCode 350, Medium)                       | 78 |
| <b>11</b> | <b>CDP 411 - OOP 1: Classes, Objects, Constructors, Encapsulation</b>                    |    |
|           | Key Concepts                                                                             | 80 |
|           | In-Class Programs                                                                        |    |
|           | Program 1 - Bank Account Manager (Easy)                                                  | 83 |
|           | Program 2 - Student Management System (Medium)                                           | 83 |
|           | Program 3 - Design Parking System (LeetCode 1603, Medium)                                | 84 |
|           | Program 4 - Employee Payroll System (Hard)                                               | 85 |
|           | Program 5 - Design HashMap Simplified (LeetCode 706, Medium)                             | 85 |
|           | Challenge Programs (Homework)                                                            |    |
|           | Challenge 1 - Library Book System (Medium)                                               | 86 |
|           | Challenge 2 - Range Sum Query Immutable (LeetCode 303, Medium)                           | 87 |
|           | Challenge 3 - Implement Stack using Queues (LeetCode 225, Medium)                        | 87 |
| <b>12</b> | <b>CDP 412 - OOP 2: Inheritance, Polymorphism, Abstract Classes + Exception Handling</b> |    |

|           |                                                                             |     |
|-----------|-----------------------------------------------------------------------------|-----|
|           | Key Concepts                                                                | 88  |
|           | In-Class Programs                                                           |     |
|           | Program 1 - Vehicle Hierarchy (Easy)                                        | 91  |
|           | Program 2 - Employee Salary System with Inheritance (Medium)                | 91  |
|           | Program 3 - Abstract Shape Calculator (Medium)                              | 92  |
|           | Program 4 - Safe Calculator with Custom Exceptions (Hard)                   | 93  |
|           | Program 5 - Inheritance + Exception Combo: Banking System (Medium)          | 94  |
|           | Challenge Programs (Homework)                                               |     |
|           | Challenge 1 - Min Stack (LeetCode 155, Medium)                              | 94  |
|           | Challenge 2 - Design Browser History (LeetCode 1472, Medium)                | 95  |
|           | Challenge 3 - Implement Queue using Stacks (LeetCode 232, Medium)           | 95  |
| <b>13</b> | <b>CDP 413 - Problem Solving 1</b>                                          |     |
|           | Key Concepts                                                                | 97  |
|           | In-Class Programs                                                           |     |
|           | Program 1 - Majority Element (LeetCode 169, Easy)                           | 98  |
|           | Program 2 - Three Sum (LeetCode 15, Medium)                                 | 98  |
|           | Program 3 - Inventory Restock Analyzer (Medium)                             | 98  |
|           | Program 4 - Product of Array Except Self (LeetCode 238, Hard)               | 99  |
|           | Program 5 - Find All Numbers Disappeared in an Array (LeetCode 448, Medium) | 100 |
|           | Challenge Programs (Homework)                                               |     |
|           | Challenge 1 - Subarray Sum Equals K (LeetCode 560, Medium)                  | 100 |
|           | Challenge 2 - Longest Consecutive Sequence (LeetCode 128, Medium)           | 101 |
|           | Challenge 3 - Container With Most Water (LeetCode 11, Hard)                 | 101 |
| <b>14</b> | <b>CDP 414 - Problem Solving 2</b>                                          |     |
|           | Key Concepts                                                                | 102 |
|           | In-Class Programs                                                           |     |
|           | Program 1 - Single Number (LeetCode 136, Easy)                              | 102 |
|           | Program 2 - Logistics Route Optimizer (Medium)                              | 103 |
|           | Program 3 - Top K Frequent Elements (LeetCode 347, Medium)                  | 103 |
|           | Program 4 - Maximum Subarray, Kadane's Algorithm (LeetCode 53, Hard)        | 104 |
|           | Program 5 - Set Matrix Zeroes (LeetCode 73, Hard)                           | 104 |
|           | Challenge Programs (Homework)                                               |     |
|           | Challenge 1 - Trapping Rain Water (LeetCode 42, Hard)                       | 105 |
|           | Challenge 2 - Rotate Array (LeetCode 189, Medium)                           | 105 |

|           |                                                                        |     |
|-----------|------------------------------------------------------------------------|-----|
|           | Challenge 3 - 4Sum II (LeetCode 454, <b>Medium</b> )                   | 106 |
| <b>15</b> | <b>CDP 415 - Problem Solving 3</b>                                     |     |
|           | Key Concepts                                                           | 107 |
|           | In-Class Programs                                                      |     |
|           | Program 1 - Flight Booking Analyzer ( <b>Medium</b> )                  | 108 |
|           | Program 2 - Employee Performance Ranker ( <b>Medium</b> )              | 108 |
|           | Program 3 - Warehouse Inventory Reconciliation ( <b>Hard</b> )         | 109 |
|           | Program 4 - Group Anagrams, Timed Revisit (LeetCode 49, <b>Hard</b> )  | 110 |
|           | Program 5 - Online Shopping Cart ( <b>Medium</b> )                     | 110 |
|           | Challenge Programs (Homework)                                          |     |
|           | Challenge 1 - Merge Intervals (LeetCode 56, <b>Medium</b> )            | 111 |
|           | Challenge 2 - Sort Colors (LeetCode 75, <b>Medium</b> )                | 111 |
|           | Challenge 3 - Sliding Window Maximum (LeetCode 239, <b>Hard</b> )      | 112 |
| <b>16</b> | <b>CDP 416 - Company Simulation 1, TCS NQT Pattern</b>                 |     |
|           | Problem 1 (Medium) - Courier Tracking Code Processor                   | 113 |
|           | Problem 2 (Hard) - Factory Production Analyzer                         | 114 |
|           | Problem 3 (Hard) - Telecom Call Record Analyzer                        | 115 |
| <b>17</b> | <b>CDP 417 - Company Simulation 2, Infosys and Cognizant Pattern</b>   |     |
|           | Problem 1 (Medium) - Social Media Hashtag Analyzer                     | 117 |
|           | Problem 2 (Hard) - Gift Box Combination Finder                         | 118 |
|           | Problem 3 (Hard) - Library Management System                           | 119 |
| <b>18</b> | <b>CDP 418 - Company Simulation 3, Capgemini and Accenture Pattern</b> |     |
|           | Problem 1 (Medium) - Hospital Records Processor                        | 121 |
|           | Problem 2 (Medium) - Vehicle Insurance Premium Calculator              | 122 |
|           | Problem 3 (Hard) - Food Delivery Platform Analytics                    | 123 |

# CDP 401 - Python Fundamentals: Environment, Data Types, Operators, I/O

## KEY CONCEPTS

### Opening

Welcome to the Python track. Over the next 15 blocks, you will learn Python from the ground up, and by the end, you will be solving the same kinds of problems that appear in TCS, Infosys, Cognizant, Capgemini, and Accenture coding tests. Python is one of the most widely used programming languages in the world. It is used in web development, data science, automation, and artificial intelligence. More importantly for you, Python is accepted in almost every company coding test, and its clean syntax lets you solve problems faster.

Before we begin, confirm that you have Python installed. Open a terminal and type `python3 --version`. You should see Python 3.12 or higher.

### The First Python Program

Let us write our first Python program.

```
print("Hello World")
```

That is it. One line. No boilerplate, no semicolons, no compilation step. You write the code, you run it. Compare this to Java or C where you need `public class`, `public static void main`, curly braces, and semicolons before you can print a single line. Python removes all of that.

To run this, save it as `hello.py` and type `python3 hello.py` in your terminal. You will see `Hello World` printed.

`print()` is the function that displays output. You will use it in every program.

### Variables and Dynamic Typing

In Python, you do not declare types. You just assign values.

```
x = 5          # x is an integer
x = "hello"    # now x is a string
name = "Alice"
age = 20
pi = 3.14
is_student = True
```

Notice: no `int x = 5`, no `String name`, no type declarations at all. Python figures out the type from the value you assign. This is called dynamic typing.

You can check the type of any variable using the `type()` function: `type(x)` will tell you `<class 'str'>`.

The four types you will use most often are:

|                    |                   |                  |                             |
|--------------------|-------------------|------------------|-----------------------------|
| <code>int</code>   | - whole numbers   | (5, -3, 1000)    | No size limit in Python!    |
| <code>float</code> | - decimal numbers | (3.14, -0.5)     |                             |
| <code>str</code>   | - text            | ("Hello", "123") | Use double or single quotes |
| <code>bool</code>  | - True or False   | (True, False)    | Capital T and F!            |

Python integers have no size limit. You can compute `2 ** 1000` and Python handles it. Java and C have overflow issues with large numbers. Python does not.





## Dynamic Typing Trap

Dynamic typing is convenient, but it has a trap.

```
x = "5"
y = 3
print(x + y)
```

What do you think this prints? Let us run it.

Python does NOT auto-convert between str and int. Java would not even let you compile this. Python lets you write it, but crashes at runtime. The fix is explicit conversion:

```
print(int(x) + y)      # converts "5" to 5, then adds: result is 8
print(x + str(y))      # converts 3 to "3", then concatenates: result is "53"
```

Remember this: Python never guesses what you mean. If you mix types, you must convert explicitly.

## Input

Most programs need input from the user. In Python, you use the `input()` function.

```
name = input("Enter your name: ")
age = input("Enter your age: ")
print(f"Hello {name}, you are {age} years old.")
```

There is one critical thing to know: `input()` ALWAYS returns a string. Even if the user types a number, Python stores it as a string.

```
age = input("Enter your age: ")
print(age + 1)  # TypeError! age is a string, not an int
```

The fix: wrap `input()` with `int()` or `float()` to convert.

```
age = int(input("Enter your age: "))      # now age is an int
price = float(input("Enter price: "))     # now price is a float
```

This pattern, `int(input())`, is what you will use in almost every program.

## Operators

Python operators work the way you expect, with a few important differences.

```
Arithmetic: + - * / // % **
Relational: == != < > <= >=
Logical:    and or not      (words, not symbols!)
Assignment: = += -= *= /= //= **=
```

The most important things to know:

`/` always returns a float: `7 / 2` gives `3.5`. This is different from Java and C, where `7 / 2` gives `3`.

`//` is floor division (integer division): `7 // 2` gives `3`. Use this when you want the integer result.

`**` is the power operator: `2 ** 10` gives `1024`. No need for `math.pow()`.

`%` is modulus (remainder): `7 % 2` gives `1`. Used for checking even/odd, extracting digits, cycling through values.

```
7 / 2 = 3.5      (always float)
7 // 2 = 3       (floor division)
```

Logical operators use words: `and`, `or`, `not`. Not `&&`, `||`, `!` like Java and C.

## Type Casting

Sometimes you need to convert between types explicitly.

`int(3.7)` gives `3`. Python truncates, it does not round.



`float(5)` gives 5.0.

`str(42)` gives "42". Useful when you need to concatenate a number into a string.

`int("5")` gives 5. This is how you convert string input to a number.

## Indentation as Syntax

In Python, indentation is not optional. It defines code blocks.

```
x = 10
if x > 5:
    print("big")
```

In Java and C, curly braces `{ }` define code blocks. In Python, indentation does. Every line inside an if, else, for, while, or function must be indented by 4 spaces.

```
x = 10
if x > 5:
    print("big")      # 4 spaces indent
else:
    print("small")    # 4 spaces indent
```

Most code editors add the 4 spaces automatically when you press Tab after a colon. But if you mix tabs and spaces, Python will give you an error. Use 4 spaces consistently.

## f-strings for Formatted Output

Python's cleanest way to print formatted output is f-strings.

```
name = "Alice"
score = 87.5
print(f"Student: {name}, Score: {score:.2f}")
# Output: Student: Alice, Score: 87.50
```

The `f` before the quote makes it an f-string. Inside curly braces `{ }`, you put any variable or expression. `:.2f` means 2 decimal places.

## Comments

Use `#` for comments. Everything after `#` on a line is ignored by Python.

```
# This is a comment
x = 5 # This is also a comment
```

# IN-CLASS PROGRAMS

## Program 1: Electricity Bill Calculator (Easy)

### Problem Statement:

A state electricity board charges customers based on units consumed. The pricing tiers are:

- First 100 units: Rs 1.50 per unit
- Next 100 units (101 to 200): Rs 2.50 per unit
- Above 200 units: Rs 4.00 per unit

Given the number of units consumed, calculate the total electricity bill amount.

### Examples:

- Input: 75. Output: 112.50 ( $75 * 1.50 = 112.50$ )
- Input: 150. Output: 275.00 ( $100 * 1.50 + 50 * 2.50 = 150 + 125 = 275.00$ )
- Input: 350. Output: 1000.00 ( $100 * 1.50 + 100 * 2.50 + 150 * 4.00 = 150 + 250 + 600 = 1000.00$ )



## Test Cases:

| # | Input | Expected Output | Notes                        |
|---|-------|-----------------|------------------------------|
| 1 | 75    | 112.50          | First tier only              |
| 2 | 150   | 275.00          | First and second tier        |
| 3 | 350   | 1000.00         | All three tiers              |
| 4 | 100   | 150.00          | Exact boundary of first tier |
| 5 | 0     | 0.00            | Edge case: zero consumption  |

## Program 2: Employee Salary Breakdown (Medium)

### Problem Statement:

A company calculates employee take-home salary using the following rules:

- HRA (House Rent Allowance) = 20% of basic salary
- DA (Dearness Allowance) = 50% of basic salary
- PF (Provident Fund) deduction = 12% of basic salary
- Tax = 10% of gross salary (where gross = basic + HRA + DA)

Given the basic salary as input, calculate and print:

1. Gross salary (basic + HRA + DA)
2. Total deductions (PF + Tax)
3. Net salary (Gross - Total deductions)

### Examples:

- Input: 50000.  
- HRA = 10000, DA = 25000, Gross = 85000.  
- PF = 6000, Tax = 8500, Total deductions = 14500.  
- Net salary = 70500.00.
- Input: 30000.  
- HRA = 6000, DA = 15000, Gross = 51000.  
- PF = 3600, Tax = 5100, Total deductions = 8700.  
- Net salary = 42300.00.

## Test Cases:

| # | Input (Basic) | Gross     | Total Deductions | Net Salary | Notes                  |
|---|---------------|-----------|------------------|------------|------------------------|
| 1 | 50000         | 85000.00  | 14500.00         | 70500.00   | Standard case          |
| 2 | 30000         | 51000.00  | 8700.00          | 42300.00   | Smaller salary         |
| 3 | 100000        | 170000.00 | 29000.00         | 141000.00  | Higher salary          |
| 4 | 10000         | 17000.00  | 2900.00          | 14100.00   | Low salary             |
| 5 | 0             | 0.00      | 0.00             | 0.00       | Edge case: zero salary |

### Program 3: Grocery Store Price Calculator (Medium)

#### Problem Statement:

A grocery store calculates the final price for a customer. Given:

- Item name (string)
- Quantity (integer)
- Price per unit (float)
- Membership status (1 = member, 0 = non-member)

Calculate the final bill as follows:

1. Subtotal = quantity \* price per unit
2. If the customer is a member, apply 10% discount on subtotal
3. If the amount after discount exceeds Rs 500, apply 5% GST. Otherwise, apply 12% GST.
4. Final total = amount after discount + GST

Print: item name, subtotal, discount (if any), GST amount, and final total.

#### Examples:

- Input: Rice, 10, 60.0, 1. Subtotal = 600.00. Member discount = 60.00. After discount = 540.00. 540 > 500, so 5% GST = 27.00. Total = 567.00.
- Input: Soap, 3, 50.0, 0. Subtotal = 150.00. No discount. 150 ≤ 500, so 12% GST = 18.00. Total = 168.00.

#### Test Cases:

| # | Item | Qty | Price | Member | Subtotal | Discount | GST   | Total  | Notes                                 |
|---|------|-----|-------|--------|----------|----------|-------|--------|---------------------------------------|
| 1 | Rice | 10  | 60.0  | 1      | 600.00   | 60.00    | 27.00 | 567.00 | Member, after discount > 500, 5% GST  |
| 2 | Rice | 10  | 60.0  | 0      | 600.00   | 0.00     | 30.00 | 630.00 | Non-member, subtotal > 500, 5% GST    |
| 3 | Soap | 3   | 50.0  | 1      | 150.00   | 15.00    | 16.20 | 151.20 | Member, after discount ≤ 500, 12% GST |
| 4 | Soap | 3   | 50.0  | 0      | 150.00   | 0.00     | 18.00 | 168.00 | Non-member, subtotal ≤ 500, 12% GST   |
| 5 | Oil  | 1   | 500.0 | 1      | 500.00   | 50.00    | 54.00 | 504.00 | Member, after discount                |

|  |  |  |  |  |  |  |  |  |                     |
|--|--|--|--|--|--|--|--|--|---------------------|
|  |  |  |  |  |  |  |  |  | 450 <= 500, 12% GST |
|--|--|--|--|--|--|--|--|--|---------------------|

## Program 4: Fuel Cost Optimizer (Hard)

### Problem Statement:

A logistics company operates trucks between cities. Given:

- Distance in km
- Truck mileage (km per litre)
- Fuel price per litre
- Toll charges (flat amount, one-way)
- Driver daily wage
- Budget for the trip

The trip takes  $\text{ceil}(\text{distance} / 500)$  days (maximum 500 km per day).

Calculate:

1. Fuel cost = (distance / mileage) \* fuel price
2. Driver cost = daily wage \* number of days
3. Total trip cost = fuel cost + toll + driver cost

If total trip cost exceeds the budget, print "OVER BUDGET by X". Otherwise, print "WITHIN BUDGET, savings = Y".

### Examples:

- Input: distance=1000, mileage=10, fuel\_price=100, toll=500, daily\_wage=800, budget=15000.  
 - Days =  $\text{ceil}(1000/500) = 2$ . Fuel =  $(1000/10)*100 = 10000$ . Driver =  $800*2 = 1600$ . Total =  $10000+500+1600 = 12100$ .  
 -  $12100 < 15000$ . Output: "WITHIN BUDGET, savings = 2900.00".
- Input: distance=2000, mileage=8, fuel\_price=105, toll=1000, daily\_wage=900, budget=30000.  
 - Days =  $\text{ceil}(2000/500) = 4$ . Fuel =  $(2000/8)*105 = 26250$ . Driver =  $900*4 = 3600$ . Total =  $26250+1000+3600 = 30850$ .  
 -  $30850 > 30000$ . Output: "OVER BUDGET by 850.00".

### Test Cases:

| # | Distance | Mileage | Fuel Price | Toll | Daily Wage | Budget | Total Cost | Result                           | Notes         |
|---|----------|---------|------------|------|------------|--------|------------|----------------------------------|---------------|
| 1 | 1000     | 10      | 100        | 500  | 800        | 15000  | 12100.00   | WITHIN BUDGET, savings = 2900.00 | Standard case |
| 2 | 2000     | 8       | 105        | 1000 | 900        | 30000  | 30850.00   | OVER BUDGET by 850.00            | Over budget   |

|   |     |    |     |     |     |       |         |                                  |                        |
|---|-----|----|-----|-----|-----|-------|---------|----------------------------------|------------------------|
| 3 | 500 | 10 | 100 | 200 | 800 | 10000 | 6000.00 | WITHIN BUDGET, savings = 4000.00 | Short distance, 1 day  |
| 4 | 750 | 10 | 100 | 300 | 800 | 10000 | 9400.00 | WITHIN BUDGET, savings = 600.00  | ceil(750/500) = 2 days |
| 5 | 0   | 10 | 100 | 0   | 800 | 5000  | 0.00    | WITHIN BUDGET, savings = 5000.00 | Edge: zero distance    |

## Program 5: Temperature Unit Converter (Easy)

### Problem Statement:

A weather monitoring station receives temperature readings in Celsius. For each reading, the system must:

1. Convert to Fahrenheit:  $F = C * 9/5 + 32$
2. Convert to Kelvin:  $K = C + 273.15$
3. Classify the weather:
  - Below 0 degrees C: "Freezing"
  - 0 to 15 degrees C: "Cold"
  - 16 to 30 degrees C: "Pleasant"
  - Above 30 degrees C: "Hot"

Given a temperature in Celsius, print the Fahrenheit value, Kelvin value, and classification.

### Examples:

- Input: 25. Output: F = 77.0, K = 298.15, Classification = Pleasant.
- Input: -5. Output: F = 23.0, K = 268.15, Classification = Freezing.
- Input: 0. Output: F = 32.0, K = 273.15, Classification = Cold.

### Test Cases:

| # | Input (C) | Fahrenheit | Kelvin | Classification | Notes                             |
|---|-----------|------------|--------|----------------|-----------------------------------|
| 1 | 25        | 77.0       | 298.15 | Pleasant       | Middle range                      |
| 2 | -5        | 23.0       | 268.15 | Freezing       | Below zero                        |
| 3 | 35        | 95.0       | 308.15 | Hot            | Above 30                          |
| 4 | 0         | 32.0       | 273.15 | Cold           | Boundary: 0 is Cold, not Freezing |
| 5 | 100       | 212.0      | 373.15 | Hot            | Boiling point of water            |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Shopkeeper's Discount Calculator (Medium)

#### Problem Statement:

A shopkeeper gives discounts based on purchase amount:

- 0 to 500: No discount
- 501 to 1000: 10% off
- 1001 to 2000: 15% off
- Above 2000: 20% off

If the customer is a member (membership = 1), they get an additional 5% off on the already discounted price.

Given the purchase amount and membership status (1 or 0), calculate and print the final amount to pay.

#### Examples:

- Input: amount=800, member=1. 10% off 800 = 720. Member 5% off 720 = 684.00. Output: 684.00.
- Input: amount=1500, member=0. 15% off 1500 = 1275.00. No member discount. Output: 1275.00.

#### Test Cases:

| # | Amount | Member | Final Amount | Notes                                |
|---|--------|--------|--------------|--------------------------------------|
| 1 | 300    | 0      | 300.00       | No discount, non-member              |
| 2 | 300    | 1      | 285.00       | No tier discount, member gets 5% off |
| 3 | 800    | 0      | 720.00       | 10% off, non-member                  |
| 4 | 800    | 1      | 684.00       | 10% off then 5% member               |
| 5 | 3000   | 1      | 2280.00      | 20% off then 5% member               |

### Challenge 2: Distance Between Two Points (Medium)

#### Problem Statement:

A drone delivery service needs to calculate the straight-line distance between the warehouse at coordinates (x1, y1) and a delivery location at coordinates (x2, y2).

Use the distance formula:  $\text{distance} = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$

Print the distance rounded to 2 decimal places.

#### Examples:

- Input: (0, 0) to (3, 4). Distance = 5.00.
- Input: (1, 1) to (4, 5). Distance = 5.00.

#### Test Cases:



| # | x1 | y1 | x2 | y2 | Distance | Notes                     |
|---|----|----|----|----|----------|---------------------------|
| 1 | 0  | 0  | 3  | 4  | 5.00     | Classic 3-4-5 triangle    |
| 2 | 1  | 1  | 4  | 5  | 5.00     | Shifted but same distance |
| 3 | 0  | 0  | 0  | 0  | 0.00     | Same point                |
| 4 | -3 | -4 | 0  | 0  | 5.00     | Negative coordinates      |
| 5 | 0  | 0  | 1  | 1  | 1.41     | Diagonal of unit square   |

### Challenge 3: BMI Calculator with Health Advisory (Medium)

#### Problem Statement:

A health clinic computes BMI (Body Mass Index) for patients.

Given weight in kg and height in meters, calculate  $BMI = \text{weight} / \text{height}^2$ .

Classify the result:

- Below 18.5: "Underweight"
- 18.5 to 24.9: "Normal"
- 25.0 to 29.9: "Overweight"
- 30.0 and above: "Obese"

Print the BMI (rounded to 2 decimal places) and the classification.

#### Examples:

- Input: weight=70, height=1.75.  $BMI = 70 / 1.75^2 = 22.86$ . Classification: Normal.
- Input: weight=110, height=1.70.  $BMI = 110 / 1.70^2 = 38.06$ . Classification: Obese.

#### Test Cases:

| # | Weight (kg) | Height (m) | BMI   | Classification | Notes              |
|---|-------------|------------|-------|----------------|--------------------|
| 1 | 50          | 1.70       | 17.30 | Underweight    | Low BMI            |
| 2 | 70          | 1.75       | 22.86 | Normal         | Normal range       |
| 3 | 85          | 1.75       | 27.76 | Overweight     | Overweight range   |
| 4 | 110         | 1.70       | 38.06 | Obese          | Obese range        |
| 5 | 60          | 1.80       | 18.52 | Normal         | Boundary near 18.5 |



# CDP 402 - Conditionals

## KEY CONCEPTS

### Opening

In Block 401, you learned to compute values and print results. Every program ran straight through, top to bottom. But real programs make decisions. Should the customer get a discount? Is the password correct? Is the student eligible for a scholarship? That is what conditionals do. They let your program choose different paths based on conditions.

### if, elif, else

The basic structure is `if`, `elif`, and `else`.

```
marks = 75

if marks >= 90:
    print("A+")
elif marks >= 80:
    print("A")
elif marks >= 70:
    print("B")
elif marks >= 60:
    print("C")
elif marks >= 50:
    print("D")
else:
    print("F")
```

Three rules to remember. First, every `if`, `elif`, and `else` line ends with a colon. Second, the code inside must be indented by 4 spaces. Third, Python checks conditions top to bottom and executes only the FIRST one that is true. If marks is 75, Python checks `75 >= 90` (false), then `75 >= 80` (false), then `75 >= 70` (true), prints 'B', and skips everything below.

`elif` is Python's version of else-if. You can have as many `elif` blocks as you need.

### Nested if

You can put an `if` inside another `if`. This is called nesting.

```
age = 25
has_license = True

if age >= 18:
    if has_license:
        print("Can drive")
    else:
        print("Need a license first")
else:
    print("Too young to drive")
```

Notice the indentation. The inner `if-else` is indented inside the outer `if`. Every level of nesting adds 4 more spaces. If you get the indentation wrong, the logic changes. Python uses indentation to determine which code belongs to which condition.



## Comparison Chaining

Python has a feature that Java and C do not have: comparison chaining.

```
# Java/C way:
if age >= 18 and age <= 60:
    print("Working age")

# Python way (cleaner):
if 18 <= age <= 60:
    print("Working age")
```

Both do the same thing, but the Python way reads more naturally, like mathematics. You can chain any number of comparisons: `1 <= x <= 10`, `a < b < c`, `0 <= score <= 100`. Use this whenever you are checking if a value falls within a range.

## Ternary Expression

Python has a one-line conditional expression.

```
result = "Pass" if marks >= 50 else "Fail"
```

Read it as: 'result is Pass IF marks >= 50, ELSE Fail.' This is useful when you want to assign a value based on a simple condition. Do not use it for complex logic, only for simple either-or assignments.

## The `in` Keyword for Membership

The `in` keyword checks if a value exists in a collection.

```
if char in "aeiou":
    print("Vowel")

if day in [1, 2, 3, 4]:
    print("Weekday (Mon-Thu)")
```

This is much cleaner than writing `if char == 'a' or char == 'e' or char == 'i' or char == 'o' or char == 'u'`. You will use `in` constantly in Python.

## Logical Operators

Python uses words for logical operators, not symbols.

```
# Python: and, or, not
if age >= 18 and has_id:
    print("Allowed")

if not is_blocked:
    print("Access granted")
```

Java and C use `&&`, `||`, `!`. Python uses `and`, `or`, `not`. The logic is the same, the syntax is different.

## Truthy and Falsy Values

In Python, every value has a 'truthiness'. Some values are considered False in a boolean context.

```
# These are all "falsy" (treated as False):
0, 0.0, "", None, [], {}, False

# Everything else is "truthy" (treated as True):
1, -1, "hello", [1, 2], True
```

This means you can write:



```
name = input("Enter name: ")
if name:
    print(f"Hello, {name}")
else:
    print("No name entered")
```

`if name:` is equivalent to `if name != ""`. An empty string is falsy, a non-empty string is truthy. This is a Python idiom you will see everywhere. Java does not have this, you would need to write `if (!name.isEmpty())` explicitly.

## IN-CLASS PROGRAMS

### Program 1: Student Grade Classifier (Easy)

#### Problem Statement:

A university assigns grades based on marks out of 100:

- 90-100: "A+"
- 80-89: "A"
- 70-79: "B"
- 60-69: "C"
- 50-59: "D"
- Below 50: "F"

Given a student's marks, print the grade. Also print:

- "Distinction" if the grade is A+ or A
- "Pass" if the grade is B, C, or D
- "Fail" if the grade is F

#### Examples:

- Input: 95. Output: Grade = A+, Distinction.
- Input: 72. Output: Grade = B, Pass.
- Input: 40. Output: Grade = F, Fail.

#### Test Cases:

| # | Input (Marks) | Grade | Status      | Notes                   |
|---|---------------|-------|-------------|-------------------------|
| 1 | 95            | A+    | Distinction | Top tier                |
| 2 | 85            | A     | Distinction | A grade                 |
| 3 | 72            | B     | Pass        | B grade                 |
| 4 | 55            | D     | Pass        | D grade, lowest passing |
| 5 | 40            | F     | Fail        | Below 50                |

### Program 2: ATM Withdrawal System (Medium)

#### Problem Statement:

An ATM machine processes withdrawal requests. Given:

- Current balance
- Withdrawal amount
- Account type (1 = Savings, 2 = Current)



The ATM applies these rules in order:

1. Maximum single withdrawal is Rs 50,000. If amount exceeds this, print "EXCEEDS MAXIMUM LIMIT".
2. Withdrawal amount must be a multiple of 100. If not, print "NOT A MULTIPLE OF 100".
3. For Savings accounts: after withdrawal, the balance must remain at least Rs 500. If not, print "INSUFFICIENT BALANCE".
4. For Current accounts: no minimum balance, but withdrawals above Rs 25,000 incur a Rs 50 fee.
5. If all checks pass, print "SUCCESS" and the new balance.

#### Examples:

- Input: balance=10000, amount=5000, type=1 (Savings). All checks pass. New balance = 5000. Output: "SUCCESS", balance = 5000.
- Input: balance=10000, amount=9600, type=1 (Savings).  $10000 - 9600 = 400 < 500$ . Output: "INSUFFICIENT BALANCE".
- Input: balance=50000, amount=30000, type=2 (Current). Withdrawal > 25000, fee = Rs 50. New balance =  $50000 - 30000 - 50 = 19950$ . Output: "SUCCESS", balance = 19950.

#### Test Cases:

| # | Balance | Amount | Type        | Result                | New Balance | Notes                          |
|---|---------|--------|-------------|-----------------------|-------------|--------------------------------|
| 1 | 10000   | 5000   | 1 (Savings) | SUCCESS               | 5000        | Valid savings withdrawal       |
| 2 | 10000   | 9600   | 1 (Savings) | INSUFFICIENT BALANCE  | 10000       | Would go below 500             |
| 3 | 10000   | 5050   | 1 (Savings) | NOT A MULTIPLE OF 100 | 10000       | Not a multiple of 100          |
| 4 | 50000   | 30000  | 2 (Current) | SUCCESS               | 19950       | Current, Rs 50 fee for > 25000 |
| 5 | 10000   | 60000  | 1 (Savings) | EXCEEDS MAXIMUM LIMIT | 10000       | Exceeds 50000 maximum          |

### Program 3: Movie Ticket Pricing Engine (Medium)

#### Problem Statement:

A cinema chain dynamically prices tickets based on multiple factors. Given:

- Day of week (1-7, where 1 = Monday)
- Show time (hour in 24-hour format, e.g., 14 for 2 PM)
- Customer age
- Seat type (1 = Regular, 2 = Premium, 3 = Recliner)

Pricing rules (applied in this order):

1. Base price: Rs 200
2. Weekday discount (Monday to Thursday, days 1-4): 20% off base price



- Age discount (below 12 or above 60): 30% off the price after weekday discount
- Seat surcharge: Regular +Rs 0, Premium +Rs 100, Recliner +Rs 250
- GST: If price after surcharge exceeds Rs 500, apply 18% GST. Otherwise, apply 12% GST.

Print the itemized breakdown and final price.

#### Examples:

- Input: day=2 (Tue), time=18, age=25, seat=1 (Regular). Base=200. Weekday discount=40. After=160. No age discount. Surcharge=0. Subtotal=160. 12% GST=19.20. Final=179.20.
- Input: day=6 (Sat), time=20, age=10, seat=3 (Recliner). Base=200. No weekday discount. After=200. Age discount=60. After=140. Surcharge=250. Subtotal=390. 12% GST=46.80. Final=436.80.

#### Test Cases:

| # | Day     | Time | Age | Seat         | Weekday Disc | Age Disc | Surcharge | GST   | Final  | Notes                  |
|---|---------|------|-----|--------------|--------------|----------|-----------|-------|--------|------------------------|
| 1 | 2 (Tue) | 18   | 25  | 1 (Regular)  | 40.00        | 0.00     | 0         | 19.20 | 179.20 | Weekday adult regular  |
| 2 | 6 (Sat) | 20   | 10  | 3 (Recliner) | 0.00         | 60.00    | 250       | 46.80 | 436.80 | Weekend child recliner |
| 3 | 1 (Mon) | 10   | 65  | 2 (Premium)  | 40.00        | 48.00    | 100       | 25.44 | 237.44 | Weekday senior premium |
| 4 | 7 (Sun) | 20   | 30  | 3 (Recliner) | 0.00         | 0.00     | 250       | 54.00 | 504.00 | Weekend adult recliner |
| 5 | 3 (Wed) | 14   | 8   | 1 (Regular)  | 40.00        | 48.00    | 0         | 13.44 | 125.44 | Weekday child regular  |

#### Program 4: Loan Eligibility Checker (Hard)

##### Problem Statement:

A bank evaluates loan applications with the following rules:

Given:

- Applicant age
- Annual income
- Credit score (300-900)
- Number of existing loans
- Employment type (1 = Salaried, 2 = Self-employed)

Eligibility rules:

- Age must be between 21 and 60 (inclusive).
- For Salaried applicants: income  $\geq$  Rs 3,00,000 AND credit score  $\geq$  650 AND existing loans  $\leq$  2.



3. For Self-employed applicants: income  $\geq$  Rs 5,00,000 AND credit score  $\geq$  700 AND existing loans  $\leq$  1.
4. Special rule: If credit score  $\geq$  800, the income requirement drops by 20% for both categories (Rs 2,40,000 for salaried, Rs 4,00,000 for self-employed).

If eligible, print "ELIGIBLE" with maximum loan amount (3x income for salaried, 2x for self-employed).

If not eligible, print "NOT ELIGIBLE" with all the specific reasons.

#### Examples:

- Input: age=30, income=500000, credit=700, loans=1, type=1 (Salaried). All criteria met. Max loan =  $500000 \times 3 = 1500000$ . Output: "ELIGIBLE, Max loan: 1500000".
- Input: age=19, income=500000, credit=700, loans=1, type=1. Age < 21. Output: "NOT ELIGIBLE: Age must be between 21 and 60".
- Input: age=30, income=250000, credit=850, loans=1, type=1. Credit  $\geq$  800, so income req drops to 240000.  $250000 \geq 240000$ . All other criteria met. Output: "ELIGIBLE, Max loan: 750000".

#### Test Cases:

| # | Age | Income | Credit | Loans | Type         | Result       | Max Loan / Reasons    | Notes                          |
|---|-----|--------|--------|-------|--------------|--------------|-----------------------|--------------------------------|
| 1 | 30  | 500000 | 700    | 1     | 1 (Salaried) | ELIGIBLE     | 1500000               | All criteria met               |
| 2 | 25  | 600000 | 750    | 0     | 2 (Self-emp) | ELIGIBLE     | 1200000               | Self-employed, all met         |
| 3 | 30  | 250000 | 850    | 1     | 1 (Salaried) | ELIGIBLE     | 750000                | High credit reduces income req |
| 4 | 19  | 500000 | 700    | 1     | 1 (Salaried) | NOT ELIGIBLE | Age out of range      | Age too young                  |
| 5 | 30  | 200000 | 600    | 3     | 1 (Salaried) | NOT ELIGIBLE | Credit, income, loans | Multiple failures              |

### Program 5: Vowel or Consonant Checker (Easy)

#### Problem Statement:

A text processing tool classifies characters. Given a single character as input, determine if it is:

- A vowel (a, e, i, o, u, both uppercase and lowercase)
- A consonant (any letter that is not a vowel)
- A digit (0-9)
- A special character (anything else)

Print the classification.



### Examples:

- Input: "a". Output: "Vowel".
- Input: "B". Output: "Consonant".
- Input: "5". Output: "Digit".
- Input: "@". Output: "Special Character".

### Test Cases:

| # | Input | Classification    | Notes               |
|---|-------|-------------------|---------------------|
| 1 | a     | Vowel             | Lowercase vowel     |
| 2 | E     | Vowel             | Uppercase vowel     |
| 3 | b     | Consonant         | Lowercase consonant |
| 4 | 5     | Digit             | Digit character     |
| 5 | @     | Special Character | Non-alphanumeric    |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Shipping Cost Calculator (Medium)

#### Problem Statement:

An e-commerce company calculates shipping cost based on:

- Weight (kg)
- Distance (km)
- Delivery speed (1 = Standard, 2 = Express, 3 = Same-day)

Pricing rules:

- Standard: Rs 5/kg + Rs 2/km
- Express: Rs 8/kg + Rs 3/km + Rs 50 flat fee
- Same-day: Rs 12/kg + Rs 5/km + Rs 150 flat fee

Additional rules:

- If total weight exceeds 50 kg, add a 25% surcharge on the total cost.
- If distance exceeds 1000 km, same-day delivery is not available. Print "SAME-DAY NOT AVAILABLE".

#### Examples:

- Input: weight=10, distance=500, speed=1. Cost =  $10 \times 5 + 500 \times 2 = 1050$ . No surcharge. Output: 1050.00.
- Input: weight=60, distance=200, speed=1. Cost =  $60 \times 5 + 200 \times 2 = 700$ . Surcharge 25% = 875.00. Output: 875.00.
- Input: weight=10, distance=1500, speed=3. Output: "SAME-DAY NOT AVAILABLE".

### Test Cases:

| # | Weight | Distance | Speed        | Cost    | Notes                  |
|---|--------|----------|--------------|---------|------------------------|
| 1 | 10     | 500      | 1 (Standard) | 1050.00 | Standard, no surcharge |



|   |    |      |              |         |                              |
|---|----|------|--------------|---------|------------------------------|
| 2 | 10 | 500  | 2 (Express)  | 1630.00 | Express with flat fee        |
| 3 | 10 | 500  | 3 (Same-day) | 2770.00 | Same-day, distance <= 1000   |
| 4 | 60 | 200  | 1 (Standard) | 875.00  | Heavy package, 25% surcharge |
| 5 | 10 | 1500 | 3 (Same-day) | N/A     | Same-day not available       |

## Challenge 2: Income Tax Calculator (Hard)

### Problem Statement:

Calculate income tax based on the Indian new tax regime slabs for FY 2024-25:

- 0 to 3,00,000: Nil
- 3,00,001 to 7,00,000: 5%
- 7,00,001 to 10,00,000: 10%
- 10,00,001 to 12,00,000: 15%
- 12,00,001 to 15,00,000: 20%
- Above 15,00,000: 30%

Tax is calculated progressively (not a flat rate on entire income). After computing tax, add 4% Health and Education Cess on the tax amount.

Print: taxable income, tax before cess, cess amount, and total tax payable.

### Examples:

- Input: 500000. Tax: first 300000 at 0% = 0, next 200000 at 5% = 10000. Tax = 10000. Cess = 400. Total = 10400.
- Input: 1000000. Tax:  $300000 \times 0\% + 400000 \times 5\% + 300000 \times 10\% = 0 + 20000 + 30000 = 50000$ . Cess = 2000. Total = 52000.

### Test Cases:

| # | Income  | Tax Before Cess | Cess (4%) | Total Tax | Notes            |
|---|---------|-----------------|-----------|-----------|------------------|
| 1 | 250000  | 0.00            | 0.00      | 0.00      | Below 3L, no tax |
| 2 | 500000  | 10000.00        | 400.00    | 10400.00  | Two slabs        |
| 3 | 1000000 | 50000.00        | 2000.00   | 52000.00  | Three slabs      |
| 4 | 1500000 | 140000.00       | 5600.00   | 145600.00 | Five slabs       |
| 5 | 2000000 | 290000.00       | 11600.00  | 301600.00 | All slabs        |

## Challenge 3: Triangle Classifier (Medium)

### Problem Statement:

Given three side lengths of a triangle, determine:

1. Whether the sides form a valid triangle (the sum of any two sides must be greater than the third side).





2. If valid, classify as Equilateral (all sides equal), Isosceles (exactly two sides equal), or Scalene (no sides equal).
3. If valid, also check if it is a right triangle using the Pythagorean theorem ( $a^2 + b^2 = c^2$ , where  $c$  is the longest side).

#### Examples:

- Input: 3, 4, 5. Valid. Scalene. Right Triangle ( $3^2 + 4^2 = 5^2$ ).
- Input: 5, 5, 5. Valid. Equilateral. Not a Right Triangle.
- Input: 1, 2, 10. Invalid ( $1 + 2 < 10$ ).

#### Test Cases:

| # | Side A | Side B | Side C | Valid | Classification | Right Triangle | Notes              |
|---|--------|--------|--------|-------|----------------|----------------|--------------------|
| 1 | 5      | 5      | 5      | Yes   | Equilateral    | No             | All equal          |
| 2 | 5      | 5      | 3      | Yes   | Isosceles      | No             | Two equal          |
| 3 | 3      | 4      | 5      | Yes   | Scalene        | Yes            | Classic 3-4-5      |
| 4 | 7      | 8      | 9      | Yes   | Scalene        | No             | Scalene, not right |
| 5 | 1      | 2      | 10     | No    | N/A            | N/A            | Invalid triangle   |

# CDP 403 - Loops + Patterns + Number Problems

## KEY CONCEPTS

### Opening

In Block 402, you learned to make decisions with if-elif-else. But what if you need to repeat something? What if you need to add up 100 numbers, check every character in a string, or print a pattern line by line? That is what loops do. They let your program repeat actions.

### for Loop with range()

The `for` loop in Python works with the `range()` function.

```
# Print numbers 0 to 4
for i in range(5):
    print(i)
# Output: 0 1 2 3 4 (each on a new line)
```

`range(5)` generates the numbers 0, 1, 2, 3, 4. Not 5. The stop value is excluded. This is a common source of confusion. If you want 1 to 5, use `range(1, 6)`.

There are three forms of `range()`:

```
range(n)           # 0 to n-1
range(start, stop)  # start to stop-1
range(start, stop, step) # start to stop-1, stepping by step

# Examples:
range(5)           # 0, 1, 2, 3, 4
range(1, 6)        # 1, 2, 3, 4, 5
range(0, 10, 2)     # 0, 2, 4, 6, 8
range(10, 0, -1)    # 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
```

Negative step counts downward. `range(10, 0, -1)` gives 10 down to 1. This is useful for reverse iteration.

### while Loop

The `while` loop repeats as long as a condition is true.

```
count = 0
while count < 5:
    print(count)
    count += 1
# Output: 0 1 2 3 4
```

Use `for` when you know how many times to repeat. Use `while` when you repeat until a condition changes.

Python has no `do-while` loop like Java or C. If you need 'execute at least once, then check', use this pattern:

```
while True:
    value = int(input("Enter a positive number: "))
    if value > 0:
        break
    print("Invalid! Try again.")
```



## break and continue

`break` exits the loop immediately. `continue` skips the rest of the current iteration and moves to the next one.

```
# break example: find first even number
for i in range(1, 100):
    if i % 2 == 0:
        print(f"First even: {i}")
        break

# continue example: skip odd numbers
for i in range(10):
    if i % 2 != 0:
        continue
    print(i)  # prints 0, 2, 4, 6, 8
```

## Loop else Clause

Python has a unique feature: `else` on loops. The `else` block runs only if the loop completes without `break`.

```
for i in range(2, n):
    if n % i == 0:
        print("Not prime")
        break
else:
    print("Prime")  # runs only if no break occurred
```

This is a Python curiosity. You will not need it in problems, but know it exists in case you see it in code.

## Nested Loops

A loop inside a loop. The inner loop runs completely for each iteration of the outer loop.

```
for row in range(3):
    for col in range(4):
        print("*", end=" ")
    print()  # new line after each row
```

Output:

```
****
****
****
```

The critical concept for nested loops:

```
Outer loop = rows
Inner loop = columns
What you print depends on the relationship between row and column counters
```

## print() with end Parameter

By default, `print()` adds a newline at the end. The `end` parameter changes this.

```
print("A", end="")  # no newline
print("B", end=" ")  # space instead of newline
print("C")          # default newline

# Output: AB C
```



For pattern printing, you will use `print("\*", end="\n")` to print stars on the same line, and `print()` with no arguments to move to the next line.

## The Pattern Printing Mental Model

Every pattern follows the same structure:

```
for row in range(1, n + 1):
    # print some spaces (depends on row)
    # print some characters (depends on row)
    # move to next line
```

For a right-aligned staircase with N=4:

```
Row 1: 3 spaces, 1 star      (n-row spaces, row stars)
Row 2: 2 spaces, 2 stars
Row 3: 1 space, 3 stars
Row 4: 0 spaces, 4 stars
```

The formula: `n - row` spaces, then `row` stars. Once you see this relationship, every pattern follows the same thinking.

## IN-CLASS PROGRAMS

### Program 1: Warehouse Stock Counter (Easy)

#### Problem Statement:

A warehouse receives deliveries daily. Given N deliveries (each a positive integer representing units), calculate:

1. Total stock (sum of all deliveries)
2. Average delivery size

Use a for loop to read N integers and accumulate the sum.

#### Examples:

- Input: N=3, deliveries = [50, 30, 20]. Total = 100, Average = 33.33.
- Input: N=5, deliveries = [10, 20, 30, 40, 50]. Total = 150, Average = 30.00.

#### Test Cases:

| # | N | Deliveries         | Total | Average | Notes           |
|---|---|--------------------|-------|---------|-----------------|
| 1 | 3 | 50, 30, 20         | 100   | 33.33   | Standard case   |
| 2 | 1 | 100                | 100   | 100.00  | Single delivery |
| 3 | 5 | 10, 20, 30, 40, 50 | 150   | 30.00   | Five deliveries |
| 4 | 3 | 0, 0, 0            | 0     | 0.00    | All zeros       |
| 5 | 2 | 999, 1             | 1000  | 500.00  | Two deliveries  |

### Program 2: PIN Validation System (Medium)

#### Problem Statement:

A banking app allows the user 3 attempts to enter the correct 4-digit PIN. The correct PIN is 1234 (hardcoded).

On each wrong attempt, print "INCORRECT, X attempts remaining" (where X is attempts left).



If all 3 attempts fail, print "ACCOUNT LOCKED".

If correct, print "ACCESS GRANTED" and stop.

### Examples:

- Input: 1234 (first try). Output: "ACCESS GRANTED".
- Input: 0000, 1111, 2222. Output: "INCORRECT, 2 attempts remaining", "INCORRECT, 1 attempts remaining", "ACCOUNT LOCKED".
- Input: 0000, 1234. Output: "INCORRECT, 2 attempts remaining", "ACCESS GRANTED".

### Test Cases:

| # | Attempts (in order)     | Final Output   | Notes                    |
|---|-------------------------|----------------|--------------------------|
| 1 | 1234                    | ACCESS GRANTED | Correct on first try     |
| 2 | 0000, 1234              | ACCESS GRANTED | Correct on second try    |
| 3 | 0000, 1111, 2222        | ACCOUNT LOCKED | All three wrong          |
| 4 | 0000, 1111, 1234        | ACCESS GRANTED | Correct on last try      |
| 5 | 5678 (if PIN were 5678) | ACCESS GRANTED | Different PIN, first try |

### Program 3: Staircase Pattern Printer (Medium)

#### Problem Statement:

A tile manufacturing company prints staircase-shaped tile arrangements for display. Given N (number of rows), print a right-aligned staircase pattern of \* characters.

Row 1 has 1 star (preceded by N-1 spaces).

Row 2 has 2 stars (preceded by N-2 spaces).

Row N has N stars (no preceding spaces).

#### Examples:

- Input: N=4.

```
*
 **
 ***
****
```

- Input: N=3.

```
*
 **
***
```

### Test Cases:

| # | N | Output (each line)                    | Notes         |
|---|---|---------------------------------------|---------------|
| 1 | 4 | " *", " **", " ***", "****"           | Standard case |
| 2 | 1 | "**"                                  | Single row    |
| 3 | 3 | " *", " **", " ***"                   | Three rows    |
| 4 | 5 | " *", " **", " ***", " ****", "*****" | Five rows     |
| 5 | 2 | " *", " **"                           | Two rows      |

## Program 4: Armstrong Number Range Finder (Hard)

### Problem Statement:

A mathematics tutoring app needs to find all Armstrong numbers in a given range [L, R].

An Armstrong number (also called a narcissistic number) of N digits is one where the sum of each digit raised to the power N equals the number itself.

- 153 is Armstrong:  $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$ .
- 370 is Armstrong:  $3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370$ .

Given L and R, print all Armstrong numbers in the range [L, R]. If none exist, print "NO ARMSTRONG NUMBERS FOUND".

### Examples:

- Input: L=100, R=200. Output: 153.
- Input: L=1, R=500. Output: 1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407.
- Input: L=500, R=600. Output: NO ARMSTRONG NUMBERS FOUND.

### Test Cases:

| # | L   | R   | Armstrong Numbers                             | Notes                    |
|---|-----|-----|-----------------------------------------------|--------------------------|
| 1 | 1   | 500 | 1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407 | Wide range               |
| 2 | 100 | 200 | 153                                           | Only one in range        |
| 3 | 150 | 160 | 153                                           | Tight range              |
| 4 | 500 | 600 | None                                          | No Armstrong numbers     |
| 5 | 0   | 0   | 0                                             | Edge: single number zero |

## Program 5: Concatenation of Array (LeetCode 1929, Easy)

### Problem Statement:

Given an integer array (list) `nums` of length `n`, create an array `ans` of length `2n` where:

- `ans[i] == nums[i]` for `0 <= i < n`
- `ans[i + n] == nums[i]` for `0 <= i < n`

Essentially, concatenate `nums` with itself.

### Examples:

- Input: `nums = [1, 2, 1]`. Output: `[1, 2, 1, 1, 2, 1]`.
- Input: `nums = [1, 3, 2, 1]`. Output: `[1, 3, 2, 1, 1, 3, 2, 1]`.

### Test Cases:

| # | Input        | Output                   | Notes          |
|---|--------------|--------------------------|----------------|
| 1 | [1, 2, 1]    | [1, 2, 1, 1, 2, 1]       | LC Example 1   |
| 2 | [1, 3, 2, 1] | [1, 3, 2, 1, 1, 3, 2, 1] | LC Example 2   |
| 3 | [1]          | [1, 1]                   | Single element |



|   |           |                    |                 |
|---|-----------|--------------------|-----------------|
| 4 | []        | []                 | Empty list      |
| 5 | [5, 5, 5] | [5, 5, 5, 5, 5, 5] | All same values |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Digit Sum Reducer (Medium)

#### Problem Statement:

A banking system computes a "check digit" by repeatedly summing the digits of a number until a single digit remains.

For example: 9875 -> 9+8+7+5 = 29 -> 2+9 = 11 -> 1+1 = 2. The check digit is 2, and it took 3 iterations.

Given a positive integer, print the single digit result and the number of iterations.

#### Examples:

- Input: 9875. Output: digit = 2, iterations = 3.
- Input: 5. Output: digit = 5, iterations = 0.
- Input: 99. Output: digit = 9, iterations = 2 (99 -> 18 -> 9).

#### Test Cases:

| # | Input  | Final Digit | Iterations | Notes                 |
|---|--------|-------------|------------|-----------------------|
| 1 | 9875   | 2           | 3          | 9875 -> 29 -> 11 -> 2 |
| 2 | 5      | 5           | 0          | Already single digit  |
| 3 | 99     | 9           | 2          | 99 -> 18 -> 9         |
| 4 | 10     | 1           | 1          | 10 -> 1               |
| 5 | 999999 | 9           | 2          | 999999 -> 54 -> 9     |

### Challenge 2: Diamond Pattern (Medium)

#### Problem Statement:

Given N (must be an odd number), print a diamond pattern of stars.

- Top half: 1, 3, 5, ... N stars, centered.
- Bottom half: N-2, N-4, ... 1 stars, centered.
- If N is even, print "INVALID INPUT".

#### Examples:

- Input: N=5.

```
*
 ***
*****
 ***
 *
```

- Input: N=4. Output: INVALID INPUT.

### Test Cases:

| # | N | Output Description       | Notes           |
|---|---|--------------------------|-----------------|
| 1 | 5 | Diamond with max width 5 | Standard case   |
| 2 | 3 | Diamond with max width 3 | Small diamond   |
| 3 | 1 | Single star              | Minimum diamond |
| 4 | 4 | INVALID INPUT            | Even number     |
| 5 | 7 | Diamond with max width 7 | Larger diamond  |

### Challenge 3: Happy Number (LeetCode 202, Medium)

#### Problem Statement:

A happy number is defined by the following process: Starting with any positive integer, replace the number by the sum of the squares of its digits, and repeat. If this process reaches 1, the number is happy. If it enters a cycle that does not include 1, the number is not happy.

Determine if a given number  $n$  is happy.

#### Examples:

- Input:  $n=19$ .  $1^2 + 9^2 = 82$ .  $8^2 + 2^2 = 68$ .  $6^2 + 8^2 = 100$ .  $1^2 + 0^2 + 0^2 = 1$ . Happy!
- Input:  $n=2$ . Enters a cycle ( $2 \rightarrow 4 \rightarrow 16 \rightarrow 37 \rightarrow 58 \rightarrow 89 \rightarrow 145 \rightarrow 42 \rightarrow 20 \rightarrow 4 \dots$ ). Not happy.

### Test Cases:

| # | Input | Result    | Notes           |
|---|-------|-----------|-----------------|
| 1 | 19    | Happy     | Classic example |
| 2 | 2     | Not Happy | Enters cycle    |
| 3 | 1     | Happy     | Already 1       |
| 4 | 7     | Happy     | 7 is happy      |
| 5 | 4     | Not Happy | Enters cycle    |



# CDP 404 - Functions

## KEY CONCEPTS

### Opening

In the first three blocks, you wrote all your code in the main body of the program. Every time you needed the same calculation, you copied and pasted the code. What if you need the same logic in 10 different places? What if you find a bug and have to fix it in all 10 places? That is why functions exist. A function is a reusable block of code with a name. You write it once, call it many times.

### Defining and Calling Functions

In Python, you define a function with the `def` keyword.

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Alice")    # Output: Hello, Alice!  
greet("Bob")      # Output: Hello, Bob!
```

Three parts: `def` keyword, function name, parameters in parentheses. The body is indented, just like `if` and `for` blocks. You call the function by writing its name followed by arguments in parentheses.

Functions can return values:

```
def square(x):  
    return x ** 2  
  
result = square(5)  
print(result)    # 25
```

The `return` statement sends a value back to the caller. Without `return`, the function returns `None` by default.

### Multiple Return Values

Python can return multiple values from a function. This is something Java and C cannot do directly.

```
def find_min_max(numbers):  
    return min(numbers), max(numbers)  
  
lowest, highest = find_min_max([3, 7, 1, 9, 4])  
print(f"Min: {lowest}, Max: {highest}")    # Min: 1, Max: 9
```

The function returns a tuple (1, 9). The assignment `lowest, highest = ...` unpacks it into two variables. This is called tuple unpacking. You will use this pattern constantly in Python.

### Default Arguments

You can give parameters default values.

```
def greet(name, greeting="Hello"):  
    print(f"{greeting}, {name}!")  
  
greet("Alice")          # Hello, Alice!  
greet("Bob", "Welcome") # Welcome, Bob!
```



If the caller does not provide a value for `greeting`, it defaults to `\\"Hello\\"`. Default arguments must come AFTER non-default arguments in the function definition. `def greet(greeting=\\\"Hello\\\", name)` would be a syntax error.

## Keyword Arguments

When calling a function, you can specify arguments by name.

```
greet(greeting="Hi", name="Alice") # Hi, Alice!
```

This lets you pass arguments in any order. Useful when a function has many parameters.

## Scope: Local vs Global

Variables created inside a function are local. They do not exist outside.

```
def calculate():
    total = 100 # local variable
    return total

print(total) # NameError! total does not exist here
```

The variable `total` lives only inside `calculate()`. This is called scope. Each function has its own scope.

If you need to access a variable from outside the function, pass it as a parameter or return it. Do NOT use the `global` keyword. It works, but it makes code harder to understand and debug. Pass values in, return values out.

## Docstrings

A docstring is a string on the first line inside a function that describes what it does.

```
def is_prime(n):
    """Return True if n is a prime number, False otherwise."""
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Good practice, not required by Python. But in interviews and team projects, docstrings are expected.

## \*args and \*\*kwargs

`*args` collects extra positional arguments as a tuple. `**kwargs` collects extra keyword arguments as a dict.

```
def total(*args):
    return sum(args)

print(total(1, 2, 3)) # 6
print(total(10, 20)) # 30
```

You will see `*args` and `**kwargs` in library documentation and code examples. You do not need to use them in your solutions for this track. Know what they mean when you see them.



## Functions as First-Class Objects

In Python, functions are objects. You can store them in variables, pass them to other functions, and return them from functions. This is called 'first-class functions.' We will not use this feature in this track, but know it exists. It is what makes Python powerful for advanced programming.

### Contrast with Java

In Java, you needed `public static int methodName(int x)` with types everywhere. In Python, `def methodName(x)` is all you need. No return type declaration, no parameter type declaration. This is faster to write, but it means YOU must be careful about what you pass and return. Python will not catch type mistakes at compile time because Python does not have a compile step.

## IN-CLASS PROGRAMS

### Program 1: Unit Converter Toolkit (Easy)

#### Problem Statement:

A measurement conversion app provides three functions:

- `km_to_miles(km)`: returns `km * 0.621371`
- `kg_to_pounds(kg)`: returns `kg * 2.20462`
- `celsius_to_fahrenheit(c)`: returns `c * 9/5 + 32`

Each function takes one parameter and returns the converted value.

In main, read a value and a conversion type (1 = km to miles, 2 = kg to pounds, 3 = C to F). Call the appropriate function and print the result with 2 decimal places.

#### Examples:

- Input: value=10, type=1. Output: 6.21 miles.
- Input: value=5, type=2. Output: 11.02 pounds.
- Input: value=100, type=3. Output: 212.00 F.

#### Test Cases:

| # | Value | Type             | Result | Notes                  |
|---|-------|------------------|--------|------------------------|
| 1 | 10.0  | 1 (km to miles)  | 6.21   | Standard conversion    |
| 2 | 5.0   | 2 (kg to pounds) | 11.02  | Weight conversion      |
| 3 | 100.0 | 3 (C to F)       | 212.00 | Boiling point          |
| 4 | 0.0   | 1 (km to miles)  | 0.00   | Zero input             |
| 5 | -40.0 | 3 (C to F)       | -40.00 | -40 is same in C and F |

### Program 2: Student Report Card Generator (Medium)

#### Problem Statement:

Design a grade processing system with the following functions:

- `calculate_average(marks_list)`: takes a list of integers, returns the average as a float.
- `determine_grade(average)`: takes an average, returns a grade string (A+:  $\geq 90$ , A:  $\geq 80$ , B:  $\geq 70$ , C:  $\geq 60$ , D:  $\geq 50$ , F:  $< 50$ ).
- `is_passed(grade)`: takes a grade string, returns True if not "F", False otherwise.



- `generate_report(name, marks_list)`: calls the above three functions internally and prints a formatted report card.

In main, read student name and 5 subject marks, call `generate_report`.

### Examples:

- Input: name="Alice", marks=[90, 85, 92, 88, 95]. Average=90.0, Grade=A+, Passed=True.
- Input: name="Charlie", marks=[30, 40, 35, 45, 20]. Average=34.0, Grade=F, Passed=False.

### Test Cases:

| # | Name    | Marks                   | Average | Grade | Passed | Notes           |
|---|---------|-------------------------|---------|-------|--------|-----------------|
| 1 | Alice   | 90, 85, 92, 88, 95      | 90.0    | A+    | True   | High scorer     |
| 2 | Bob     | 50, 55, 48, 60, 52      | 53.0    | D     | True   | Low but passing |
| 3 | Charlie | 30, 40, 35, 45, 20      | 34.0    | F     | False  | Failing         |
| 4 | Diana   | 70, 75, 80, 65, 70      | 72.0    | B     | True   | B grade         |
| 5 | Eve     | 100, 100, 100, 100, 100 | 100.0   | A+    | True   | Perfect score   |

### Program 3: Prime Number Toolkit (Medium)

#### Problem Statement:

A mathematics library provides three functions:

- `is_prime(n)`: returns True if n is prime, False otherwise.
- `primes_in_range(start, end)`: returns a list of all primes in the range [start, end] by calling `is_prime` for each number.
- `prime_factorization(n)`: returns a list of prime factors (e.g., 12 returns [2, 2, 3]).

In main, demonstrate all three functions.

### Examples:

- `is_prime(17)` returns True. `is_prime(4)` returns False.
- `primes_in_range(1, 20)` returns [2, 3, 5, 7, 11, 13, 17, 19].
- `prime_factorization(12)` returns [2, 2, 3].

### Test Cases:

| # | Function                         | Input   | Output                       | Notes          |
|---|----------------------------------|---------|------------------------------|----------------|
| 1 | <code>is_prime</code>            | 17      | True                         | Prime          |
| 2 | <code>is_prime</code>            | 4       | False                        | Not prime      |
| 3 | <code>primes_in_range</code>     | (1, 20) | [2, 3, 5, 7, 11, 13, 17, 19] | Standard range |
| 4 | <code>prime_factorization</code> | 12      | [2, 2, 3]                    | 12 = 2*2*3     |



|   |                     |     |              |                                   |
|---|---------------------|-----|--------------|-----------------------------------|
| 5 | prime_factorization | 100 | [2, 2, 5, 5] | $100 = 2 \cdot 2 \cdot 5 \cdot 5$ |
|---|---------------------|-----|--------------|-----------------------------------|

## Program 4: Password Strength Analyzer (Hard)

### Problem Statement:

A security system evaluates password strength using multiple validation functions:

- `has_min_length(password, min_len=8)`: returns True if password length  $\geq$  min\_len. Uses default argument.
- `has_uppercase(password)`: returns True if password contains at least one uppercase letter.
- `has_lowercase(password)`: returns True if password contains at least one lowercase letter.
- `has_digit(password)`: returns True if password contains at least one digit.
- `has_special_char(password, specials="!@#$%^&*")`: returns True if password contains at least one character from the specials string. Uses default argument.

Each function returns True/False.

A master function `analyze_password(password)` calls all five, counts how many pass (score 0-5), and returns a tuple: (strength\_label, score, failed\_checks) where:

- Score 1-2: "Weak"
- Score 3: "Medium"
- Score 4: "Strong"
- Score 5: "Very Strong"

`failed_checks` is a list of the names of checks that failed.

### Examples:

- Input: "Abc@1234". All 5 checks pass. Output: ("Very Strong", 5, []).
- Input: "abcdefgh". Length passes, lowercase passes. No uppercase, no digit, no special. Output: ("Weak", 2, ["Uppercase", "Digit", "Special character"]).

### Test Cases:

| # | Password | Strength    | Score | Failed Checks                       | Notes                   |
|---|----------|-------------|-------|-------------------------------------|-------------------------|
| 1 | Abc@1234 | Very Strong | 5     | None                                | All pass                |
| 2 | abcdefgh | Weak        | 2     | Uppercase, Digit, Special character | Only length + lowercase |
| 3 | Ab1      | Medium      | 3     | Minimum length, Special character   | Short but diverse       |
| 4 | ABCDEFGH | Weak        | 2     | Lowercase, Digit, Special character | Only length + uppercase |
| 5 | Ab@12345 | Very Strong | 5     | None                                | All pass, longer        |

## Program 5: Concatenation of Array + Build Array from Permutation (LeetCode 1929/1920, Medium)

### Problem Statement (LC 1929):

Given an integer list `nums` of length `n`, return a list `ans` of length `2n` where `ans = nums` concatenated with itself. Write as a function.

### Problem Statement (LC 1920):

Given a zero-based permutation `nums`, build a list `ans` where `ans[i] = nums[nums[i]]`. Write as a function.

### Examples:

- LC 1929: `get_concatenation([1, 2, 1])` returns `[1, 2, 1, 1, 2, 1]`.
- LC 1920: `build_array([0, 2, 1, 5, 3, 4])` returns `[0, 1, 2, 4, 5, 3]`.

### Test Cases:

| # | Problem | Input              | Output                   | Notes        |
|---|---------|--------------------|--------------------------|--------------|
| 1 | LC 1929 | [1, 2, 1]          | [1, 2, 1, 1, 2, 1]       | Standard     |
| 2 | LC 1929 | [1, 3, 2, 1]       | [1, 3, 2, 1, 1, 3, 2, 1] | Longer list  |
| 3 | LC 1920 | [0, 2, 1, 5, 3, 4] | [0, 1, 2, 4, 5, 3]       | LC Example 1 |
| 4 | LC 1920 | [5, 0, 1, 2, 3, 4] | [4, 5, 0, 1, 2, 3]       | LC Example 2 |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: GCD and LCM Calculator (Medium)

#### Problem Statement:

Write two functions:

- `gcd(a, b)`: computes the Greatest Common Divisor using Euclid's algorithm.
- While `b != 0`: set `a, b = b, a % b`. Return `a`.
- `lcm(a, b)`: computes the Least Common Multiple using the formula: `lcm = (a * b) // gcd(a, b)`.

Given two numbers, print their GCD and LCM.

#### Examples:

- Input: 12, 8. GCD = 4, LCM = 24.
- Input: 7, 13. GCD = 1, LCM = 91 (coprime).

### Test Cases:

| # | a   | b   | GCD | LCM | Notes           |
|---|-----|-----|-----|-----|-----------------|
| 1 | 12  | 8   | 4   | 24  | Standard case   |
| 2 | 15  | 25  | 5   | 75  | Common factor 5 |
| 3 | 7   | 13  | 1   | 91  | Coprime         |
| 4 | 100 | 100 | 100 | 100 | Same number     |
| 5 | 48  | 18  | 6   | 144 | GCD = 6         |



## Challenge 2: Statistical Calculator (Medium)

### Problem Statement:

Write four functions:

- `find_min(numbers)`: returns the minimum value. Do NOT use built-in `min()`.
- `find_max(numbers)`: returns the maximum value. Do NOT use built-in `max()`.
- `find_mean(numbers)`: returns the mean (average). Do NOT use built-in `sum()`.
- `find_median(numbers)`: returns the median (sort the list manually, handle even/odd length). Do NOT use built-in `sorted()`.

Given N numbers, print all four statistics.

### Examples:

- Input: [5, 3, 8, 1, 9]. Min=1, Max=9, Mean=5.20, Median=5.
- Input: [10, 20, 30, 40]. Min=10, Max=40, Mean=25.00, Median=25.0.

### Test Cases:

| # | Numbers             | Min | Max | Mean  | Median | Notes          |
|---|---------------------|-----|-----|-------|--------|----------------|
| 1 | 5, 3, 8, 1, 9       | 1   | 9   | 5.20  | 5      | Odd count      |
| 2 | 10, 20, 30, 40      | 10  | 40  | 25.00 | 25.0   | Even count     |
| 3 | 7                   | 7   | 7   | 7.00  | 7      | Single element |
| 4 | 1, 1, 1, 1          | 1   | 1   | 1.00  | 1.0    | All same       |
| 5 | 100, 50, 75, 25, 80 | 25  | 100 | 66.00 | 75     | Mixed          |

## Challenge 3: Power of Two (LeetCode 231, Medium)

### Problem Statement:

Given an integer n, return True if it is a power of two, False otherwise. An integer n is a power of two if there exists an integer x such that  $n == 2^x$ .

Write as a function: `def is_power_of_two(n):`

### Examples:

- Input: n=1. Output: True ( $2^0 = 1$ ).
- Input: n=16. Output: True ( $2^4$ ).
- Input: n=3. Output: False.

### Test Cases:

| # | Input | Output | Notes            |
|---|-------|--------|------------------|
| 1 | 1     | True   | $2^0$            |
| 2 | 16    | True   | $2^4$            |
| 3 | 3     | False  | Not a power of 2 |
| 4 | 0     | False  | Zero             |
| 5 | 1024  | True   | $2^{10}$         |



## CDP 405 - Lists (1D)

### KEY CONCEPTS

#### Opening

In Block 404, you learned to write functions that take values and return values. But so far, you have mostly worked with single values: one number, one string. What if you need to store 100 student marks? 1000 product prices? A list of names? You cannot create 100 separate variables. That is where lists come in. A list is a collection of values stored in a single variable.

#### List Creation and Basics

```
nums = [1, 2, 3, 4, 5]      # list with values
empty = []                 # empty list
zeros = [0] * 5            # [0, 0, 0, 0, 0]
names = ["Alice", "Bob"]   # list of strings
mixed = [1, "hello", True] # mixed types (allowed but rare)
```

Python lists are dynamic. They grow and shrink as needed. Unlike Java arrays or C arrays, you do not declare a fixed size. You can add and remove elements at any time.

#### Indexing and Slicing

```
nums = [10, 20, 30, 40, 50]

# Indexing
nums[0]      # 10 (first element)
nums[-1]     # 50 (last element)
nums[-2]     # 40 (second from last)

# Slicing
nums[1:3]    # [20, 30] (index 1 to 2, stop excluded)
nums[:3]     # [10, 20, 30] (first 3)
nums[2:]     # [30, 40, 50] (from index 2)
nums[::2]    # [10, 30, 50] (every other element)
nums[::-1]   # [50, 40, 30, 20, 10] (reversed!)
```

Slicing creates a NEW list. It does not modify the original. The `[::-1]` pattern to reverse a list is used everywhere in Python. Memorize it.

#### List Methods

```
nums = [3, 1, 4, 1, 5]

nums.append(9)      # [3, 1, 4, 1, 5, 9] add to end
nums.insert(0, 7)   # [7, 3, 1, 4, 1, 5, 9] insert at index
nums.remove(1)      # [7, 3, 4, 1, 5, 9] remove first occurrence
nums.pop()          # [7, 3, 4, 1, 5] remove and return last
nums.pop(0)         # [3, 4, 1, 5] remove and return at index

len(nums)           # 4
nums.sort()         # [1, 3, 4, 5] in-place sort
sorted(nums)        # returns new sorted list, original unchanged
nums.reverse()      # [5, 4, 3, 1] in-place reverse
nums.index(4)       # 1 (index of first occurrence)
nums.count(3)       # 1 (how many times 3 appears)
```





Two sorting options: `nums.sort()` modifies the original list. `sorted(nums)` creates a new sorted list and leaves the original untouched. Know the difference.

## Iteration

```
nums = [10, 20, 30]

# By value
for x in nums:
    print(x)          # 10, 20, 30

# By index
for i in range(len(nums)):
    print(i, nums[i]) # 0 10, 1 20, 2 30

# By index AND value (Pythonic)
for i, x in enumerate(nums):
    print(i, x)       # 0 10, 1 20, 2 30
```

Use `enumerate()` when you need both the index and the value. It is cleaner than `range(len(nums))`. Python developers prefer `enumerate()`.

## List Comprehensions

This is one of Python's most powerful features. A list comprehension builds a new list from an existing one in a single line.

```
# Without comprehension (loop):
squares = []
for x in range(10):
    squares.append(x ** 2)
# squares = [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# With comprehension (one line):
squares = [x ** 2 for x in range(10)]
```

Same result. The comprehension is shorter and more Pythonic.

You can add a condition:

```
# Only even numbers
evens = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

The pattern is: `[expression for variable in iterable if condition]`.

Loop version first. Comprehension second.

Both are correct. Loop version is what DSA expects.

In this track, you will always solve problems with explicit loops first. Then the comprehension version is shown as the Pythonic shorthand. For DSA, you must know the loop version because you will need to do things comprehensions cannot do (like break early or maintain state). But for Python interviews and production code, comprehensions are expected.

## Built-in Functions and Membership

```
nums = [5, 3, 8, 1]

sum(nums)    # 17
min(nums)    # 1
max(nums)    # 8
```



```
5 in nums    # True
9 in nums    # False
```

These are convenient shortcuts. But you must know how to compute sum, min, and max with a loop, because interviews may ask you to implement them without built-ins.

## IN-CLASS PROGRAMS

### Program 1: Inventory Stock Tracker (Easy)

#### Problem Statement:

A store maintains a list of product quantities. Given N products, read their quantities into a list. Find:

1. Total stock (sum of all quantities)
2. Product with highest stock (maximum value and its index)
3. Product with lowest stock (minimum value and its index)
4. Average stock

Implement WITHOUT using built-in `sum()`, `min()`, `max()`. Use explicit loops.

After solving, show the one-liner versions using built-ins.

#### Examples:

- Input: N=5, quantities = [50, 30, 80, 20, 60]. Total=240, Max=80 at index 2, Min=20 at index 3, Avg=48.0.
- Input: N=1, quantities = [100]. Total=100, Max=100 at index 0, Min=100 at index 0, Avg=100.0.

#### Test Cases:

| # | Quantities         | Total | Max (Index) | Min (Index) | Avg    | Notes                          |
|---|--------------------|-------|-------------|-------------|--------|--------------------------------|
| 1 | 50, 30, 80, 20, 60 | 240   | 80 (2)      | 20 (3)      | 48.00  | Standard case                  |
| 2 | 100                | 100   | 100 (0)     | 100 (0)     | 100.00 | Single product                 |
| 3 | 10, 10, 10         | 30    | 10 (0)      | 10 (0)      | 10.00  | All same                       |
| 4 | 5, 1, 3, 9, 2      | 20    | 9 (3)       | 1 (1)       | 4.00   | Min/max at different positions |
| 5 | 0, 0, 0, 0         | 0     | 0 (0)       | 0 (0)       | 0.00   | All zeros                      |

### Program 2: Student Marks Filter (Medium)

#### Problem Statement:

A university filters student marks for different purposes. Given a list of N marks:

1. Create a new list of only passing marks ( $\geq 50$ ) using a loop.
2. Create the same list using a list comprehension.
3. Find the count of students who scored strictly above the class average.

Print both versions of the filtered list (loop and comprehension) to show they are identical.



### Examples:

- Input: [80, 45, 60, 30, 90, 55]. Passing: [80, 60, 90, 55]. Average = 60.0. Above average (strictly > 60): 80, 90. Count = 2.
- Input: [90, 85, 95]. All passing. Average = 90.0. Above 90: 95. Count = 1.

### Test Cases:

| # | Marks                  | Passing          | Avg   | Above Avg Count | Notes                                |
|---|------------------------|------------------|-------|-----------------|--------------------------------------|
| 1 | 80, 45, 60, 30, 90, 55 | [80, 60, 90, 55] | 60.00 | 2               | Mixed marks, 60 is NOT above 60      |
| 2 | 90, 85, 95             | [90, 85, 95]     | 90.00 | 1               | All passing                          |
| 3 | 20, 30, 40             | []               | 30.00 | 1               | None passing, 40 is above avg        |
| 4 | 50                     | [50]             | 50.00 | 0               | Exactly 50, passes but not above avg |
| 5 | 100, 0, 100, 0, 100    | [100, 100, 100]  | 60.00 | 3               | Extremes                             |

### Program 3: Vote Counter (Medium)

#### Problem Statement:

An election system counts votes. Given N votes, where each vote is an integer representing a candidate ID (from 1 to K), count the votes for each candidate.

Use a frequency list of size K+1 (index = candidate ID, value = vote count).

Find the winner (candidate with the most votes). Tie-breaking rule: if two candidates have equal votes, the one with the smaller ID wins.

#### Examples:

- Input: N=6, K=3, votes = [1, 2, 3, 1, 2, 1]. Frequencies: [0, 3, 2, 1]. Winner: Candidate 1 (3 votes).
- Input: N=4, K=2, votes = [1, 2, 1, 2]. Frequencies: [0, 2, 2]. Tie. Winner: Candidate 1 (smaller ID).

### Test Cases:

| # | Votes            | K | Frequencies (1 to K) | Winner | Notes                |
|---|------------------|---|----------------------|--------|----------------------|
| 1 | 1, 2, 3, 1, 2, 1 | 3 | [3, 2, 1]            | 1      | Clear winner         |
| 2 | 3, 3, 3, 2, 2, 1 | 3 | [1, 2, 3]            | 3      | Candidate 3 wins     |
| 3 | 1, 2, 1, 2       | 2 | [2, 2]               | 1      | Tie, smaller ID wins |
| 4 | 1                | 1 | [1]                  | 1      | Single vote          |



|   |               |   |           |   |                                   |
|---|---------------|---|-----------|---|-----------------------------------|
| 5 | 2, 2, 3, 3, 1 | 3 | [1, 2, 2] | 2 | Tie between 2 and 3, smaller wins |
|---|---------------|---|-----------|---|-----------------------------------|

#### Program 4: Best Time to Buy and Sell Stock (LeetCode 121, Hard)

##### Problem Statement:

You are given a list `prices` where `prices[i]` is the price of a stock on day `i`. You want to maximize profit by buying on one day and selling on a later day. Return the maximum profit. If no profit is possible, return 0.

Note: You must buy before you sell (you cannot sell on day `i` and buy on day `j` where `j > i`).

##### Examples:

- Input: [7, 1, 5, 3, 6, 4]. Buy on day 2 (price 1), sell on day 5 (price 6). Profit = 5.
- Input: [7, 6, 4, 3, 1]. Prices only decrease. No profit. Output: 0.

##### Test Cases:

| # | Prices             | Max Profit | Notes                 |
|---|--------------------|------------|-----------------------|
| 1 | [7, 1, 5, 3, 6, 4] | 5          | Buy at 1, sell at 6   |
| 2 | [7, 6, 4, 3, 1]    | 0          | Decreasing, no profit |
| 3 | [1, 2]             | 1          | Minimum viable case   |
| 4 | [2, 4, 1]          | 2          | Buy at 2, sell at 4   |
| 5 | [3, 3, 3]          | 0          | All same, no profit   |

#### Program 5: Running Sum of Array (Medium)

##### Problem Statement:

A financial tracker computes running totals. Given a list of daily transactions (positive for income, negative for expenses), compute the running sum where:

- `running_sum[0] = transactions[0]`
- `running_sum[i] = sum of all transactions from index 0 to i`

Print the original transaction list and the running sum list.

##### Examples:

- Input: [1, 2, 3, 4]. Running sum: [1, 3, 6, 10].
- Input: [3, 1, 2, 10, 1]. Running sum: [3, 4, 6, 16, 17].

##### Test Cases:

| # | Transactions     | Running Sum       | Notes          |
|---|------------------|-------------------|----------------|
| 1 | [1, 2, 3, 4]     | [1, 3, 6, 10]     | Standard       |
| 2 | [1, 1, 1, 1, 1]  | [1, 2, 3, 4, 5]   | All ones       |
| 3 | [3, 1, 2, 10, 1] | [3, 4, 6, 16, 17] | Mixed          |
| 4 | [5]              | [5]               | Single element |
| 5 | [0, 0, 0]        | [0, 0, 0]         | All zeros      |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Remove Duplicates from List (Medium)

#### Problem Statement:

A data cleaning tool removes duplicate entries from a list while preserving the order of first occurrences. Given a list of N integers, create a new list containing only the first occurrence of each element. Do NOT use `set()`.

#### Examples:

- Input: [1, 2, 3, 2, 1, 4]. Output: [1, 2, 3, 4].
- Input: [3, 1, 3, 1, 2]. Output: [3, 1, 2].

#### Test Cases:

| # | Input              | Output          | Notes                            |
|---|--------------------|-----------------|----------------------------------|
| 1 | [1, 2, 3, 2, 1, 4] | [1, 2, 3, 4]    | Mixed duplicates                 |
| 2 | [1, 1, 1]          | [1]             | All same                         |
| 3 | [5, 4, 3, 2, 1]    | [5, 4, 3, 2, 1] | No duplicates                    |
| 4 | [1]                | [1]             | Single element                   |
| 5 | [3, 1, 3, 1, 2]    | [3, 1, 2]       | Preserves first occurrence order |

### Challenge 2: Move Zeroes (LeetCode 283, Medium)

#### Problem Statement:

Given a list, move all 0s to the end while maintaining the relative order of non-zero elements. Do it in-place (modify the original list, do not create a new one).

#### Examples:

- Input: [0, 1, 0, 3, 12]. Output: [1, 3, 12, 0, 0].
- Input: [0, 0, 1]. Output: [1, 0, 0].

#### Test Cases:

| # | Input            | Output           | Notes                   |
|---|------------------|------------------|-------------------------|
| 1 | [0, 1, 0, 3, 12] | [1, 3, 12, 0, 0] | LC Example 1            |
| 2 | [0]              | [0]              | Single zero             |
| 3 | [1]              | [1]              | Single non-zero         |
| 4 | [0, 0, 1]        | [1, 0, 0]        | Multiple zeros at start |
| 5 | [1, 2, 3]        | [1, 2, 3]        | No zeros                |

### Challenge 3: Single Number (LeetCode 136, Medium)

#### Problem Statement:

Given a non-empty list where every element appears exactly twice except for one element which appears once, find that single element.

Hint: XOR has these properties:  $a \oplus a = 0$  and  $a \oplus 0 = a$ . XOR all elements together.



**Examples:**

- Input: [2, 2, 1]. Output: 1.
- Input: [4, 1, 2, 1, 2]. Output: 4.

**Test Cases:**

| # | Input           | Output | Notes          |
|---|-----------------|--------|----------------|
| 1 | [2, 2, 1]       | 1      | LC Example 1   |
| 2 | [4, 1, 2, 1, 2] | 4      | LC Example 2   |
| 3 | [1]             | 1      | Single element |
| 4 | [5, 3, 5, 3, 9] | 9      | Larger list    |
| 5 | [0, 1, 0]       | 1      | With zeros     |

# CDP 406 - Lists (2D) and Tuples

## KEY CONCEPTS

### Opening

In Block 405, you worked with flat lists: one row of data. But real data often has rows AND columns: a spreadsheet of student marks, a chessboard, a seating chart. That is what 2D lists give you. A list of lists.

### 2D List Creation and Access

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
matrix[0]          # [1, 2, 3] (first row)  
matrix[1][2]       # 6 (row 1, column 2)  
matrix[-1][-1]     # 9 (last row, last column)
```

A 2D list is a list where each element is itself a list. `matrix[i]` gives you row `i`. `matrix[i][j]` gives you the element at row `i`, column `j`.

To create a 2D list of zeros:

```
rows, cols = 3, 4  
matrix = [[0] * cols for _ in range(rows)]
```

### The 2D List Creation Trap

There is a trap that catches almost every Python beginner.

```
# WRONG - DO NOT USE THIS  
matrix = [[0] * 4] * 3  
matrix[0][0] = 99  
print(matrix)  
# [[99, 0, 0, 0], [99, 0, 0, 0], [99, 0, 0, 0]]  
# ALL rows changed!
```

The `* 3` creates three references to the SAME inner list. Modifying one row modifies all of them because they are the same object in memory.

```
# CORRECT  
matrix = [[0] * 4 for _ in range(3)]  
matrix[0][0] = 99  
print(matrix)  
# [[99, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]  
# Only row 0 changed
```

The list comprehension creates a NEW inner list for each row. Always use `[[0] * cols for _ in range(rows)]`.

### Traversal

Two ways to traverse a 2D list:

```
# Row-wise (natural order)  
for row in matrix:  
    for val in row:
```



```
    print(val, end=" ")
print()

# By index (when you need row/column numbers)
for i in range(len(matrix)):
    for j in range(len(matrix[0])):
        print(matrix[i][j], end=" ")
    print()
```

**Column-wise traversal (needed for per-subject averages, etc.):**

```
# Column-wise: outer loop on columns, inner on rows
rows = len(matrix)
cols = len(matrix[0])
for j in range(cols):
    for i in range(rows):
        print(matrix[i][j], end=" ")
    print()
```

Many students confuse row-wise and column-wise. The difference is which loop is outer. Row-wise: outer on rows. Column-wise: outer on columns.

## Reading Matrix Input

```
rows = int(input("Enter rows: "))
cols = int(input("Enter cols: "))
matrix = []
for i in range(rows):
    row = list(map(int, input(f"Row {i + 1}: ").split()))
    matrix.append(row)
```

`list(map(int, input().split()))` reads a line of space-separated numbers and converts them to a list of integers. This is the standard pattern for reading matrix input.

## Tuples

A tuple is like a list, but immutable. Once created, you cannot change it.

```
t = (1, 2, 3)          # tuple
t = 1, 2, 3            # also a tuple (parentheses optional)
single = (5,)          # single-element tuple (comma required!)
not_a_tuple = (5)      # this is just the number 5, NOT a tuple

# Accessing
t[0]                   # 1
t[-1]                  # 3

# Immutable
t[0] = 10              # TypeError!
```

**When to use tuples vs lists:**

- Lists: data that changes (student marks, shopping cart items).
- Tuples: data that should not change (coordinates, RGB colors, function return values).

**Tuple unpacking:**

```
point = (3, 5)
x, y = point          # x=3, y=5

# Multiple return values from functions use tuples
def min_max(nums):
```





```
return min(nums), max(nums)
```

```
lo, hi = min_max([4, 1, 7, 2]) # lo=1, hi=7
```

One important fact for Block 410: lists cannot be dictionary keys because they are mutable. Tuples CAN be dictionary keys because they are immutable. This will matter when we group anagrams.

## IN-CLASS PROGRAMS

### Program 1: Classroom Seating Map (Easy)

#### Problem Statement:

A school management system stores a classroom seating arrangement as a 2D list (rows x columns). Each cell contains 1 (occupied) or 0 (empty).

Given the seating grid, find:

1. Total occupied seats
2. Total empty seats
3. The row with the most empty seats (0-indexed). If tie, return the first such row.

#### Examples:

- Input: `[[1, 0, 1], [0, 0, 1], [1, 1, 1]]`. Occupied=6, Empty=3, Row with most empty=1 (2 empty seats).

#### Test Cases:

| # | Grid                                         | Occupied | Empty | Row Most Empty | Notes                       |
|---|----------------------------------------------|----------|-------|----------------|-----------------------------|
| 1 | <code>[[1,0,1],[0,0,1],[1,1,1]]</code>       | 6        | 3     | 1              | Row 1 has 2 empty           |
| 2 | <code>[[1,1],[1,1]]</code>                   | 4        | 0     | 0              | All occupied, default row 0 |
| 3 | <code>[[0,0],[0,0]]</code>                   | 0        | 4     | 0              | All empty, tie at row 0     |
| 4 | <code>[[1]]</code>                           | 1        | 0     | 0              | 1x1 occupied                |
| 5 | <code>[[0,1,0,1],[1,1,1,1],[0,0,0,0]]</code> | 6        | 6     | 2              | Row 2 all empty             |

### Program 2: Exam Score Analyzer (Medium)

#### Problem Statement:

A university stores exam scores in a matrix where rows represent students and columns represent subjects.

Given the matrix, compute:

1. Each student's average (row-wise average)
2. Each subject's average (column-wise average)
3. The student with the highest overall average
4. The subject with the lowest overall average

#### Examples:

- Input: `[[80, 70, 90], [60, 50, 70], [90, 85, 95]]`.



- Student averages: [80.0, 60.0, 90.0]. Best student: index 2 (avg 90.0).
- Subject averages: [76.67, 68.33, 85.0]. Weakest subject: index 1 (avg 68.33).

### Test Cases:

| # | Matrix                             | Best Student (index) | Weakest Subject (index) | Notes                      |
|---|------------------------------------|----------------------|-------------------------|----------------------------|
| 1 | [[80,70,90],[60,50,70],[90,85,95]] | 2                    | 1                       | 3 students, 3 subjects     |
| 2 | [[100,100],[50,50]]                | 0                    | 0 (tie)                 | 2x2, tie defaults to first |
| 3 | [[70,80,60,90]]                    | 0                    | 2                       | 1 student, 4 subjects      |

### Program 3: Matrix Transpose (Medium)

#### Problem Statement:

A data transformation system transposes a matrix (converts rows to columns and columns to rows). Given an M x N matrix, produce the N x M transposed matrix.

#### Examples:

- Input: [[1,2,3],[4,5,6]] (2x3). Transposed: [[1,4],[2,5],[3,6]] (3x2).
- Input: [[1,2,3],[4,5,6],[7,8,9]] (3x3 square). Transposed: [[1,4,7],[2,5,8],[3,6,9]].

### Test Cases:

| # | Input                     | Transposed                | Notes      |
|---|---------------------------|---------------------------|------------|
| 1 | [[1,2,3],[4,5,6]]         | [[1,4],[2,5],[3,6]]       | 2x3 to 3x2 |
| 2 | [[1,2],[3,4],[5,6]]       | [[1,3,5],[2,4,6]]         | 3x2 to 2x3 |
| 3 | [[1,2,3],[4,5,6],[7,8,9]] | [[1,4,7],[2,5,8],[3,6,9]] | 3x3 square |
| 4 | [[5]]                     | [[5]]                     | 1x1        |
| 5 | [[1,2,3,4]]               | [[1],[2],[3],[4]]         | 1x4 to 4x1 |

### Program 4: Rotate Image (LeetCode 48, Hard)

#### Problem Statement:

You are given an n x n 2D matrix representing an image. Rotate the image by 90 degrees clockwise. You must do this in-place (modify the input matrix directly, do not create a new matrix).

#### Examples:

- Input: [[1,2,3],[4,5,6],[7,8,9]]. Output: [[7,4,1],[8,5,2],[9,6,3]].
- Input: [[1,2],[3,4]]. Output: [[3,1],[4,2]].

### Test Cases:

| # | Input                     | Output                    | Notes |
|---|---------------------------|---------------------------|-------|
| 1 | [[1,2,3],[4,5,6],[7,8,9]] | [[7,4,1],[8,5,2],[9,6,3]] | 3x3   |



|   |                                                  |                                                  |     |
|---|--------------------------------------------------|--------------------------------------------------|-----|
| 2 | [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]] | [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]] | 4x4 |
| 3 | [[1]]                                            | [[1]]                                            | 1x1 |
| 4 | [[1,2],[3,4]]                                    | [[3,1],[4,2]]                                    | 2x2 |

## Program 5: Diagonal Sum Calculator (Medium)

### Problem Statement:

A matrix analytics tool computes diagonal properties of a square matrix. Given a square matrix of size N, calculate:

1. Primary diagonal sum: elements where row index equals column index ( $\text{matrix}[i][i]$ ).
2. Secondary diagonal sum: elements where row + column = N - 1 ( $\text{matrix}[i][N-1-i]$ ).
3. Combined sum: primary + secondary. If N is odd, subtract the center element once (it appears in both diagonals).

### Examples:

- Input:  $[[1, 2, 3], [4, 5, 6], [7, 8, 9]]$ . Primary =  $1+5+9 = 15$ . Secondary =  $3+5+7 = 15$ . Center (5) counted twice. Combined =  $15+15-5 = 25$ .

### Test Cases:

| # | Matrix                                    | Primary | Secondary | Combined | Notes                     |
|---|-------------------------------------------|---------|-----------|----------|---------------------------|
| 1 | [[1,2,3],[4,5,6],[7,8,9]]                 | 15      | 15        | 25       | 3x3, center=5 subtracted  |
| 2 | [[1,2],[3,4]]                             | 5       | 5         | 10       | 2x2, even size, no center |
| 3 | [[5]]                                     | 5       | 5         | 5        | 1x1, center subtracted    |
| 4 | [[1,0,0,0],[0,2,0,0],[0,0,3,0],[0,0,0,4]] | 10      | 0         | 10       | 4x4, secondary all zeros  |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Boundary Traversal (Medium)

#### Problem Statement:

A surveillance system monitors boundary cameras of a building grid. Given an M x N matrix, print all boundary elements in clockwise order: top row (left to right), right column (top to bottom, excluding corners already counted), bottom row (right to left), left column (bottom to top, excluding corners).

Handle edge cases: single row, single column, 1x1 matrix.

#### Examples:

- Input:  $[[1, 2, 3], [4, 5, 6], [7, 8, 9]]$ . Boundary:  $[1, 2, 3, 6, 9, 8, 7, 4]$ .



### Test Cases:

| # | Matrix                    | Boundary          | Notes         |
|---|---------------------------|-------------------|---------------|
| 1 | [[1,2,3],[4,5,6],[7,8,9]] | [1,2,3,6,9,8,7,4] | 3x3           |
| 2 | [[1,2,3,4]]               | [1,2,3,4]         | Single row    |
| 3 | [[1],[2],[3]]             | [1,2,3]           | Single column |
| 4 | [[5]]                     | [5]               | 1x1           |
| 5 | [[1,2],[3,4],[5,6]]       | [1,2,4,6,5,3]     | 3x2           |

### Challenge 2: Row with Maximum Ones (Medium)

#### Problem Statement:

Given a binary matrix (containing only 0s and 1s), find the row that contains the maximum number of 1s. If multiple rows have the same count, return the first one (smallest index).

#### Examples:

- Input: `[[0,1,1],[1,1,1],[0,0,1]]`. Row 1 has 3 ones. Output: Row 1.

### Test Cases:

| # | Matrix                    | Row Index | Count | Notes               |
|---|---------------------------|-----------|-------|---------------------|
| 1 | [[0,1,1],[1,1,1],[0,0,1]] | 1         | 3     | Row 1 has most      |
| 2 | [[1,1],[1,1]]             | 0         | 2     | Tie, first row wins |
| 3 | [[0,0],[0,1]]             | 1         | 1     | Row 1 has only one  |
| 4 | [[1,0,1,0,1]]             | 0         | 3     | Single row          |
| 5 | [[0]]                     | 0         | 0     | Single zero         |

### Challenge 3: Spiral Matrix (LeetCode 54, Medium)

#### Problem Statement:

Given an m x n matrix, return all elements of the matrix in spiral order.

#### Examples:

- Input: `[[1,2,3],[4,5,6],[7,8,9]]`. Output: `[1,2,3,6,9,8,7,4,5]`.
- Input: `[[1,2,3,4],[5,6,7,8],[9,10,11,12]]`. Output: `[1,2,3,4,8,12,11,10,9,5,6,7]`.

### Test Cases:

| # | Matrix                             | Spiral Order                 | Notes      |
|---|------------------------------------|------------------------------|------------|
| 1 | [[1,2,3],[4,5,6],[7,8,9]]          | [1,2,3,6,9,8,7,4,5]          | 3x3        |
| 2 | [[1,2,3,4],[5,6,7,8],[9,10,11,12]] | [1,2,3,4,8,12,11,10,9,5,6,7] | 3x4        |
| 3 | [[1]]                              | [1]                          | 1x1        |
| 4 | [[1,2],[3,4]]                      | [1,2,4,3]                    | 2x2        |
| 5 | [[1,2,3]]                          | [1,2,3]                      | Single row |



# CDP 407 - Strings (Priority Block)

## KEY CONCEPTS

### Opening

This is one of the most important blocks in the entire track. Strings appear in every company coding test. Password validation, text processing, pattern matching, anagram detection. DSA Block 503 is entirely about string problem solving, and it assumes you are already fluent with string manipulation. This block makes you fluent.

### Strings are Immutable

In Python, strings cannot be changed after creation.

```
s = "hello"
s[0] = "H"      # TypeError! Strings are immutable.

# To "modify" a string, you create a new one:
s = "H" + s[1:]  # s is now "Hello" (new string)
```

Every time you modify a string, Python creates a new string object. This matters for performance: if you build a string character by character in a loop using +, each + creates a new string. For DSA, when you need to modify characters, convert to a list first, modify the list, then join back.

```
# The DSA string mutation pattern:
chars = list(s)      # convert to list
# ... modify chars ...
result = "".join(chars) # convert back to string
```

### Indexing and Slicing

```
s = "Hello World"

# Indexing
s[0]      # 'H'
s[-1]     # 'd'
s[6]      # 'W'

# Slicing (same rules as lists)
s[0:5]    # 'Hello'
s[6:]     # 'World'
s[::-1]   # 'dlroW olleH' (reversed)
s[::2]    # 'HloWrD' (every other char)
```

Slicing works exactly like list slicing. If you know list slicing from Block 405, you already know string slicing.

### Key String Methods

Python strings have many built-in methods. Here are the ones you will use most.

```
s = "  Hello, World!  "

len(s)          # 17
s.upper()       # "  HELLO, WORLD!  "
s.lower()       # "  hello, world!  "
s.strip()       # "Hello, World!" (remove leading/trailing whitespace)
s.split()       # ['Hello,', ' ', 'World!'] (split on whitespace)
```



```
s.split(',')      # [' Hello', ' World! '] (split on comma)
" ".join(['a','b']) # "a b" (join list with separator)
s.replace("World", "Python") # " Hello, Python! "
s.find("World")    # 9 (index, or -1 if not found)
s.index("World")   # 9 (index, or ValueError if not found)
s.count("l")       # 3
s.startswith(" He") # True
s.endswith("! ")   # True
```

**Character classification methods:**

```
c = "A"
c.isalpha()      # True (is a letter)
c.isdigit()      # False (is a digit)
c.isalnum()      # True (letter or digit)
c.isupper()      # True
c.islower()      # False
```

**Note:** Python has no `char` type. `s[i]` returns a string of length 1. These methods work on single-character strings.

## ASCII Conversion: `ord()` and `chr()`

```
ord('A')        # 65
ord('a')        # 97
ord('0')        # 48
chr(65)         # 'A'
chr(97)         # 'a'

# Character arithmetic:
ord('c') - ord('a') # 2 (position in alphabet, 0-indexed)
chr(ord('a') + 3)   # 'd' (shift by 3)
```

This is how you do character-level math: converting characters to numbers with `ord()`, computing, and converting back with `chr()`. You will use this for Caesar cipher, frequency counting with arrays, and many DSA problems.

## `split()` and `join()`

The `split()` and `join()` combo is one of Python's most powerful string tools.

```
sentence = "the sky is blue"
words = sentence.split()          # ['the', 'sky', 'is', 'blue']
reversed_sentence = " ".join(words[::-1]) # 'blue is sky the'

# split() without arguments handles:
# - multiple spaces between words
# - leading/trailing spaces
# All cleaned automatically!

csv_line = "Alice,25,Engineer"
fields = csv_line.split(",")      # ['Alice', '25', 'Engineer']
```

**Note:** `join()` is called on the SEPARATOR, not the list. `" ".join(words)`, not `words.join(" ")`. This confuses many students initially.



## No == vs .equals() Trap

Good news: Python does not have Java's `==` vs `.equals()` trap.

```
"abc" == "abc"      # True (compares content)
"abc" is "abc"      # Usually True (implementation detail, don't rely on it)
```

In Python, `==` compares content. Use `==` for string comparison. The `is` keyword compares identity (same object in memory), which you rarely need.

## f-strings Review

```
name = "Alice"
score = 87.5
print(f"Student: {name}, Score: {score:.2f}")
# Output: Student: Alice, Score: 87.50
```

You learned f-strings in Block 401. They are the cleanest way to build formatted output. Use them everywhere.

## IN-CLASS PROGRAMS

### Program 1: Username Validator (Easy)

#### Problem Statement:

A registration system validates usernames. A valid username must satisfy all of these rules:

1. Length must be 6 to 15 characters (inclusive).
2. Must start with a letter.
3. Can only contain letters, digits, and underscores.
4. Must not contain spaces.

Given a username, print "VALID" if all rules pass. Otherwise, print "INVALID" and the specific rule(s) that failed.

#### Examples:

- Input: "alice\_99". Output: VALID.
- Input: "ab". Output: INVALID (too short).
- Input: "1alice". Output: INVALID (starts with digit).

#### Test Cases:

| # | Username  | Result  | Notes                                 |
|---|-----------|---------|---------------------------------------|
| 1 | alice_99  | VALID   | Letters, digits, underscore, length 8 |
| 2 | ab        | INVALID | Too short (length 2)                  |
| 3 | 1alice    | INVALID | Starts with digit                     |
| 4 | alice doe | INVALID | Contains space                        |
| 5 | alice@99  | INVALID | Contains special character @          |

## Program 2: Valid Palindrome (LeetCode 125, Medium)

### Problem Statement:

A phrase is a palindrome if, after converting all uppercase letters to lowercase and removing all non-alphanumeric characters, it reads the same forward and backward.

Given a string *s*, return True if it is a palindrome, False otherwise.

### Examples:

- Input: "A man, a plan, a canal: Panama". Output: True.
- Input: "race a car". Output: False.
- Input: " ". Output: True (empty after cleaning).

### Test Cases:

| # | Input                            | Result | Notes                |
|---|----------------------------------|--------|----------------------|
| 1 | "A man, a plan, a canal: Panama" | True   | Classic palindrome   |
| 2 | "race a car"                     | False  | Not a palindrome     |
| 3 | " "                              | True   | Empty after cleaning |
| 4 | "ab"                             | False  | Two different chars  |
| 5 | "aA"                             | True   | Case insensitive     |

## Program 3: Word Reversal Engine (Medium)

### Problem Statement:

A text processing system reverses the order of words in a sentence. Given "the sky is blue", output "blue is sky the".

Handle edge cases: multiple spaces between words, leading/trailing spaces. These should not produce empty words in the output.

### Examples:

- Input: "the sky is blue". Output: "blue is sky the".
- Input: " hello world ". Output: "world hello".
- Input: "a good example". Output: "example good a".

### Test Cases:

| # | Input             | Output            | Notes                   |
|---|-------------------|-------------------|-------------------------|
| 1 | "the sky is blue" | "blue is sky the" | Standard                |
| 2 | " hello world "   | "world hello"     | Leading/trailing spaces |
| 3 | "a good example"  | "example good a"  | Multiple spaces between |
| 4 | "hello"           | "hello"           | Single word             |
| 5 | ""                | ""                | Empty string            |





## Program 4: Longest Common Prefix (LeetCode 14, Hard)

### Problem Statement:

Write a function to find the longest common prefix string amongst a list of strings. If there is no common prefix, return "".

### Examples:

- Input: ["flower", "flow", "flight"]. Output: "fl".
- Input: ["dog", "racecar", "car"]. Output: "".

### Test Cases:

| # | Input                                          | Output   | Notes                |
|---|------------------------------------------------|----------|----------------------|
| 1 | ["flower", "flow", "flight"]                   | "fl"     | LC Example 1         |
| 2 | ["dog", "racecar", "car"]                      | ""       | No common prefix     |
| 3 | ["abc", "abc", "abc"]                          | "abc"    | All identical        |
| 4 | ["a"]                                          | "a"      | Single string        |
| 5 | ["", "abc"]                                    | ""       | Empty string in list |
| 6 | ["interspecies", "interstellar", "interstate"] | "inters" | Longer prefix        |

## Program 5: Anagram Checker (Medium)

### Problem Statement:

A word game app checks if two words are anagrams (same letters, different order). Given two strings, determine if they are anagrams. Ignore case.

Solve using two approaches:

1. Sorting: sort both strings and compare.
2. Frequency counting: count character frequencies and compare.

### Examples:

- Input: "listen", "silent". Output: True (anagrams).
- Input: "hello", "world". Output: False.
- Input: "abc", "cba". Output: True.

### Test Cases:

| # | String 1   | String 2     | Result | Notes                          |
|---|------------|--------------|--------|--------------------------------|
| 1 | listen     | silent       | True   | Classic anagram                |
| 2 | hello      | world        | False  | Not anagrams                   |
| 3 | abc        | cba          | True   | Reversed                       |
| 4 | aab        | abb          | False  | Same chars, different counts   |
| 5 | Astronomer | Moon starrer | False  | Different lengths (with space) |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Caesar Cipher Encoder/Decoder (Medium)

#### Problem Statement:

A messaging app encrypts messages using Caesar cipher. Given a string and a shift value  $K$ , shift each letter by  $K$  positions in the alphabet. Wrapping: 'z' shifted by 1 becomes 'a'. Non-letter characters remain unchanged.

Write two functions: `encode(text, k)` and `decode(text, k)`.

#### Examples:

- Input: "Hello, World!",  $K=3$ . Encoded: "Khood, Zruog!". Decoded back: "Hello, World!".
- Input: "xyz",  $K=3$ . Encoded: "abc" (wraps around).

#### Test Cases:

| # | Text          | K  | Encoded       | Notes                    |
|---|---------------|----|---------------|--------------------------|
| 1 | Hello, World! | 3  | Khood, Zruog! | Standard shift           |
| 2 | abc           | 1  | bcd           | Simple shift             |
| 3 | xyz           | 3  | abc           | Wrap around              |
| 4 | ABC           | 26 | ABC           | Full rotation, no change |
| 5 | Hello!        | 0  | Hello!        | Zero shift               |

### Challenge 2: Find the Index of the First Occurrence in a String (LeetCode 28, Medium)

#### Problem Statement:

Given two strings `haystack` and `needle`, return the index of the first occurrence of `needle` in `haystack`, or -1 if `needle` is not part of `haystack`. Implement without using `.find()`.

#### Examples:

- Input: `haystack="sadbutsad"`, `needle="sad"`. Output: 0.
- Input: `haystack="leetcode"`, `needle="leeto"`. Output: -1.

#### Test Cases:

| # | Haystack  | Needle  | Output | Notes             |
|---|-----------|---------|--------|-------------------|
| 1 | sadbutsad | sad     | 0      | Found at start    |
| 2 | leetcode  | leeto   | -1     | Not found         |
| 3 | hello     | ll      | 2      | Found in middle   |
| 4 | a         | a       | 0      | Single char match |
| 5 | abc       | (empty) | 0      | Empty needle      |

### Challenge 3: Roman to Integer (LeetCode 13, Medium)

#### Problem Statement:

Given a Roman numeral string, convert it to an integer. Roman numerals use: I=1, V=5, X=10, L=50, C=100, D=500, M=1000. Subtraction rule: if a smaller value appears before a larger value, subtract (e.g., IV=4, IX=9, XL=40, XC=90, CD=400, CM=900).

#### Examples:

- Input: "III". Output: 3.
- Input: "MCMXCIV". Output: 1994 (M=1000, CM=900, XC=90, IV=4).

#### Test Cases:

| # | Input   | Output | Notes                      |
|---|---------|--------|----------------------------|
| 1 | III     | 3      | Simple addition            |
| 2 | LVIII   | 58     | L=50, V=5, III=3           |
| 3 | MCMXCIV | 1994   | Multiple subtraction cases |
| 4 | IV      | 4      | Subtraction case           |
| 5 | IX      | 9      | Subtraction case           |

# CDP 408 - Recursion

## KEY CONCEPTS

### Opening

Every problem so far, you have solved with loops. Loops are powerful. But some problems have a natural structure where the solution looks like a smaller version of the same problem. Factorial of 5 is 5 times factorial of 4. The reverse of a string is the reverse of everything except the first character, with the first character at the end. When a problem naturally breaks into smaller versions of itself, recursion is the right tool.

### What is Recursion

A recursive function is a function that calls itself.

Every recursive function needs two things:

1. **Base case:** when to stop. Without this, the function calls itself forever.
2. **Recursive case:** how to break the problem into a smaller version.

```
def factorial(n):  
    if n <= 1:          # base case  
        return 1  
    return n * factorial(n - 1) # recursive case  
  
print(factorial(5)) # 120
```

factorial(5) calls factorial(4), which calls factorial(3), which calls factorial(2), which calls factorial(1). factorial(1) hits the base case and returns 1. Then the results multiply back up: 1, 2, 6, 24, 120.

### Call Stack Visualization

Every function call creates a frame on the call stack.

```
factorial(4) called  
  factorial(3) called  
    factorial(2) called  
      factorial(1) called  
        returns 1          <-- base case  
      returns 2 * 1 = 2  
    returns 3 * 2 = 6  
  returns 4 * 6 = 24
```

Each call waits for the one below it to finish. factorial(4) cannot return until factorial(3) returns. factorial(3) cannot return until factorial(2) returns. This is the call stack.

What happens without a base case?

```
def infinite(n):  
    return infinite(n - 1) # no base case!
```

Python allows about 1000 recursive calls by default. After that, you get `RecursionError: maximum recursion depth exceeded`. This is Python's version of Java's `StackOverflowError`.

### Recursion Template

```
def solve(problem):  
    if base_case:  
        return base_value  
    else:  
        return combine(solve(smaller_problem))
```



Every recursive function follows this template. The base case handles the simplest version of the problem. The recursive case makes the problem smaller and combines the result.

## Fibonacci and the DP Seed

```
def fib(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fib(n - 1) + fib(n - 2)
```

This works. `fib(5)` returns 5. But watch this.

### Call Tree for `fib(5)`:

```

      fib(5)
     /    \
  fib(4)   fib(3)
  /  \   /  \
fib(3) fib(2) fib(2) fib(1)
 /  \ /  \ /  \
fib(2) fib(1) fib(1) fib(0) fib(1) fib(0)
 /  \
fib(1) fib(0)
```

Count the total function calls. `fib(5)` makes 15 calls. `fib(3)` is computed TWICE. `fib(2)` is computed THREE times. This is wasteful.

`fib(10)` makes 177 calls. `fib(30)` makes over 2 million calls. `fib(50)` would take hours.

In DSA Block 515, you will learn Dynamic Programming. DP stores results so each value is computed only once. `fib(50)` takes 50 operations instead of billions. Remember this moment. When you reach DP, everything will click.

## Recursion vs Iteration

Any recursive solution can be written with loops. Any loop can be written with recursion. They are equivalent in power.

Use recursion when:

- The problem naturally divides into smaller versions of itself.
- The problem has a tree-like structure (binary trees, file systems).
- The code is significantly cleaner with recursion.

Use iteration when:

- Performance matters (recursion has function call overhead).
- The recursion depth might exceed Python's limit (1000).
- The iterative version is equally clear.

## Python-Specific: Default Arguments for Recursive Helpers

In Java, if you need an index parameter for recursion, you write a helper method. In Python, you can use default arguments:

```
def list_sum(arr, index=0):
    if index == len(arr):
        return 0
    return arr[index] + list_sum(arr, index + 1)
```



```
# Called as: list_sum([1, 2, 3]) -- index defaults to 0
```

The caller does not need to know about the index parameter. It is hidden as a default.

## IN-CLASS PROGRAMS

### Program 1: Power Calculator (Easy)

#### Problem Statement:

A scientific calculator computes  $x^n$  ( $x$  raised to the power  $n$ ) for positive integer  $n$ .

Write a recursive function `power(x, n)`:

- Base case:  $n == 0$ , return 1.
- Recursive case: return  $x * \text{power}(x, n - 1)$ .

After solving, show the optimized version: if  $n$  is even,  $\text{power}(x, n) = \text{power}(x, n/2) * \text{power}(x, n/2)$ . This halves the recursive calls.

#### Examples:

- $\text{power}(2, 10) = 1024$ .  $\text{power}(5, 3) = 125$ .  $\text{power}(3, 0) = 1$ .

#### Test Cases:

| # | x | n   | Result | Notes                     |
|---|---|-----|--------|---------------------------|
| 1 | 2 | 10  | 1024   | Standard                  |
| 2 | 3 | 0   | 1      | $\text{Any}^0 = 1$        |
| 3 | 5 | 3   | 125    | $5^3$                     |
| 4 | 1 | 100 | 1      | $1^{\text{anything}} = 1$ |
| 5 | 2 | 1   | 2      | Base + 1 step             |

### Program 2: List Sum and Max Finder (Medium)

#### Problem Statement:

A data processing system computes list statistics recursively. Write two functions:

1. `list_sum(arr, index=0)`: returns the sum of all elements from index to the end.
- Base case:  $\text{index} == \text{len}(\text{arr})$ , return 0.
  - Recursive case:  $\text{arr}[\text{index}] + \text{list\_sum}(\text{arr}, \text{index} + 1)$ .
1. `list_max(arr, index=0)`: returns the maximum element.
- Base case:  $\text{index} == \text{len}(\text{arr}) - 1$ , return  $\text{arr}[\text{index}]$ .
  - Recursive case:  $\max(\text{arr}[\text{index}], \text{list\_max}(\text{arr}, \text{index} + 1))$ .

#### Examples:

- $\text{list\_sum}([1, 2, 3, 4, 5]) = 15$ .
- $\text{list\_max}([1, 2, 3, 4, 5]) = 5$ .
- $\text{list\_max}([-3, -1, -7]) = -1$ .

#### Test Cases:

| # | List            | Sum | Max | Notes    |
|---|-----------------|-----|-----|----------|
| 1 | [1, 2, 3, 4, 5] | 15  | 5   | Standard |



|   |                  |     |     |                |
|---|------------------|-----|-----|----------------|
| 2 | [10]             | 10  | 10  | Single element |
| 3 | [-3, -1, -7, -2] | -13 | -1  | All negative   |
| 4 | [5, 5, 5]        | 15  | 5   | All same       |
| 5 | [100, 1, 50, 99] | 250 | 100 | Max at start   |

### Program 3: String Reversal and Palindrome Checker (Medium)

#### Problem Statement:

A text analytics tool needs recursive string operations. Write:

1. `reverse_str(s)`: reverses a string recursively.
  - Base case: `len(s) <= 1`, return `s`.
  - Recursive case: `reverse_str(s[1:]) + s[0]`.
1. `is_palindrome(s, left=0, right=None)`: checks if a string is a palindrome recursively.
  - If `right` is `None`, set `right = len(s) - 1`.
  - Base case: `left >= right`, return `True`.
  - Recursive case: if `s[left] != s[right]`, return `False`. Else, `is_palindrome(s, left+1, right-1)`.

#### Examples:

- `reverse_str("hello") = "olleh"`. `reverse_str("racecar") = "racecar"`.
- `is_palindrome("racecar") = True`. `is_palindrome("hello") = False`.

#### Test Cases:

| # | Function                   | Input     | Output  | Notes                  |
|---|----------------------------|-----------|---------|------------------------|
| 1 | <code>reverse_str</code>   | "hello"   | "olleh" | Standard               |
| 2 | <code>reverse_str</code>   | "a"       | "a"     | Single char            |
| 3 | <code>is_palindrome</code> | "racecar" | True    | Odd-length palindrome  |
| 4 | <code>is_palindrome</code> | "abba"    | True    | Even-length palindrome |
| 5 | <code>is_palindrome</code> | "hello"   | False   | Not palindrome         |

### Program 4: Fibonacci Number (LeetCode 509, Hard)

#### Problem Statement:

The Fibonacci numbers are:  $F(0) = 0$ ,  $F(1) = 1$ ,  $F(n) = F(n-1) + F(n-2)$  for  $n > 1$ .

Given  $n$ , calculate  $F(n)$  using naive recursion.

Then, count how many function calls are made for  $F(5)$  and  $F(10)$ .

#### Examples:

- $F(0) = 0$ .  $F(1) = 1$ .  $F(5) = 5$ .  $F(10) = 55$ .
- $F(5)$  makes 15 function calls.  $F(10)$  makes 177 function calls.

#### Test Cases:

| # | n | F(n) | Notes |
|---|---|------|-------|
|---|---|------|-------|



|   |    |     |           |
|---|----|-----|-----------|
| 1 | 0  | 0   | Base case |
| 2 | 1  | 1   | Base case |
| 3 | 5  | 5   | F(5)      |
| 4 | 10 | 55  | F(10)     |
| 5 | 15 | 610 | F(15)     |

### Program 5: Power of Two (LeetCode 231, Medium)

#### Problem Statement:

Given an integer  $n$ , return True if it is a power of two using recursion.

#### Examples:

- $n=1$ : True ( $2^0$ ).  $n=16$ : True ( $2^4$ ).  $n=3$ : False.

#### Test Cases:

| # | n    | Result | Notes          |
|---|------|--------|----------------|
| 1 | 1    | True   | $2^0$          |
| 2 | 16   | True   | $2^4$          |
| 3 | 3    | False  | Not power of 2 |
| 4 | 0    | False  | Zero           |
| 5 | 1024 | True   | $2^{10}$       |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Tower of Hanoi (Medium)

#### Problem Statement:

The classic Tower of Hanoi puzzle: move  $N$  discs from source peg to destination peg using an auxiliary peg. Rules: only one disc at a time, never place a larger disc on a smaller one.

Print each move: "Move disc  $X$  from [source] to [destination]".

#### Examples:

- $N=1$ : 1 move.  $N=2$ : 3 moves.  $N=3$ : 7 moves. Formula:  $2^N - 1$  moves.

#### Test Cases:

| # | N | Number of Moves | Notes     |
|---|---|-----------------|-----------|
| 1 | 1 | 1               | Trivial   |
| 2 | 2 | 3               | 3 moves   |
| 3 | 3 | 7               | Classic   |
| 4 | 4 | 15              | $2^4 - 1$ |
| 5 | 5 | 31              | $2^5 - 1$ |





## Challenge 2: Pow(x, n) (LeetCode 50, Medium)

### Problem Statement:

Implement  $\text{pow}(x, n)$  which calculates  $x$  raised to the power  $n$ . Handle negative  $n$ :  $x^{(-n)} = 1 / x^n$ . Use optimized recursion: if  $n$  is even,  $\text{pow}(x, n) = \text{pow}(x, n/2)^2$ .

### Examples:

- $\text{pow}(2.0, 10) = 1024.0$ .  $\text{pow}(2.0, -2) = 0.25$ .

### Test Cases:

| # | x   | n    | Result | Notes                     |
|---|-----|------|--------|---------------------------|
| 1 | 2.0 | 10   | 1024.0 | Standard                  |
| 2 | 2.1 | 3    | 9.261  | Decimal base              |
| 3 | 2.0 | -2   | 0.25   | Negative exponent         |
| 4 | 1.0 | 1000 | 1.0    | $1^{\text{anything}} = 1$ |
| 5 | 0.5 | 2    | 0.25   | Fractional base           |

## Challenge 3: Reverse String (LeetCode 344, Medium)

### Problem Statement:

Write a function that reverses a list of characters in-place using recursion. Swap the first and last characters, then recurse on the sublist between them.

### Examples:

- Input: ["h","e","l","l","o"]. Output: ["o","l","l","e","h"].

### Test Cases:

| # | Input                     | Output                    | Notes       |
|---|---------------------------|---------------------------|-------------|
| 1 | ["h","e","l","l","o"]     | ["o","l","l","e","h"]     | Standard    |
| 2 | ["H","a","n","n","a","h"] | ["h","a","n","n","a","H"] | Mixed case  |
| 3 | ["a"]                     | ["a"]                     | Single char |
| 4 | ["a","b"]                 | ["b","a"]                 | Two chars   |

# CDP 409 - Searching and Sorting

## KEY CONCEPTS

### Opening

Every company coding test has at least one problem that requires searching or sorting. Finding an element in a list, arranging data in order, searching in a sorted dataset. This block covers the core algorithms. You will implement them from scratch, understand their costs, and then see Python's built-in shortcuts.

### Linear Search

The simplest search: check every element one by one.

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

Time:  $O(n)$ . You check every element in the worst case. This is what `if x in list` does internally.

### Binary Search

If the list is sorted, you can do much better. Binary search eliminates half the elements with each comparison.

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

This is the binary search template. Memorize it. You will use variations of this in DSA Block 504.

Time:  $O(\log n)$ . For a list of 1,000,000 elements, binary search checks at most 20 elements. Linear search checks up to 1,000,000. That is the power of  $O(\log n)$ .

Requirement: the list **MUST** be sorted. Binary search on an unsorted list gives wrong results.

### Bubble Sort

The simplest sorting algorithm. Repeatedly compare adjacent elements and swap if out of order.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

After each pass, the largest unsorted element 'bubbles' to its correct position. Time:  $O(n^2)$ . Slow for large lists.



## Selection Sort

Find the smallest element, put it first. Find the next smallest, put it second. Repeat.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

Time:  $O(n^2)$ . Same as bubble sort. Both are slow. They exist so you understand what sorting means.

The sort you will actually use for efficiency is merge sort.

## Merge Sort

Merge sort uses recursion (Block 408) to divide the list in half, sort each half, and merge them back.

```
[38, 27, 43, 3, 9, 82, 10]
      /           \
[38, 27, 43]       [3, 9, 82, 10]
 /   \           /   \
[38]  [27, 43]   [3, 9]  [82, 10]
 /   \         /   \   /   \
[27] [43]    [3]  [9] [82] [10]
 \   /       \   /   \   /
[27, 43]    [3, 9]  [10, 82]
      \       /       |
[27, 38, 43]    [3, 9, 10, 82]
              \       /
[3, 9, 10, 27, 38, 43, 82]
```

Two functions: `merge_sort(arr)` splits the list recursively. `merge(left, right)` combines two sorted lists into one sorted list.

Time:  $O(n \log n)$ . Much faster than  $O(n^2)$  for large lists.

Python-specific: `arr[:mid]` and `arr[mid:]` create new lists for the split. This is cleaner than Java's index-based approach.

## Built-in Sorting

After understanding how sorting works, here are Python's built-in tools:

```
nums = [3, 1, 4, 1, 5]

sorted(nums)                # [1, 1, 3, 4, 5] - new list
nums.sort()                 # sorts in-place, returns None
sorted(nums, reverse=True)  # [5, 4, 3, 1, 1] - descending
sorted(nums, key=lambda x: -x) # same as reverse=True

# Custom sorting with key:
students = [("Alice", 85), ("Bob", 92), ("Charlie", 78)]
sorted(students, key=lambda s: s[1], reverse=True)
# [("Bob", 92), ("Alice", 85), ("Charlie", 78)]

Manual for learning. Built-in for production.
```



You must know how to implement binary search, bubble sort, selection sort, and merge sort from scratch. For actual coding tests, `sorted()` is fine for sorting. But binary search, you always implement yourself because variations require template modifications.

## IN-CLASS PROGRAMS

### Program 1: Library Book Finder (Easy)

#### Problem Statement:

A library search system helps users find books. Given a sorted list of book IDs and a target book ID, implement iterative binary search to find the book's position. If found, print "BOOK FOUND AT POSITION X". If not found, print "BOOK NOT FOUND".

#### Examples:

- Input: books = [101, 205, 310, 415, 520, 625, 730], target = 415. Output: "BOOK FOUND AT POSITION 3".
- Input: same list, target = 999. Output: "BOOK NOT FOUND".

#### Test Cases:

| # | Book IDs                            | Target | Result     | Notes          |
|---|-------------------------------------|--------|------------|----------------|
| 1 | [101, 205, 310, 415, 520, 625, 730] | 415    | Found at 3 | Middle element |
| 2 | [101, 205, 310, 415, 520, 625, 730] | 101    | Found at 0 | First element  |
| 3 | [101, 205, 310, 415, 520, 625, 730] | 730    | Found at 6 | Last element   |
| 4 | [101, 205, 310, 415, 520, 625, 730] | 999    | Not found  | Target absent  |
| 5 | [42]                                | 42     | Found at 0 | Single element |

### Program 2: Student Rank Sorter (Medium)

#### Problem Statement:

A school ranking system sorts students by total marks. Given a list of marks:

- Implement selection sort to sort in descending order (highest first).
- Implement bubble sort on the same data for comparison.
- Print the sorted list with rank numbers.
- After both, show `sorted(marks, reverse=True)` produces the same result.

#### Examples:

- Input: [75, 90, 60, 85, 70]. Selection sort descending: [90, 85, 75, 70, 60]. Bubble sort descending: [90, 85, 75, 70, 60]. `sorted()`: [90, 85, 75, 70, 60]. All identical.

#### Test Cases:

| # | Input                | Sorted (Descending)  | Notes    |
|---|----------------------|----------------------|----------|
| 1 | [75, 90, 60, 85, 70] | [90, 85, 75, 70, 60] | Standard |
| 2 | [50, 50, 50]         | [50, 50, 50]         | All same |



|   |                      |                      |                         |
|---|----------------------|----------------------|-------------------------|
| 3 | [100]                | [100]                | Single element          |
| 4 | [10, 20, 30, 40, 50] | [50, 40, 30, 20, 10] | Ascending to descending |
| 5 | [90, 80, 70, 60, 50] | [90, 80, 70, 60, 50] | Already sorted          |

### Program 3: Merge Sort Implementation (Medium)

#### Problem Statement:

A data pipeline needs to sort large datasets efficiently. Implement merge sort from scratch:

1. `merge_sort(arr)`: recursively divides the list into halves until single elements remain.
2. `merge(left, right)`: merges two sorted lists into one sorted list.

Test on a list of at least 7 elements. Print the list before and after sorting.

#### Examples:

- Input: [38, 27, 43, 3, 9, 82, 10]. Output: [3, 9, 10, 27, 38, 43, 82].

#### Test Cases:

| # | Input                      | Output                     | Notes          |
|---|----------------------------|----------------------------|----------------|
| 1 | [38, 27, 43, 3, 9, 82, 10] | [3, 9, 10, 27, 38, 43, 82] | Standard       |
| 2 | [5, 4, 3, 2, 1]            | [1, 2, 3, 4, 5]            | Reverse sorted |
| 3 | [1, 2, 3, 4, 5]            | [1, 2, 3, 4, 5]            | Already sorted |
| 4 | [1]                        | [1]                        | Single element |
| 5 | [3, 3, 1, 1, 2, 2]         | [1, 1, 2, 2, 3, 3]         | Duplicates     |

### Program 4: Find First and Last Position of Element in Sorted Array (LeetCode 34, Hard)

#### Problem Statement:

Given a sorted list of integers and a target value, find the starting and ending position of the target. If the target is not found, return [-1, -1]. Your solution must run in  $O(\log n)$  time.

#### Examples:

- Input: `nums = [5, 7, 7, 8, 8, 10]`, `target = 8`. Output: [3, 4].
- Input: `nums = [5, 7, 7, 8, 8, 10]`, `target = 6`. Output: [-1, -1].

#### Test Cases:

| # | nums                | target | Output   | Notes           |
|---|---------------------|--------|----------|-----------------|
| 1 | [5, 7, 7, 8, 8, 10] | 8      | [3, 4]   | Two occurrences |
| 2 | [5, 7, 7, 8, 8, 10] | 6      | [-1, -1] | Not found       |
| 3 | []                  | 0      | [-1, -1] | Empty list      |
| 4 | [1]                 | 1      | [0, 0]   | Single element  |
| 5 | [2, 2, 2, 2, 2]     | 2      | [0, 4]   | All same        |

## Program 5: Sort Array By Parity (LeetCode 905, Medium)

### Problem Statement:

Given a list of integers, move all even integers to the front and all odd integers to the back. Return any valid arrangement where all evens come before all odds.

### Examples:

- Input: [3, 1, 2, 4]. Output: [4, 2, 1, 3] (or any valid arrangement).

### Test Cases:

| # | Input        | Valid Output (evens first) | Notes             |
|---|--------------|----------------------------|-------------------|
| 1 | [3, 1, 2, 4] | e.g., [4, 2, 1, 3]         | Evens before odds |
| 2 | [0]          | [0]                        | Single even       |
| 3 | [1]          | [1]                        | Single odd        |
| 4 | [2, 4, 6]    | [2, 4, 6]                  | All even          |
| 5 | [1, 3, 5]    | [1, 3, 5]                  | All odd           |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Sort Colors (LeetCode 75, Medium)

#### Problem Statement:

Given a list with elements 0, 1, and 2 (representing red, white, and blue), sort them in-place in ONE pass. Do not use a library sort function.

#### Examples:

- Input: [2, 0, 2, 1, 1, 0]. Output: [0, 0, 1, 1, 2, 2].

### Test Cases:

| # | Input                       | Output                      | Notes          |
|---|-----------------------------|-----------------------------|----------------|
| 1 | [2, 0, 2, 1, 1, 0]          | [0, 0, 1, 1, 2, 2]          | LC Example 1   |
| 2 | [2, 0, 1]                   | [0, 1, 2]                   | Three elements |
| 3 | [0]                         | [0]                         | Single element |
| 4 | [1, 0]                      | [0, 1]                      | Two elements   |
| 5 | [2, 2, 2, 0, 0, 0, 1, 1, 1] | [0, 0, 0, 1, 1, 1, 2, 2, 2] | Groups         |

### Challenge 2: Search Insert Position (LeetCode 35, Medium)

#### Problem Statement:

Given a sorted list and a target value, return the index if the target is found. If not found, return the index where it would be inserted to keep the list sorted.

#### Examples:

- Input: [1, 3, 5, 6], target = 5. Output: 2 (found).
- Input: [1, 3, 5, 6], target = 2. Output: 1 (insert between 1 and 3).



### Test Cases:

| # | nums         | target | Output | Notes           |
|---|--------------|--------|--------|-----------------|
| 1 | [1, 3, 5, 6] | 5      | 2      | Found           |
| 2 | [1, 3, 5, 6] | 2      | 1      | Insert position |
| 3 | [1, 3, 5, 6] | 7      | 4      | Insert at end   |
| 4 | [1, 3, 5, 6] | 0      | 0      | Insert at start |
| 5 | [1]          | 0      | 0      | Single element  |

### Challenge 3: Merge Intervals (LeetCode 56, Hard)

#### Problem Statement:

Given a list of intervals [start, end], merge all overlapping intervals. Return the merged list.

#### Examples:

- Input: [[1,3],[2,6],[8,10],[15,18]]. Output: [[1,6],[8,10],[15,18]].
- Input: [[1,4],[4,5]]. Output: [[1,5]].

### Test Cases:

| # | Input                        | Output                 | Notes              |
|---|------------------------------|------------------------|--------------------|
| 1 | [[1,3],[2,6],[8,10],[15,18]] | [[1,6],[8,10],[15,18]] | Standard           |
| 2 | [[1,4],[4,5]]                | [[1,5]]                | Touching intervals |
| 3 | [[1,4],[0,4]]                | [[0,4]]                | Unsorted input     |
| 4 | [[1,4],[2,3]]                | [[1,4]]                | Contained interval |
| 5 | [[1,2]]                      | [[1,2]]                | Single interval    |

# CDP 410 - Dictionaries and Sets

## KEY CONCEPTS

### Opening

You have worked with lists, which are great for ordered collections of data. But what if you need to look something up by a name, not by a position? What if you need to count how many times each word appears in a document? What if you need to check whether an element exists, instantly, without scanning the entire list? That is what dictionaries and sets are for. This block introduces the two most important data structures for placement coding tests after lists.

### Dictionary Basics

A dictionary stores key-value pairs. Think of it as a real dictionary: you look up a word (the key) and get its definition (the value).

```
student = {"name": "Alice", "age": 20, "grade": "A"}
empty = {}
```

Creating a dict uses curly braces with key: value pairs separated by commas. An empty dict is just {}.

Accessing values:

```
print(student["name"])      # Alice
print(student["age"])      # 20
print(student["city"])     # KeyError! Key does not exist
print(student.get("city")) # None (no error)
print(student.get("city", "Unknown")) # "Unknown" (default value)
```

There are two ways to access values. Square brackets raise a `KeyError` if the key is missing. The `.get()` method returns `None` if missing, or a default value you specify. In most programs, use `.get()` because it is safer.

Adding and updating:

```
student["city"] = "Bangalore" # adds new key
student["age"] = 21           # updates existing key
del student["grade"]          # deletes a key
```

The same syntax adds or updates. Python does not distinguish between the two.

Key methods you will use constantly:

```
student.keys()    # dict_keys(["name", "age", "city"])
student.values()  # dict_values(["Alice", 21, "Bangalore"])
student.items()   # dict_items([("name", "Alice"), ("age", 21),
                              ("city", "Bangalore")])
```

### Dictionary Iteration

Iterating over a dict:

```
# Keys only (default)
for key in student:
    print(key, student[key])

# Key-value pairs (preferred)
for key, value in student.items():
    print(key, value)
```





Use `.items()` when you need both key and value. It is cleaner than accessing `d[key]` inside the loop.

Dict comprehension:

```
squares = {x: x**2 for x in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Same concept as list comprehension, but builds a dict.

## The Frequency Counting Pattern

```
freq = {}
for item in data:
    freq[item] = freq.get(item, 0) + 1
```

This is the single most important pattern in this block. Read it carefully. For each item: get its current count (0 if not seen before), add 1, store the new count. After the loop, `freq` contains the count of every unique item.

Example: counting letters in a word.

```
word = "banana"
freq = {}
for char in word:
    freq[char] = freq.get(char, 0) + 1
print(freq) # {"b": 1, "a": 3, "n": 2}
```

This pattern appears in nearly every DSA block from Block 502 onward. In placement tests, more than half the problems involving dicts use this exact pattern. Memorize it.

## Sets

A set is an unordered collection with no duplicates. Think of it as a dict with only keys and no values.

```
s = {1, 2, 3, 4, 5}
empty_set = set() # NOT {} (that is an empty dict!)
```

Creating a set uses curly braces, but an empty set **MUST** use `set()`. This is a Python trap. `{}` creates an empty dict, not an empty set.

Key operations:

```
s.add(6)          # adds element
s.remove(3)       # removes, raises KeyError if missing
s.discard(99)     # removes, NO error if missing
print(5 in s)     # True, O(1) lookup
```

The `in` operator on a set is  $O(1)$ , constant time. On a list, `in` is  $O(n)$ , it checks every element. This is the key advantage of sets: instant membership checking.

Set operations:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)      # Union: {1, 2, 3, 4, 5, 6}
print(a & b)      # Intersection: {3, 4}
print(a - b)      # Difference: {1, 2} (in a but not in b)
print(a ^ b)      # Symmetric difference: {1, 2, 5, 6}
```

These four operations are tested in interviews. Union combines both sets. Intersection finds common elements. Difference finds elements in one set but not the other. Symmetric difference finds elements in either but not both.



## Common Use Cases: Dict vs Set

When to use which:

Use a dict when you need to associate a key with a value: word counts, student records, configuration settings.

Use a set when you only care about existence: has this ID been seen before? What are the unique elements? What is the overlap between two collections?

Both provide  $O(1)$  lookup. Lists provide  $O(n)$  lookup. When you need to check 'does X exist?', prefer set or dict over list.

## Convenience Tools: Counter and defaultdict

Python provides shortcuts for common dict patterns. I am showing these AFTER the manual approach because you must know the manual approach for DSA and interviews. These shortcuts save keystrokes in production code.

```
from collections import Counter

words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
counter = Counter(words)
print(counter)  # Counter({'apple': 3, 'banana': 2, 'cherry': 1})
print(counter.most_common(2))  # [("apple", 3), ("banana", 2)]
```

Counter does the same thing as our manual frequency pattern, in one line. `.most_common(n)` returns the n most frequent items as a list of tuples.

```
from collections import defaultdict

dd = defaultdict(int)  # missing keys default to 0
for word in words:
    dd[word] += 1      # no .get() needed
print(dict(dd))  # {"apple": 3, "banana": 2, "cherry": 1}

dd2 = defaultdict(list)  # missing keys default to empty list
dd2["fruits"].append("apple")
dd2["fruits"].append("banana")
print(dict(dd2))  # {"fruits": ["apple", "banana"]}
```

`defaultdict(int)` means: if a key is missing, its default value is 0. So `dd[word] += 1` works without `.get()`. `defaultdict(list)` means: if a key is missing, its default value is an empty list. So `.append()` works without checking if the key exists.

Remember: Counter and defaultdict are shortcuts. If an interviewer asks you to implement frequency counting, use the manual `.get()` approach. If you are writing production code or solving a timed test, use Counter.

## IN-CLASS PROGRAMS

### Program 1: Word Frequency Counter (Easy)

#### Problem Statement:

A document analysis tool counts word frequencies. Given a sentence, split it into words, count each word's frequency using a plain dict with the `.get()` pattern, and print all words with their frequencies. Find and print the most frequent word. After solving with the manual dict approach, show the Counter version.



### Examples:

- Input: "the cat sat on the mat the cat". Output: the: 3, cat: 2, sat: 1, on: 1, mat: 1. Most frequent: the (3 times).
- Input: "hello". Output: hello: 1. Most frequent: hello (1 time).

### Test Cases:

| # | Input Sentence                   | Frequency Dict                   | Most Frequent | Notes                   |
|---|----------------------------------|----------------------------------|---------------|-------------------------|
| 1 | "the cat sat on the mat the cat" | the:3, cat:2, sat:1, on:1, mat:1 | the (3)       | Standard sentence       |
| 2 | "hello"                          | hello:1                          | hello (1)     | Single word             |
| 3 | "go go go stop stop go"          | go:4, stop:2                     | go (4)        | Two words, clear winner |
| 4 | "a b c a b c a"                  | a:3, b:2, c:2                    | a (3)         | Single-char words       |
| 5 | "data data data data data"       | data:5                           | data (5)      | All same word           |

### Program 2: Two Sum (LeetCode 1, Medium)

#### Problem Statement:

Given a list of integers `nums` and an integer `target`, return the indices of the two numbers that add up to `target`. You may assume that each input has exactly one solution, and you may not use the same element twice. Return the answer as a list of two indices.

#### Examples:

- Input: `nums = [2, 7, 11, 15]`, `target = 9`. Output: `[0, 1]`. ( $2 + 7 = 9$ )
- Input: `nums = [3, 2, 4]`, `target = 6`. Output: `[1, 2]`. ( $2 + 4 = 6$ )

### Test Cases:

| # | nums                 | target | Output | Notes                    |
|---|----------------------|--------|--------|--------------------------|
| 1 | [2, 7, 11, 15]       | 9      | [0, 1] | First two elements       |
| 2 | [3, 2, 4]            | 6      | [1, 2] | Not the first pair       |
| 3 | [3, 3]               | 6      | [0, 1] | Duplicate values         |
| 4 | [1, 5, 3, 7, 2]      | 9      | [3, 4] | Non-adjacent pair (7+2)  |
| 5 | [-1, -2, -3, -4, -5] | -8     | [2, 4] | Negative numbers (-3+-5) |

### Program 3: Student Grade Analyzer with Dict (Medium)

#### Problem Statement:

A university stores student records in a dict where each key is a student name and the value is a list of marks in 5 subjects. Given N students with their names and marks:

- Build the dict from input.
- Find each student's average.
- Find the topper (student with the highest average).



(d) Find all students who passed all subjects (scored  $\geq 50$  in every subject).

(e) Find the subject (by index, 1 to 5) with the lowest class average.

#### Examples:

- Input: 3 students. Alice: [80, 70, 90, 85, 75], Bob: [60, 55, 45, 70, 65], Charlie: [90, 95, 88, 92, 85].
- Averages: Alice=80.0, Bob=59.0, Charlie=90.0. Topper: Charlie (90.0). Passed all: Alice, Charlie (Bob failed subject 3 with 45). Subject averages: S1=76.67, S2=73.33, S3=74.33, S4=82.33, S5=75.0. Lowest: Subject 2 (73.33).

#### Test Cases:

| # | Students                                                                     | Topper         | Passed All       | Lowest Subject    | Notes                       |
|---|------------------------------------------------------------------------------|----------------|------------------|-------------------|-----------------------------|
| 1 | Alice:[80,70,90,85,75],<br>Bob:[60,55,45,70,65],<br>Charlie:[90,95,88,92,85] | Charlie (90.0) | Alice, Charlie   | Subject 2 (73.33) | Standard, Bob fails one     |
| 2 | Dan:[50,50,50,50,50]                                                         | Dan (50.0)     | Dan              | Subject 1 (50.0)  | Single student, all 50s     |
| 3 | Eve:[100,100,100,100,100],<br>Frank:[49,49,49,49,49]                         | Eve (100.0)    | Eve              | Subject 1 (74.5)  | One perfect, one all-fail   |
| 4 | Grace:[80,90,70,60,50],<br>Hank:[50,60,70,90,80],<br>Ivy:[70,70,70,70,70]    | Grace (70.0)   | Grace, Hank, Ivy | Subject 1 (66.67) | Tie in averages, first wins |
| 5 | Jack:[30,80,90,40,70],<br>Kate:[70,30,60,80,40]                              | Jack (62.0)    | (none)           | Subject 1 (50.0)  | No one passes all           |

#### Program 4: Group Anagrams (LeetCode 49, Hard)

##### Problem Statement:

Given a list of strings, group the anagrams together. Two words are anagrams if they contain the same characters in the same frequencies (e.g., "eat" and "tea"). Return a list of groups, where each group is a list of anagram strings. You can return the groups in any order.

##### Examples:

- Input: ["eat", "tea", "tan", "ate", "nat", "bat"]. Output: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]].
- Input: ["a"]. Output: [["a"]].

#### Test Cases:

| # | Input                                      | Output Groups                                      | Notes               |
|---|--------------------------------------------|----------------------------------------------------|---------------------|
| 1 | ["eat", "tea", "tan", "ate", "nat", "bat"] | [[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]] | Standard LC example |
| 2 | [""]                                       | [[[""]]]                                           | Single empty string |
| 3 | ["a"]                                      | [[["a"]]]                                          | Single character    |
| 4 | ["abc", "bca", "cab", "xyz", "zyx"]        | [[["abc", "bca", "cab"], ["xyz", "zyx"]]]          | Two groups          |



|   |                            |                                  |                        |
|---|----------------------------|----------------------------------|------------------------|
| 5 | ["no","on","is","si","am"] | [["no","on"],["is","si"],["am"]] | Multiple 2-char groups |
|---|----------------------------|----------------------------------|------------------------|

## Program 5: Duplicate Finder with Set (Medium)

### Problem Statement:

A data cleaning system processes customer IDs that may contain duplicates. Given a list of IDs, perform three tasks:

- Find all duplicate IDs (IDs that appear more than once).
- Create a new list with duplicates removed, preserving the order of first occurrences.
- Given two lists A and B, find IDs that appear in A but not in B using set difference.

### Examples:

- Input IDs: [101, 202, 303, 101, 404, 202, 505]. List A: [101, 202, 303, 404]. List B: [202, 404, 606].
- (a) Duplicates: [101, 202]. (b) Deduped: [101, 202, 303, 404, 505]. (c) In A not B: [101, 303].

### Test Cases:

| # | IDs                           | List A            | List B        | Duplicates | Deduped               | A - B     | Notes             |
|---|-------------------------------|-------------------|---------------|------------|-----------------------|-----------|-------------------|
| 1 | [101,202,303,101,404,202,505] | [101,202,303,404] | [202,404,606] | [101,202]  | [101,202,303,404,505] | [101,303] | Standard          |
| 2 | [1,2,3,4,5]                   | [1,2,3]           | [4,5,6]       | []         | [1,2,3,4,5]           | [1,2,3]   | No dups, disjoint |
| 3 | [7,7,7,7]                     | [7]               | [7]           | [7]        | [7]                   | []        | All same          |
| 4 | [10]                          | [10,20]           | []            | []         | [10]                  | [10,20]   | Single, B empty   |
| 5 | [5,3,5,1,3,1,5]               | [1,2,3,4,5]       | [2,4]         | [5,3,1]    | [5,3,1]               | [1,3,5]   | Multiple dups     |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Valid Anagram (LeetCode 242, Medium)

#### Problem Statement:

Given two strings *s* and *t*, return `True` if *t* is an anagram of *s*, and `False` otherwise. An anagram uses all the original letters exactly once.

#### Examples:

- Input: *s* = "anagram", *t* = "nagaram". Output: `True`.
- Input: *s* = "rat", *t* = "car". Output: `False`.



## Test Cases:

| # | s         | t         | Output | Notes             |
|---|-----------|-----------|--------|-------------------|
| 1 | "anagram" | "nagaram" | True   | Standard anagram  |
| 2 | "rat"     | "car"     | False  | Not anagram       |
| 3 | ""        | ""        | True   | Both empty        |
| 4 | "a"       | "a"       | True   | Single same char  |
| 5 | "ab"      | "a"       | False  | Different lengths |

## Challenge 2: Contains Duplicate (LeetCode 217, Medium)

### Problem Statement:

Given an integer list `nums`, return `True` if any value appears at least twice in the list, and `False` if every element is distinct.

### Examples:

- Input: [1, 2, 3, 1]. Output: True.
- Input: [1, 2, 3, 4]. Output: False.

## Test Cases:

| # | nums                  | Output | Notes               |
|---|-----------------------|--------|---------------------|
| 1 | [1, 2, 3, 1]          | True   | Duplicate at end    |
| 2 | [1, 2, 3, 4]          | False  | All unique          |
| 3 | [1,1,1,3,3,4,3,2,4,2] | True   | Multiple duplicates |
| 4 | [1]                   | False  | Single element      |
| 5 | [0, 0]                | True   | Two zeros           |

## Challenge 3: Intersection of Two Arrays II (LeetCode 350, Medium)

### Problem Statement:

Given two integer lists `nums1` and `nums2`, return a list of their intersection. Each element in the result must appear as many times as it shows in both lists. The result can be in any order.

### Examples:

- Input: `nums1 = [1,2,2,1]`, `nums2 = [2,2]`. Output: `[2,2]`.
- Input: `nums1 = [4,9,5]`, `nums2 = [9,4,9,8,4]`. Output: `[4,9]`.

## Test Cases:

| # | nums1     | nums2       | Output | Notes                 |
|---|-----------|-------------|--------|-----------------------|
| 1 | [1,2,2,1] | [2,2]       | [2,2]  | Standard LC example   |
| 2 | [4,9,5]   | [9,4,9,8,4] | [4,9]  | Standard LC example 2 |
| 3 | [1,2,3]   | [4,5,6]     | []     | No intersection       |



|   |         |       |       |                      |
|---|---------|-------|-------|----------------------|
| 4 | [1]     | [1]   | [1]   | Single element match |
| 5 | [3,3,3] | [3,3] | [3,3] | Min count applies    |

# CDP 411 - OOP 1: Classes, Objects, Constructors, Encapsulation

## KEY CONCEPTS

### Opening

In the previous block, you stored student data in a dict: `{name: [marks]}`. It worked, but there was no structure. Anyone could add the wrong type of value. There was no way to attach behavior to the data, no way to say 'a student can compute their own grade.' Object-Oriented Programming solves this. You define a blueprint (a class), and every object created from that blueprint has the same structure and the same capabilities. This block teaches you to design your own data types.

### Defining a Class

A class is a blueprint for creating objects.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")
```

Three things to notice. First, `class Student:` with a colon and an indented body. Second, `__init__` is the constructor. It runs automatically when you create an object. Third, every method takes `self` as the first parameter. `self` refers to the specific object being created or used.

Creating objects:

```
s1 = Student("Alice", 20)
s2 = Student("Bob", 22)
s1.display() # Name: Alice, Age: 20
s2.display() # Name: Bob, Age: 22
```

No `new` keyword (unlike Java). You just call the class name like a function. Python calls `__init__` automatically and passes the new object as `self`.

### The `self` Parameter

```
When you write: s1 = Student("Alice", 20)
Python calls:   Student.__init__(s1, "Alice", 20)
                  ^
                  self = s1
```

Python passes the object being created as the first argument to every method. You name it `self` by convention. Inside `__init__`, `self.name = name` means 'store the name on THIS specific object.' When `s1.display()` is called, `self` inside `display` is `s1`.

This is the most common confusion point. If you forget `self` in a method definition, you will get: `TypeError: display() takes 0 positional arguments but 1 was given`. Python tried to pass the object, but your method did not accept it.

If you forget `self`. when accessing an attribute inside a method, you will get a `NameError` or use a local variable instead of the object's attribute. Always use `self.attribute_name` inside methods.





## Instance Variables vs Class Variables

```
class Student:
    student_count = 0          # class variable (shared by all instances)

    def __init__(self, name):
        self.name = name      # instance variable (unique to each object)
        Student.student_count += 1
```

Instance variables (created with `self.something`) belong to one specific object. Class variables (declared directly in the class body) are shared across all instances.

### Demonstrate live:

```
s1 = Student("Alice")
s2 = Student("Bob")
print(Student.student_count)  # 2 (shared)
print(s1.name)                # Alice (instance-specific)
print(s2.name)                # Bob (instance-specific)
```

Modify the class variable through the class name: `Student.student_count = 0` resets for all. This distinction (instance vs class) is a common interview question.

## Class Methods

```
class Student:
    student_count = 0

    def __init__(self, name):
        self.name = name
        Student.student_count += 1

    @classmethod
    def get_student_count(cls):
        return cls.student_count
```

A class method uses the `@classmethod` decorator. Its first parameter is `cls` (the class itself), not `self` (an instance). Use it for operations on the class, not on a specific object.

`Student.get_student_count()` returns the count without needing any specific student object.

This is Python's equivalent of Java's `static` methods.

## Encapsulation: Naming Conventions

In Java, you use `private` and `public` keywords to control access. Python does not have access modifiers. Instead, Python uses naming conventions.

```
self.name          # public (anyone can access)
self._salary       # protected (convention: internal use only)
self.__secret      # name-mangled (hard to access accidentally)
```

Single underscore `_` means 'this is internal, please do not access it from outside.' But Python does not enforce this. You can still access `obj._salary`. It is a gentleman's agreement.

Double underscore `__` triggers name mangling. Python renames `self.__secret` to `self._ClassName__secret`. This makes accidental access harder, but it is still possible. It is not true privacy.

Python's philosophy: we are all consenting adults here. The language trusts you to follow conventions.



## Properties: @property Decorator

Properties let you control access to attributes without changing how they are used.

```
class Employee:
    def __init__(self, emp_id, name):
        self._emp_id = emp_id    # stored with underscore
        self.name = name

    @property
    def emp_id(self):            # getter
        return self._emp_id

    # No setter defined = read-only
```

With `@property`, you access `emp.emp_id` like a regular attribute (no parentheses), but it is actually a method call. Because we defined no setter, attempting `emp.emp_id = 'NEW'` raises `AttributeError`. This makes `emp_id` read-only.

To add a setter with validation:

```
@property
def name(self):
    return self._name

@name.setter
def name(self, value):
    if not value.strip():
        raise ValueError("Name cannot be empty")
    self._name = value
```

The setter runs whenever you assign to `emp.name`. It validates before storing. This is Python's way of doing controlled access, cleaner than Java's explicit getter/setter methods.

## The `__str__` Method

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"Student({self.name}, {self.age})"
```

Without `__str__`, printing an object gives something like `<__main__.Student object at 0x7f...>`. Useless. With `__str__`, `print(s)` shows `Student(Alice, 20)`. Define `__str__` in every class you write.

There is also `__repr__` for developer-friendly debugging output. For this block, focus on `__str__`.

## Contrast with Java

Quick comparison. In Java, a Student class needs: `public` class, access modifiers on every field, type declarations, explicit getter/setter methods, `new` keyword for object creation, and `toString()` override. In Python: `class`, `self`, `__init__`, and you are done. But Python's flexibility means you must be disciplined. There is no compiler checking your types or access levels.



## IN-CLASS PROGRAMS

### Program 1: Bank Account Manager (Easy)

#### Problem Statement:

Design a `BankAccount` class for a banking system. The class should have:

- `__init__(self, account_number, holder_name, balance)`: initializes the account.
- `deposit(amount)`: adds amount to balance. Validates amount > 0.
- `withdraw(amount)`: subtracts amount from balance. Validates amount > 0 and ensures balance does not drop below minimum of 500.
- `display()`: returns a formatted string with account info.
- `__str__`: returns a developer-friendly string representation.

Create 3 accounts, perform deposits and withdrawals, and print each account.

#### Examples:

- Create account ACC001 for Alice with balance 5000. Deposit 1000 (new balance 6000). Withdraw 2000 (new balance 4000). Withdraw 4000 (FAILED, would leave balance below 500).

#### Test Cases:

| # | Operation                   | Input                             | Result           | Notes                 |
|---|-----------------------------|-----------------------------------|------------------|-----------------------|
| 1 | Create + deposit            | ACC001, Alice, 5000, deposit 1000 | Balance: 6000.00 | Basic deposit         |
| 2 | Withdraw (valid)            | Balance 6000, withdraw 2000       | Balance: 4000.00 | Leaves above 500      |
| 3 | Withdraw (exactly 500 left) | Balance 1500, withdraw 1000       | Balance: 500.00  | Boundary: exactly min |
| 4 | Withdraw (below min)        | Balance 1000, withdraw 600        | FAILED           | Would leave 400 < 500 |
| 5 | Invalid deposit             | deposit(-100)                     | INVALID          | Negative amount       |

### Program 2: Student Management System (Medium)

#### Problem Statement:

Design a `Student` class:

- `__init__(self, roll_no, name, marks)` where marks is a list of 5 integers.
- `get_average()`: returns the average of marks as a float.
- `get_grade()`: calls `get_average()` internally and returns a grade string (A+ for 90+, A for 80-89, B for 70-79, C for 60-69, D for 50-59, F for below 50).
- `is_passed()`: returns True if grade is not F.
- Class variable `student_count` initialized to 0, incremented in `__init__`.
- `@classmethod get_student_count(cls)`: returns the current count.

In main, create a list of 5 `Student` objects. Write a function `find_topper(students)` that returns the student with the highest average.

#### Examples:



- Students: Alice [80,70,90,85,75] (avg 80.0, grade A), Bob [60,55,45,70,65] (avg 59.0, grade D), Charlie [90,95,88,92,85] (avg 90.0, grade A+). Topper: Charlie. Student count: 3.

### Test Cases:

| # | Student | Marks            | Average | Grade | Passed | Notes          |
|---|---------|------------------|---------|-------|--------|----------------|
| 1 | Alice   | [80,70,90,85,75] | 80.0    | A     | True   | Standard       |
| 2 | Bob     | [60,55,45,70,65] | 59.0    | D     | True   | Low but passes |
| 3 | Charlie | [90,95,88,92,85] | 90.0    | A+    | True   | Topper         |
| 4 | Dan     | [40,30,35,45,50] | 40.0    | F     | False  | Fails          |
| 5 | Eve     | [50,50,50,50,50] | 50.0    | D     | True   | Boundary D     |

### Program 3: Design Parking System (LeetCode 1603, Medium)

#### Problem Statement:

Design a parking system for a parking lot that has three kinds of parking spaces: big, medium, and small, with a fixed number of slots for each size.

Implement the `ParkingSystem` class:

- `__init__(self, big, medium, small)`: initializes with the number of slots for each size.
- `add_car(car_type)`: checks whether there is a parking space available for the given car type. `car_type` can be 1 (big), 2 (medium), or 3 (small). A car can only park in a space of its corresponding size. Return `True` if a space is available, `False` otherwise.

#### Examples:

- `ParkingSystem(1, 1, 0)`. `add_car(1)` returns `True` (1 big slot, now 0). `add_car(2)` returns `True` (1 medium slot, now 0). `add_car(3)` returns `False` (0 small slots). `add_car(1)` returns `False` (0 big slots left).

### Test Cases:

| # | Init          | Operations                                                                                                     | Results                     | Notes               |
|---|---------------|----------------------------------------------------------------------------------------------------------------|-----------------------------|---------------------|
| 1 | (1,1,0)       | <code>add_car(1)</code> ,<br><code>add_car(2)</code> ,<br><code>add_car(3)</code> ,<br><code>add_car(1)</code> | True, True, False,<br>False | Standard LC example |
| 2 | (0,0,0)       | <code>add_car(1)</code> ,<br><code>add_car(2)</code> ,<br><code>add_car(3)</code>                              | False, False, False         | No slots            |
| 3 | (3,3,3)       | <code>add_car(1)</code> x4,<br><code>add_car(2)</code> x4                                                      | T,T,T,F, T,T,T,F            | Fill all slots      |
| 4 | (1,0,0)       | <code>add_car(1)</code> ,<br><code>add_car(1)</code>                                                           | True, False                 | Single big slot     |
| 5 | (100,100,100) | <code>add_car(1)</code> ,<br><code>add_car(2)</code> ,<br><code>add_car(3)</code>                              | True, True, True            | Large capacity      |

## Program 4: Employee Payroll System (Hard)

### Problem Statement:

Design an `Employee` class for a payroll system:

- `__init__(self, emp_id, name, basic_salary, department="General")`: default department is "General".
- `calculate_hra()`: returns 20% of basic salary.
- `calculate_da()`: returns 50% of basic salary.
- `calculate_pf()`: returns 12% of basic salary.
- `calculate_tax()`: returns 10% of gross salary (basic + HRA + DA).
- `get_net_salary()`: computes gross, subtracts PF and tax, returns net.
- `__str__`: returns formatted payslip string.
- Use `@property` for `emp_id` (read-only, no setter).
- Use `self._basic_salary` naming convention for "protected" attributes.

In main, create employees with both full arguments and default department.

### Examples:

- Employee E001, Alice, basic 50000, dept Engineering. HRA=10000, DA=25000, Gross=85000, PF=6000, Tax=8500, Net=70500.
- Employee E002, Bob, basic 30000 (default dept "General"). Net=42300.

### Test Cases:

| # | emp_id | Name    | Basic  | Dept        | HRA   | DA    | PF    | Tax   | Net    | Notes                 |
|---|--------|---------|--------|-------------|-------|-------|-------|-------|--------|-----------------------|
| 1 | E001   | Alice   | 50000  | Engineering | 10000 | 25000 | 6000  | 8500  | 70500  | Full args             |
| 2 | E002   | Bob     | 30000  | General     | 6000  | 15000 | 3600  | 5100  | 42300  | Default dept          |
| 3 | E003   | Charlie | 100000 | Management  | 20000 | 50000 | 12000 | 17000 | 141000 | High salary           |
| 4 | E004   | Dan     | 10000  | Support     | 2000  | 5000  | 1200  | 1700  | 14100  | Low salary            |
| 5 | E005   | Eve     | 75000  | General     | 15000 | 37500 | 9000  | 12750 | 105750 | Read-only emp_id test |

## Program 5: Design HashMap Simplified (LeetCode 706, Medium)

### Problem Statement:

Design a simplified HashMap (without collision handling) that supports three operations:

- `put(key, value)`: inserts or updates a key-value pair. Keys are integers in range [0, 999].
- `get(key)`: returns the value associated with the key, or -1 if not found.
- `remove(key)`: removes the key-value pair. Subsequent `get` should return -1.

Use a fixed-size list of 1000 as the underlying storage with direct indexing (key = index).



### Examples:

- put(1, 1), put(2, 2). get(1) returns 1. get(3) returns -1. put(2, 1) updates. get(2) returns 1. remove(2). get(2) returns -1.

### Test Cases:

| # | Operations                                                              | Results           | Notes               |
|---|-------------------------------------------------------------------------|-------------------|---------------------|
| 1 | put(1,1), put(2,2), get(1), get(3), put(2,1), get(2), remove(2), get(2) | _,_,1,-1,_,1,_,-1 | Standard LC example |
| 2 | put(0,100), get(0), remove(0), get(0)                                   | _,100,_,-1        | Key 0 edge case     |
| 3 | put(999,42), get(999), get(500)                                         | _,42,-1           | High key            |
| 4 | put(5,10), put(5,20), put(5,30), get(5)                                 | _,_,_,30          | Multiple updates    |
| 5 | remove(7), get(7)                                                       | _,-1              | Remove non-existent |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Library Book System (Medium)

#### Problem Statement:

Design two classes:

- **Book** with attributes: title, author, isbn, is\_available (default True). Methods: checkout() (marks unavailable, fails if already checked out), return\_book() (marks available, fails if not checked out), display() (returns formatted info).
- **Library** with a list of books. Methods: add\_book(book) (adds to list), search\_by\_title(title) (returns books whose title contains the search string, case-insensitive), display\_available() (returns list of available books).

This tests object composition: Library has-a list of Books.

### Test Cases:

| # | Operation                                    | Result        | Notes                     |
|---|----------------------------------------------|---------------|---------------------------|
| 1 | Add 3 books, checkout one, display available | 2 available   | Basic flow                |
| 2 | Checkout already checked-out book            | Error message | Double checkout           |
| 3 | Return book not checked out                  | Error message | Double return             |
| 4 | Search "Python" in 3 books (2 match)         | 2 results     | Partial, case-insensitive |
| 5 | Search "Java" (no match)                     | 0 results     | No match                  |

## Challenge 2: Range Sum Query Immutable (LeetCode 303, Medium)

### Problem Statement:

Design a `NumArray` class:

- `__init__(self, nums)`: initializes with an integer list. Computes prefix sum array during construction.
- `sum_range(left, right)`: returns the sum of elements between indices `left` and `right` inclusive, in  $O(1)$  time.

### Examples:

- `NumArray([-2, 0, 3, -5, 2, -1])`. `sum_range(0, 2)` returns 1. `sum_range(2, 5)` returns -1. `sum_range(0, 5)` returns -3.

### Test Cases:

| # | nums             | Query          | Result | Notes                |
|---|------------------|----------------|--------|----------------------|
| 1 | [-2,0,3,-5,2,-1] | sum_range(0,2) | 1      | Standard LC example  |
| 2 | [-2,0,3,-5,2,-1] | sum_range(2,5) | -1     | Middle to end        |
| 3 | [1,2,3,4,5]      | sum_range(0,4) | 15     | Entire array         |
| 4 | [1,2,3,4,5]      | sum_range(2,2) | 3      | Single element       |
| 5 | [5]              | sum_range(0,0) | 5      | Single element array |

## Challenge 3: Implement Stack using Queues (LeetCode 225, Medium)

### Problem Statement:

Implement a stack (LIFO) using a list. Support:

- `push(x)`: pushes element to top.
- `pop()`: removes and returns the top element.
- `top()`: returns the top element without removing.
- `empty()`: returns True if stack is empty.

### Examples:

- `push(1), push(2)`. `top()` returns 2. `pop()` returns 2. `empty()` returns False. `pop()` returns 1. `empty()` returns True.

### Test Cases:

| # | Operations                                     | Results      | Notes               |
|---|------------------------------------------------|--------------|---------------------|
| 1 | push(1), push(2), top(), pop(), empty()        | _,_,2,False  | Standard LC example |
| 2 | push(5), top(), pop(), empty()                 | _,5,5,True   | Single element      |
| 3 | push(1), push(2), push(3), pop(), pop(), pop() | _,_,_,3,2,1  | LIFO order verified |
| 4 | empty(), push(10), empty()                     | True,_,False | Empty check         |
| 5 | push(-1), push(0), top(), pop()                | _,_,0,0      | Negative and zero   |



# CDP 412 - OOP 2: Inheritance, Polymorphism, Abstract Classes + Exception Handling

## KEY CONCEPTS

### Opening

In Block 411, you built standalone classes. A BankAccount, a Student, an Employee. Each class was independent. But in the real world, types share common properties. A Manager is a type of Employee. A Circle is a type of Shape. A SavingsAccount is a type of Account. Inheritance lets you define a general class and create specialized versions of it. This block teaches inheritance, polymorphism, abstract classes, and exception handling.

### Inheritance

Inheritance means one class acquires the attributes and methods of another.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks"

class Cat(Animal):
    def speak(self):
        return f"{self.name} meows"
```

Dog IS-A Animal. Cat IS-A Animal. The parentheses `class Dog(Animal)` mean Dog inherits from Animal. Dog gets `name` and `speak()` from Animal. But Dog overrides `speak()` with its own version. The `super()` function calls the parent class:

```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name) # calls Animal.__init__
        self.breed = breed     # Dog-specific attribute
```

When Dog has its own `__init__`, it MUST call `super().__init__()` to initialize the parent's attributes. Otherwise, `self.name` will not exist.

### Polymorphism

Polymorphism means the same method name behaves differently depending on the object.

```
animals = [Dog("Rex"), Cat("Whiskers"), Dog("Buddy")]

for animal in animals:
    print(animal.speak())
```

Output: 'Rex barks', 'Whiskers meows', 'Buddy barks'. The same `.speak()` call produces different results. Python does not check the type. It just calls whatever `speak()` method the object has. This is polymorphism.





`isinstance()` checks:

```
d = Dog("Rex")
print(isinstance(d, Dog))      # True
print(isinstance(d, Animal))   # True (Dog IS-A Animal)
print(isinstance(d, Cat))      # False
```

## Method Resolution and `super()`

When you call a method, Python looks for it in this order: the object's class first, then the parent class, then the grandparent, and so on. This is the Method Resolution Order (MRO).

When overriding a method, if you want to extend (not replace) the parent's behavior, call `super()` inside your override:

```
class Manager(Employee):
    def calculate_salary(self):
        base = super().calculate_salary()    # get parent's result
        return base + self.bonus             # add bonus on top
```

## Abstract Classes

Sometimes the parent class should not be instantiated directly. You cannot create a generic 'Shape'. You must create a specific Circle or Rectangle.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

    def describe(self):
        return f"{type(self).__name__} with area {self.area():.2f}"
```

Three things. First, `Shape(ABC)` inherits from `ABC` (Abstract Base Class). Second, `@abstractmethod` marks methods that subclasses **MUST** implement. Third, `describe()` is a concrete method that calls the abstract `area()`. It works because by the time `describe()` runs, the object is a concrete subclass (Circle, Rectangle) that has `area()` implemented.

### Demonstrate live:

```
try:
    s = Shape()
except TypeError as e:
    print(e)
# Output: Can't instantiate abstract class Shape with abstract methods area,
# perimeter
```

You cannot create a Shape directly. Python enforces this. This is Python's equivalent of Java's abstract class and interface. Python does not have a separate interface keyword. `ABC` serves both purposes.

## Duck Typing

Python does not require formal inheritance for polymorphism. If an object has the method you call, it works.



```
class FakeAnimal:
    def speak(self):
        return "I am not an Animal, but I can speak"

# Works! Python does not check if FakeAnimal IS-A Animal.
# It just checks: does the object have speak()? Yes. Call it.
```

This is duck typing: if it walks like a duck and quacks like a duck, it is a duck. Python cares about behavior, not lineage. Know this concept exists. You will not use it in placement tests, but it explains why Python's OOP feels more flexible than Java's.

## Multiple Inheritance

Python supports inheriting from multiple parents: `class C(A, B):`. Python uses a Method Resolution Order (MRO) to decide which parent method to call first. `C.__mro__` shows the order.

Advice: avoid multiple inheritance in practice. The diamond problem (two parents share a grandparent) creates confusing behavior. Single inheritance covers every placement test scenario.

## Exception Handling

Exceptions are Python's way of handling errors without crashing the program.

```
try:
    # Code that might fail
    result = 10 / 0
except ZeroDivisionError as e:
    # Handle the specific error
    print(f"Error: {e}")
except ValueError as e:
    # Handle a different error
    print(f"Invalid value: {e}")
else:
    # Runs ONLY if no exception occurred
    print(f"Result: {result}")
finally:
    # Runs ALWAYS, with or without exception
    print("Done")
```

Four parts. `try:` the risky code. `except:` catches specific errors. `else:` runs only on success (no exception). `finally:` runs always, even if an exception occurred.

The `else` block is the one students forget. It separates the happy path (success) from error handling (except). Use it when you want to do something only if no error occurred.

Common exceptions:

|                                |                                                    |
|--------------------------------|----------------------------------------------------|
| <code>ValueError</code>        | - wrong value type ( <code>int("abc")</code> )     |
| <code>TypeError</code>         | - wrong operation on type ( <code>"5" + 3</code> ) |
| <code>IndexError</code>        | - list index out of range                          |
| <code>KeyError</code>          | - dict key not found                               |
| <code>ZeroDivisionError</code> | - division by zero                                 |
| <code>FileNotFoundError</code> | - file does not exist                              |

Raising exceptions:

```
def withdraw(amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if amount > balance:
        raise InsufficientFundsError("Not enough money")
```



Custom exceptions:

```
class InsufficientFundsError(Exception):
    pass
```

That is it. Inherit from `Exception`, add `pass`. The class name IS the error message identifier. You can add custom attributes if needed, but `pass` is sufficient for most cases.

Exception handling is a tool, not a deep concept. Learn try-except-finally, learn raise, learn custom exceptions, move on.

## IN-CLASS PROGRAMS

### Program 1: Vehicle Hierarchy (Easy)

#### Problem Statement:

Design a vehicle class hierarchy:

- `Vehicle` (parent) with `__init__(self, make, model, year)` and method `display_info()`.
- `Car(Vehicle)` with additional attribute `num_doors`, overrides `display_info()` to include door count.
- `Bike(Vehicle)` with additional attribute `engine_cc`, overrides `display_info()` to include engine size.

Create a list of mixed `Vehicle`, `Car`, and `Bike` objects. Iterate and call `display_info()` on each (polymorphism). Demonstrate `isinstance()` checks.

#### Examples:

- `Car("Toyota", "Camry", 2023, 4).display_info()` returns "Car: 2023 Toyota Camry, 4 doors".
- `Bike("Honda", "CBR", 2022, 600).display_info()` returns "Bike: 2022 Honda CBR, 600cc".

#### Test Cases:

| # | Object                                         | display_info() Output             | Notes            |
|---|------------------------------------------------|-----------------------------------|------------------|
| 1 | <code>Car("Toyota", "Camry", 2023, 4)</code>   | "Car: 2023 Toyota Camry, 4 doors" | Car override     |
| 2 | <code>Bike("Honda", "CBR", 2022, 600)</code>   | "Bike: 2022 Honda CBR, 600cc"     | Bike override    |
| 3 | <code>Vehicle("Generic", "Model", 2020)</code> | "Vehicle: 2020 Generic Model"     | Parent class     |
| 4 | <code>isinstance(car, Vehicle)</code>          | True                              | Car IS-A Vehicle |
| 5 | <code>isinstance(car, Bike)</code>             | False                             | Car is not Bike  |

### Program 2: Employee Salary System with Inheritance (Medium)

#### Problem Statement:

Design an employee salary system with inheritance:

- `Employee` (parent): `__init__(self, name, emp_id, basic_salary)`, method `calculate_salary()` returns `basic_salary`.
- `Manager(Employee)`: additional bonus. Overrides `calculate_salary()` to return `basic + bonus`.



- `Developer(Employee):` additional `overtime_hours` and `overtime_rate`. Overrides `calculate_salary()` to return `basic + (overtime_hours * overtime_rate)`.
- `Intern(Employee):` additional `stipend`. Overrides `calculate_salary()` to return `stipend` only (ignores `basic_salary`).

Use `super().__init__()` in every subclass. Create a mixed list of employees, loop and print each person's salary. Polymorphic `calculate_salary()` resolves to the correct subclass version.

#### Examples:

- `Manager("Alice", "M001", 80000, 20000).calculate_salary()` returns 100000.
- `Developer("Bob", "D001", 60000, 20, 500).calculate_salary()` returns 70000.
- `Intern("Charlie", "I001", 15000).calculate_salary()` returns 15000.

#### Test Cases:

| # | Type       | Name    | Basic    | Extra          | Salary                      | Notes            |
|---|------------|---------|----------|----------------|-----------------------------|------------------|
| 1 | Manager    | Alice   | 80000    | bonus=20000    | 100000                      | basic + bonus    |
| 2 | Developer  | Bob     | 60000    | 20hrs * 500/hr | 70000                       | basic + overtime |
| 3 | Intern     | Charlie | N/A      | stipend=15000  | 15000                       | stipend only     |
| 4 | Employee   | Dan     | 50000    | N/A            | 50000                       | base class       |
| 5 | Mixed list | A,B,C,D | (varied) | (varied)       | 100000, 63000, 12000, 45000 | Polymorphic      |

### Program 3: Abstract Shape Calculator (Medium)

#### Problem Statement:

Design an abstract shape hierarchy:

- `Shape(ABC)` with `@abstractmethod area()`, `@abstractmethod perimeter()`, and concrete method `describe()` that returns `f"{type(self).__name__} with area {self.area():.2f}"`.
- `Circle(Shape)` with `radius`. Implements `area` ( $\pi * r^2$ ) and `perimeter` ( $2 * \pi * r$ ).
- `Rectangle(Shape)` with `length` and `width`. Implements `area` ( $l * w$ ) and `perimeter` ( $2 * (l + w)$ ).
- `Triangle(Shape)` with `base`, `height`, and three sides. Implements `area` ( $0.5 * base * height$ ) and `perimeter` (sum of sides).

Create a list of shapes, call `describe()` on each. Write `find_largest(shapes)` to return the shape with the largest area. Demonstrate that `Shape()` raises `TypeError`.

#### Examples:

- `Circle(5):` area=78.54, perimeter=31.42, describe="Circle with area 78.54".
- `Rectangle(10, 5):` area=50.00, describe="Rectangle with area 50.00".

## Test Cases:

| # | Shape                                      | Params                      | Area      | Perimeter         | Notes              |
|---|--------------------------------------------|-----------------------------|-----------|-------------------|--------------------|
| 1 | Circle                                     | r=5                         | 78.54     | 31.42             | $\pi * r^2$        |
| 2 | Rectangle                                  | 10 x 5                      | 50.00     | 30.00             | $l * w$            |
| 3 | Triangle                                   | base=6, h=4,<br>sides 5,5,6 | 12.00     | 16.00             | $0.5 * b * h$      |
| 4 | Shape()                                    | N/A                         | TypeError | TypeError         | Cannot instantiate |
| 5 | find_largest([Circle(3), Rect(10,5), Tri]) | N/A                         | Rectangle | Largest area = 50 | Rectangle wins     |

## Program 4: Safe Calculator with Custom Exceptions (Hard)

### Problem Statement:

Design a calculator with robust error handling:

- Custom exceptions: `InvalidOperationError(Exception)` and `DivisionByZeroError(Exception)`.
- Class `Calculator` with methods: `add(a, b)`, `subtract(a, b)`, `multiply(a, b)`, `divide(a, b)` (raises `DivisionByZeroError` if `b == 0`).
- Method `evaluate(expression_str)` parses "5 + 3" format. Uses try-except-else-finally:
  - try: parse and compute.
  - except `ValueError`: bad number.
  - except `InvalidOperationError`: unknown operator.
  - except `DivisionByZeroError`: division by zero.
  - else: print result (only on success).
  - finally: log "Evaluation complete" (always).

Test with valid and invalid inputs to show each exception being caught.

### Examples:

- `evaluate("5 + 3")`: SUCCESS, result = 8.0.
- `evaluate("5 / 0")`: ERROR, division by zero.
- `evaluate("5 % 3")`: ERROR, unknown operator.
- `evaluate("abc + 3")`: ERROR, invalid number.

## Test Cases:

| # | Expression | Result | Exception                          | Notes            |
|---|------------|--------|------------------------------------|------------------|
| 1 | "5 + 3"    | 8.0    | None                               | Valid addition   |
| 2 | "10 / 4"   | 2.5    | None                               | Valid division   |
| 3 | "5 / 0"    | None   | <code>DivisionByZeroError</code>   | Division by zero |
| 4 | "5 % 3"    | None   | <code>InvalidOperationError</code> | Unknown operator |
| 5 | "abc + 3"  | None   | <code>ValueError</code>            | Bad number       |



## Program 5: Inheritance + Exception Combo: Banking System (Medium)

### Problem Statement:

Design a banking system combining inheritance with exception handling:

- Custom exception: `InsufficientFundsError(Exception)`.
- `Account` (parent): `__init__(self, account_number, holder_name, balance)`, `deposit(amount)`, `withdraw(amount)` (raises `InsufficientFundsError` if `amount > balance`).
- `SavingsAccount(Account)`: overrides `withdraw()` to enforce minimum balance of 500. Raises `InsufficientFundsError` if withdrawal would leave balance below 500.
- `CurrentAccount(Account)`: overrides `withdraw()` to charge Rs 50 fee for withdrawals above 25000. Raises `InsufficientFundsError` if total debit (`amount + fee`) exceeds balance.

In main, create accounts, perform transactions wrapped in try-except to demonstrate exception handling with inheritance.

### Examples:

- `SavingsAccount("SA001", "Alice", 5000).withdraw(4000)`: succeeds (leaves 1000  $\geq$  500). `withdraw(600)`: `InsufficientFundsError` (would leave 400  $<$  500).
- `CurrentAccount("CA001", "Bob", 30000).withdraw(26000)`: succeeds, balance = 30000 - 26000 - 50 = 3950.

### Test Cases:

| # | Account Type | Balance | Withdraw | Result                              | Notes                        |
|---|--------------|---------|----------|-------------------------------------|------------------------------|
| 1 | Savings      | 5000    | 4000     | Balance: 1000                       | Valid (1000 $\geq$ 500)      |
| 2 | Savings      | 1000    | 600      | <code>InsufficientFundsError</code> | Would leave 400 $<$ 500      |
| 3 | Current      | 30000   | 26000    | Balance: 3950                       | Fee: 26000 + 50 = 26050      |
| 4 | Current      | 30000   | 25000    | Balance: 5000                       | No fee (25000 is threshold)  |
| 5 | Current      | 26000   | 25990    | <code>InsufficientFundsError</code> | 25990 + 50 = 26040 $>$ 26000 |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Min Stack (LeetCode 155, Medium)

#### Problem Statement:

Design a stack that supports push, pop, top, and retrieving the minimum element, all in  $O(1)$  time.

- `push(val)`: pushes element onto stack.
- `pop()`: removes top element.
- `top()`: returns top element.
- `get_min()`: returns minimum element in the stack.

Use two lists: one for the actual stack, one for tracking the current minimum at each level.



## Test Cases:

| # | Operations                                              | get_min Results | Notes                   |
|---|---------------------------------------------------------|-----------------|-------------------------|
| 1 | push(-2), push(0), push(-3), get_min, pop, top, get_min | -3, _, 0, -2    | Standard LC example     |
| 2 | push(5), get_min                                        | 5               | Single element          |
| 3 | push(1..5), get_min, pop x2, get_min                    | 1, 1            | Ascending, min stays    |
| 4 | push(5..1), get_min, pop, get_min                       | 1, 2            | Descending, min updates |
| 5 | push(2,2,1,1), pop, get_min, pop, get_min               | 1, 2            | Duplicate minimums      |

## Challenge 2: Design Browser History (LeetCode 1472, Medium)

### Problem Statement:

Design a browser history system:

- `BrowserHistory(homepage)`: initializes with homepage.
- `visit(url)`: visits url, clears all forward history.
- `back(steps)`: moves back up to `steps` pages. Returns current URL.
- `forward(steps)`: moves forward up to `steps` pages. Returns current URL.

Use a list with a current index pointer.

### Test Cases:

| # | Operations                               | Result        | Notes                   |
|---|------------------------------------------|---------------|-------------------------|
| 1 | visit google, facebook, youtube, back(1) | facebook.com  | Standard navigation     |
| 2 | back(10) from second page                | homepage      | Clamp to start          |
| 3 | forward(10) from middle                  | last visited  | Clamp to end            |
| 4 | visit after back (clears forward)        | new page only | Forward history cleared |
| 5 | No visits, back and forward              | homepage      | Homepage only           |

## Challenge 3: Implement Queue using Stacks (LeetCode 232, Medium)

### Problem Statement:

Implement a FIFO queue using two lists (as stacks):

- `push(x)`: pushes element to back of queue.
- `pop()`: removes and returns front element.
- `peek()`: returns front element without removing.
- `empty()`: returns True if queue is empty.

Use the lazy transfer approach: push onto `in_stack`, transfer to `out_stack` only when `out_stack` is empty and a pop/peek is requested.



**Test Cases:**

| # | Operations                               | Results                 | Notes               |
|---|------------------------------------------|-------------------------|---------------------|
| 1 | push(1), push(2), peek, pop, empty       | 1, 1, False             | Standard FIFO       |
| 2 | push(1), push(2), pop, push(3), pop, pop | 1, 2, 3                 | Interleaved FIFO    |
| 3 | push(42), peek, pop, empty               | 42, 42, True            | Single element      |
| 4 | push(1..5), pop x5                       | 1, 2, 3, 4, 5           | FIFO order verified |
| 5 | empty, push, empty, pop, empty           | True, _, False, _, True | Empty checks        |



# CDP 413 - Problem Solving 1

## KEY CONCEPTS

### Opening

Over the last 12 blocks, you have learned every tool: variables and operators, conditionals, loops, functions, lists, strings, recursion, searching, sorting, dictionaries, sets, and OOP. Starting from this block, the challenge changes. Nobody tells you which tool to use. The problem statement gives clues, and your job is to recognize the pattern.

This is how company coding tests work. You get a problem. You decide the approach. You code the solution. You test it. Blocks 413, 414, and 415 build this skill.

### Problem-Solving Framework

1. READ: Inputs, outputs, constraints, examples
2. THINK: Which pattern? Which data structure?
3. BRUTE FORCE: Get a working solution first
4. OPTIMIZE: Can it be faster? Better data structure?
5. CODE: Clean variable names, proper functions
6. TEST: Run examples, check edge cases

Step 2 is where pattern recognition matters. Here are the patterns you already know:

| Pattern            | When to Use                                      |
|--------------------|--------------------------------------------------|
| Frequency counting | "count occurrences", "most common", "duplicates" |
| Two-pointer        | sorted array, "pair that sums to", palindrome    |
| Prefix sum/product | "sum of subarray", "product except self"         |
| Sorting + logic    | "sort then process", "rank", "k-th largest"      |
| Binary search      | sorted data, "find position", $O(\log n)$ needed |
| Dict for $O(1)$    | "check if exists", "two sum", lookup table       |
| Set for membership | "unique elements", "in A but not B", "exists?"   |
| Recursion          | self-similar subproblems, trees, backtracking    |
| Class design       | "design a system", "implement a data structure"  |

When you read a problem, scan for these keywords. They point you to the right pattern.

### Common Mistakes in Problem Solving

Three mistakes students make in coding tests:

First: starting to code immediately. Resist this. Spend 3 to 5 minutes thinking before writing a single line. Understand the problem, identify the pattern, plan the approach.

Second: giving up on brute force. A correct  $O(n^2)$  solution is better than an incorrect  $O(n)$  solution. Get brute force working first, then optimize.

Third: ignoring edge cases. Empty list, single element, all same values, negative numbers, zero. Test these before submitting.

### What This Block Covers

This block focuses on Medium difficulty problems. Each problem combines two or more concepts from previous blocks, including dict frequency counting, sorting with two-pointer, lists with conditionals and lambda sorting, prefix sum extended to prefix product, and set difference.



## IN-CLASS PROGRAMS

### Program 1: Majority Element (LeetCode 169, Easy)

#### Problem Statement:

Given a list of integers `nums` of size `n`, return the majority element. The majority element is the element that appears more than  $n/2$  times. You may assume the majority element always exists.

#### Examples:

- Input: [3, 2, 3]. Output: 3. (3 appears 2 times,  $n/2 = 1$ .  $2 > 1$ .)
- Input: [2, 2, 1, 1, 1, 2, 2]. Output: 2. (2 appears 4 times,  $n/2 = 3$ .  $4 > 3$ .)

#### Test Cases:

| # | nums                  | Output | Notes               |
|---|-----------------------|--------|---------------------|
| 1 | [3, 2, 3]             | 3      | Standard LC example |
| 2 | [2, 2, 1, 1, 1, 2, 2] | 2      | Longer list         |
| 3 | [1]                   | 1      | Single element      |
| 4 | [6, 6, 6, 7, 7]       | 6      | Majority at start   |
| 5 | [1, 1, 1, 1, 2, 3, 4] | 1      | Majority $> n/2$    |

### Program 2: Three Sum (LeetCode 15, Medium)

#### Problem Statement:

Given a list of integers `nums`, return all unique triplets `[nums[i], nums[j], nums[k]]` such that  $i \neq j$ ,  $i \neq k$ ,  $j \neq k$ , and  $nums[i] + nums[j] + nums[k] == 0$ . The solution must not contain duplicate triplets.

#### Examples:

- Input: [-1, 0, 1, 2, -1, -4]. Output: [[-1, -1, 2], [-1, 0, 1]].
- Input: [0, 1, 1]. Output: []. (No triplet sums to 0.)

#### Test Cases:

| # | nums                  | Output                    | Notes               |
|---|-----------------------|---------------------------|---------------------|
| 1 | [-1, 0, 1, 2, -1, -4] | [[-1, -1, 2], [-1, 0, 1]] | Standard LC example |
| 2 | [0, 1, 1]             | []                        | No valid triplet    |
| 3 | [0, 0, 0]             | [[0, 0, 0]]               | All zeros           |
| 4 | [-2, 0, 1, 1, 2]      | [[-2, 0, 2], [-2, 1, 1]]  | Two triplets        |
| 5 | [-3, -2, -1, 1, 2, 4] | [[-3, -1, 4], [-3, 1, 2]] | Mixed values        |

### Program 3: Inventory Restock Analyzer (Medium)

#### Problem Statement:

A warehouse analyzes restocking needs. Given:

- `current_stock`: list of `N` integers (current quantities per product).
- `minimum_required`: list of `N` integers (minimum required quantities).
- `unit_cost`: list of `N` integers (cost per unit to restock).
- `budget`: integer (total budget for restocking).



Find:

- (a) Products below minimum (where  $\text{current\_stock}[i] < \text{minimum\_required}[i]$ ).
- (b) Restock quantity needed per product ( $\text{minimum\_required}[i] - \text{current\_stock}[i]$ ).
- (c) Restock cost per product ( $\text{deficit} * \text{unit\_cost}[i]$ ).
- (d) Total restock cost and whether it fits budget.
- (e) If over budget, print products prioritized by highest deficit descending.

**Examples:**

- $\text{current}=[50,80,30,100,20]$ ,  $\text{minimum}=[60,70,50,90,40]$ ,  $\text{cost}=[10,15,20,5,25]$ ,  $\text{budget}=1000$ .
- Deficits: Product 0 (deficit 10, cost 100), Product 2 (deficit 20, cost 400), Product 4 (deficit 20, cost 500). Total=1000. Within budget (exactly).

**Test Cases:**

| # | Stocks            | Min              | Cost            | Budget | Total Cost | Within? | Notes                 |
|---|-------------------|------------------|-----------------|--------|------------|---------|-----------------------|
| 1 | [50,80,30,100,20] | [60,70,50,90,40] | [10,15,20,5,25] | 1000   | 1000       | Yes     | Exactly within        |
| 2 | Same as TC1       | Same             | Same            | 500    | 1000       | No      | Over budget           |
| 3 | [100,100,100]     | [50,50,50]       | [10,10,10]      | 500    | 0          | Yes     | All above min         |
| 4 | [0]               | [100]            | [5]             | 500    | 500        | Yes     | Single product        |
| 5 | [10,20,30,40]     | [50,50,50,50]    | [2,3,4,5]       | 200    | 300        | No      | Prioritize by deficit |

## Program 4: Product of Array Except Self (LeetCode 238, Hard)

**Problem Statement:**

Given a list of integers `nums`, return a list `output` such that `output[i]` is equal to the product of all elements in `nums` except `nums[i]`. You must solve it without using division and in  $O(n)$  time.

**Examples:**

- Input: [1, 2, 3, 4]. Output: [24, 12, 8, 6].
- Input: [-1, 1, 0, -3, 3]. Output: [0, 0, 9, 0, 0].

**Test Cases:**

| # | nums              | Output           | Notes               |
|---|-------------------|------------------|---------------------|
| 1 | [1, 2, 3, 4]      | [24, 12, 8, 6]   | Standard LC example |
| 2 | [-1, 1, 0, -3, 3] | [0, 0, 9, 0, 0]  | Contains zero       |
| 3 | [2, 3]            | [3, 2]           | Two elements        |
| 4 | [1, 1, 1, 1]      | [1, 1, 1, 1]     | All ones            |
| 5 | [5, 2, 4, 3]      | [24, 60, 30, 40] | General case        |



## Program 5: Find All Numbers Disappeared in an Array (LeetCode 448, Medium)

### Problem Statement:

Given a list of  $n$  integers where  $1 \leq \text{nums}[i] \leq n$ , some elements appear twice and some appear once. Find all integers in the range  $[1, n]$  that do not appear in the array. Return them in ascending order.

### Examples:

- Input: [4, 3, 2, 7, 8, 2, 3, 1]. Output: [5, 6].
- Input: [1, 1]. Output: [2].

### Test Cases:

| # | nums              | Output       | Notes                     |
|---|-------------------|--------------|---------------------------|
| 1 | [4,3,2,7,8,2,3,1] | [5, 6]       | Standard LC example       |
| 2 | [1, 1]            | [2]          | Duplicate, one missing    |
| 3 | [1]               | []           | Nothing missing           |
| 4 | [2, 2]            | [1]          | 1 is missing              |
| 5 | [1, 2, 3, 4, 5]   | []           | Complete, nothing missing |
| 6 | [5, 5, 5, 5, 5]   | [1, 2, 3, 4] | All same, most missing    |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Subarray Sum Equals K (LeetCode 560, Medium)

#### Problem Statement:

Given a list of integers `nums` and an integer `k`, return the total number of continuous subarrays whose sum equals `k`.

#### Examples:

- Input: `nums = [1, 1, 1]`, `k = 2`. Output: 2. (Subarrays: [1,1] starting at index 0, [1,1] starting at index 1.)
- Input: `nums = [1, 2, 3]`, `k = 3`. Output: 2. (Subarrays: [1,2] and [3].)

### Test Cases:

| # | nums      | k | Output | Notes               |
|---|-----------|---|--------|---------------------|
| 1 | [1, 1, 1] | 2 | 2      | Standard LC example |
| 2 | [1, 2, 3] | 3 | 2      | [1,2] and [3]       |
| 3 | [1]       | 1 | 1      | Single match        |
| 4 | [1]       | 0 | 0      | No match            |
| 5 | [0, 0, 0] | 0 | 6      | All zeros           |

## Challenge 2: Longest Consecutive Sequence (LeetCode 128, Medium)

### Problem Statement:

Given an unsorted list of integers, find the length of the longest consecutive elements sequence. Must run in  $O(n)$  time.

### Examples:

- Input: [100, 4, 200, 1, 3, 2]. Output: 4. (Sequence: 1, 2, 3, 4.)
- Input: [0, 3, 7, 2, 5, 8, 4, 6, 0, 1]. Output: 9. (Sequence: 0 through 8.)

### Test Cases:

| # | nums                   | Output | Notes              |
|---|------------------------|--------|--------------------|
| 1 | [100, 4, 200, 1, 3, 2] | 4      | 1,2,3,4            |
| 2 | [0,3,7,2,5,8,4,6,0,1]  | 9      | 0 through 8        |
| 3 | []                     | 0      | Empty              |
| 4 | [1]                    | 1      | Single             |
| 5 | [1, 2, 0, 1]           | 3      | Duplicates (0,1,2) |

## Challenge 3: Container With Most Water (LeetCode 11, Hard)

### Problem Statement:

Given a list of non-negative integers `height` where each represents a vertical line, find two lines that together with the x-axis form a container that holds the most water. Return the maximum amount of water.

### Examples:

- Input: [1, 8, 6, 2, 5, 4, 8, 3, 7]. Output: 49. (Lines at index 1 (height 8) and index 8 (height 7). Width=7, height=7. Area=49.)

### Test Cases:

| # | height               | Output | Notes               |
|---|----------------------|--------|---------------------|
| 1 | [1,8,6,2,5,4,8,3,7]  | 49     | Standard LC example |
| 2 | [1, 1]               | 1      | Two elements        |
| 3 | [4, 3, 2, 1, 4]      | 16     | $\min(4,4)*4$       |
| 4 | [1, 2, 1]            | 2      | Small array         |
| 5 | [1,8,6,2,5,4,8,25,7] | 49     | Tall bar in middle  |

## CDP 414 - Problem Solving 2

### KEY CONCEPTS

#### Opening

Block 413 was your introduction to problem solving. You recognized patterns and applied them. Block 414 raises the difficulty. Problems are Medium to Hard. They combine more concepts. Some have multiple valid approaches with different trade-offs. The goal is the same: read the problem, identify the pattern, code the solution, test it.

#### Approaching Harder Problems

Three strategies for harder problems:

First, identify the brute force immediately. For every problem, there is a straightforward  $O(n^2)$  or  $O(n^3)$  solution. Write it. Test it. Only then think about optimization. A correct slow solution scores more than an incorrect fast solution.

Second, look for the data structure hint. If the problem asks about 'most frequent', think dict. If it asks about 'subarray sum', think prefix sum. If it asks about 'unique' or 'exists', think set. If the data is sorted or can be sorted, think two-pointer or binary search.

Third, trace through examples on paper before coding. Pick the first example. Walk through your algorithm step by step. Write out the variables at each step. If your trace produces the correct answer, code it. If not, fix the algorithm on paper first.

#### Time Management for Company Tests

Easy: 10-15 minutes

Medium: 20-25 minutes

Hard: 25-30 minutes

If stuck for 10+ minutes: MOVE ON, come back later.

In a 75-minute company test with 3 problems, you cannot afford to spend 40 minutes on one problem. Solve what you can first. Come back to the hard one with remaining time.

Edge case checklist before submitting:

- Empty input (`[]`, `""`)
- Single element (`[1]`)
- All same values (`[5,5,5]`)
- Negative numbers (`[-1,-2]`)
- Zero in data
- Very large input (will my  $O(n^2)$  time out?)

#### This Block's Focus

### IN-CLASS PROGRAMS

#### Program 1: Single Number (LeetCode 136, Easy)

##### Problem Statement:

Given a non-empty list of integers where every element appears exactly twice except for one, find that single element.

##### Examples:

- Input: `[2, 2, 1]`. Output: 1.



- Input: [4, 1, 2, 1, 2]. Output: 4.

### Test Cases:

| # | nums            | Output | Notes                 |
|---|-----------------|--------|-----------------------|
| 1 | [2, 2, 1]       | 1      | Standard LC example   |
| 2 | [4, 1, 2, 1, 2] | 4      | Standard LC example 2 |
| 3 | [1]             | 1      | Single element        |
| 4 | [-1, -1, -2]    | -2     | Negative numbers      |
| 5 | [0, 1, 0]       | 1      | Zero in list          |

### Program 2: Logistics Route Optimizer (Medium)

#### Problem Statement:

A delivery company has  $N+1$  warehouses connected by  $N$  route segments. Given a list of route distances between consecutive warehouses and a truck fuel capacity  $K$  (maximum distance on a full tank), determine:

- Whether the route is possible (any single segment exceeding  $K$  makes it impossible).
- How many refueling stops are needed.
- At which warehouses to refuel (greedy: travel as far as possible on current fuel, refuel when the next segment would exceed remaining fuel).

#### Examples:

- distances = [100, 200, 150, 100, 250], capacity = 300. Route is possible. Refuel at warehouse 2 (before segment 2) and warehouse 4 (before segment 4). Total stops: 2.
- distances = [100, 500, 100], capacity = 300. Impossible (segment 500 > capacity 300).

### Test Cases:

| # | Distances             | Capacity | Possible | Stops | Notes                    |
|---|-----------------------|----------|----------|-------|--------------------------|
| 1 | [100,200,150,100,250] | 300      | Yes      | 2     | Refuel at WH 2 and 4     |
| 2 | [100,100,100]         | 300      | Yes      | 0     | Total fits capacity      |
| 3 | [100,500,100]         | 300      | No       | N/A   | Segment exceeds capacity |
| 4 | [50]                  | 100      | Yes      | 0     | Single segment           |
| 5 | [100,100,100,100,100] | 100      | Yes      | 4     | Refuel at every stop     |

### Program 3: Top K Frequent Elements (LeetCode 347, Medium)

#### Problem Statement:

Given an integer list `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

#### Examples:



- Input: `nums = [1,1,1,2,2,3]`, `k = 2`. Output: `[1,2]`.
- Input: `nums = [1]`, `k = 1`. Output: `[1]`.

### Test Cases:

| # | nums                | k | Output      | Notes               |
|---|---------------------|---|-------------|---------------------|
| 1 | [1,1,1,2,2,3]       | 2 | [1, 2]      | Standard LC example |
| 2 | [1]                 | 1 | [1]         | Single element      |
| 3 | [4,4,4,3,3,2,2,2,1] | 3 | [2, 4, 3]   | Top 3 by frequency  |
| 4 | [1,2,3,4,5]         | 5 | [1,2,3,4,5] | All unique, k=n     |
| 5 | [-1,-1,2,2,2,3]     | 1 | [2]         | Negative numbers    |

### Program 4: Maximum Subarray, Kadane's Algorithm (LeetCode 53, Hard)

#### Problem Statement:

Given an integer list `nums`, find the contiguous subarray (containing at least one element) which has the largest sum and return that sum.

#### Examples:

- Input: `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`. Output: 6. (Subarray `[4, -1, 2, 1]`.)
- Input: `[5, 4, -1, 7, 8]`. Output: 23. (Entire array.)

### Test Cases:

| # | nums                    | Output | Subarray     | Notes                       |
|---|-------------------------|--------|--------------|-----------------------------|
| 1 | [-2,1,-3,4,-1,2,1,-5,4] | 6      | [4,-1,2,1]   | Standard LC example         |
| 2 | [1]                     | 1      | [1]          | Single element              |
| 3 | [5,4,-1,7,8]            | 23     | Entire array | All positive (nearly)       |
| 4 | [-1]                    | -1     | [-1]         | Single negative             |
| 5 | [-2,-1,-3,-4]           | -1     | [-1]         | All negative                |
| 6 | [1,2,3,-6,4,5]          | 9      | [4,5]        | Reset after negative prefix |

### Program 5: Set Matrix Zeroes (LeetCode 73, Hard)

#### Problem Statement:

Given an `m x n` integer matrix, if an element is 0, set its entire row and column to 0. You must do it in-place.

#### Examples:

- Input: `[[1,1,1],[1,0,1],[1,1,1]]`. Output: `[[1,0,1],[0,0,0],[1,0,1]]`.
- Input: `[[0,1,2,0],[3,4,5,2],[1,3,1,5]]`. Output: `[[0,0,0,0],[0,4,5,0],[0,3,1,0]]`.





## Test Cases:

| # | Matrix                          | Output                          | Notes               |
|---|---------------------------------|---------------------------------|---------------------|
| 1 | [[1,1,1],[1,0,1],[1,1,1]]       | [[1,0,1],[0,0,0],[1,0,1]]       | Standard LC example |
| 2 | [[0,1,2,0],[3,4,5,2],[1,3,1,5]] | [[0,0,0,0],[0,4,5,0],[0,3,1,0]] | Multiple zeros      |
| 3 | [[1,2],[3,4]]                   | [[1,2],[3,4]]                   | No zeros            |
| 4 | [[0]]                           | [[0]]                           | Single zero         |
| 5 | [[1,0],[0,1]]                   | [[0,0],[0,0]]                   | Diagonal zeros      |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Trapping Rain Water (LeetCode 42, Hard)

#### Problem Statement:

Given  $n$  non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

#### Examples:

- Input: [0,1,0,2,1,0,1,3,2,1,2,1]. Output: 6.

## Test Cases:

| # | height                    | Output | Notes                 |
|---|---------------------------|--------|-----------------------|
| 1 | [0,1,0,2,1,0,1,3,2,1,2,1] | 6      | Standard LC example   |
| 2 | [4,2,0,3,2,5]             | 9      | Standard LC example 2 |
| 3 | []                        | 0      | Empty                 |
| 4 | [1,2,3,4,5]               | 0      | Ascending, no trap    |
| 5 | [3,0,3]                   | 3      | Simple valley         |

### Challenge 2: Rotate Array (LeetCode 189, Medium)

#### Problem Statement:

Given a list `nums`, rotate the list to the right by  $k$  steps.

#### Examples:

- Input: `nums = [1,2,3,4,5,6,7]`,  $k = 3$ . Output: `[5,6,7,1,2,3,4]`.

## Test Cases:

| # | nums            | k | Output          | Notes                 |
|---|-----------------|---|-----------------|-----------------------|
| 1 | [1,2,3,4,5,6,7] | 3 | [5,6,7,1,2,3,4] | Standard LC example   |
| 2 | [-1,-100,3,99]  | 2 | [3,99,-1,-100]  | Standard LC example 2 |
| 3 | [1,2,3]         | 0 | [1,2,3]         | $k=0$                 |
| 4 | [1,2,3]         | 3 | [1,2,3]         | $k=n$                 |
| 5 | [1,2,3]         | 5 | [2,3,1]         | $k > n$ ( $k\%3=2$ )  |



### Challenge 3: 4Sum II (LeetCode 454, Medium)

#### Problem Statement:

Given four integer lists A, B, C, D, count how many tuples (i, j, k, l) satisfy  $A[i] + B[j] + C[k] + D[l] == 0$ .

#### Examples:

- A = [1,2], B = [-2,-1], C = [-1,2], D = [0,2]. Output: 2.

#### Test Cases:

| # | A       | B       | C       | D       | Output | Notes               |
|---|---------|---------|---------|---------|--------|---------------------|
| 1 | [1,2]   | [-2,-1] | [-1,2]  | [0,2]   | 2      | Standard LC example |
| 2 | [0]     | [0]     | [0]     | [0]     | 1      | All zeros           |
| 3 | [1]     | [1]     | [1]     | [1]     | 0      | Sum=4, impossible   |
| 4 | [-1,-1] | [-1,1]  | [-1,1]  | [1,1]   | 8      | Many combos         |
| 5 | [1,2]   | [3,4]   | [-5,-6] | [-1,-2] | 1      | One valid tuple     |

## CDP 415 - Problem Solving 3

### KEY CONCEPTS

#### Opening

This is the last content block in the Python track. Everything you have learned in Blocks 401 through 414 comes together here. Every problem is scenario-based and mirrors the format of actual company coding tests.

Every problem in this block is scenario-based and mirrors the format of actual company tests. The goal is for you to solve them with minimal guidance.

#### Company Test Patterns

| Company   | Problems | Time      | Style                         |
|-----------|----------|-----------|-------------------------------|
| TCS NQT   | 2-3      | 60-90 min | Scenario-wrapped, correctness |
| Infosys   | 3-4      | 3 hours   | Broader scope, algorithmic    |
| Cognizant | 2-3      | Moderate  | Lists, strings, math logic    |
| Capgemini | 2-3      | Moderate  | Practical computation         |
| Accenture | 2-3      | Moderate  | Clean code, correctness       |

TCS NQT is the most common test. Problems are wrapped in scenarios: a factory computing output, a delivery system optimizing routes, a school grading students. You read the scenario, extract the algorithm, and code it. TCS tests correctness and edge case handling more than algorithmic sophistication.

Infosys gives more time but harder problems. They test lists, strings, dicts, recursion, and sometimes sorting-based algorithms.

Cognizant and Capgemini focus on practical data processing. Given records, compute summaries, filter, sort, generate reports.

Accenture emphasizes clean code. Variable names matter. Code structure matters.

Company tests typically give 2 to 3 problems in 60 to 90 minutes. Practice solving under that kind of time pressure.

#### Time Management Drill

75 minutes, 3 problems:

- Problem 1 (Medium): 15-20 min
- Problem 2 (Medium/Hard): 20-25 min
- Problem 3 (Hard): 25-30 min
- Buffer: 5-10 min for review

RULE: If stuck for 10 minutes, MOVE ON.

Allocate time before you start. Read all three problems first (2 minutes). Identify the easiest one. Solve it first. Then the medium. Then the hard. Do not start with the hardest problem.

Edge case checklist before submitting each solution:

- Empty input
- Single element
- All same values
- Negative numbers
- Boundary values (0, max)
- Input exactly at threshold boundaries



## IN-CLASS PROGRAMS

### Program 1: Flight Booking Analyzer (Medium)

#### Problem Statement:

An airline tracks bookings. Given a list of N bookings, each as a tuple (passenger\_name, flight\_code, fare), compute:

- Total revenue across all bookings.
- Revenue per flight (use a dict).
- The flight with the highest revenue.
- Passengers who booked more than one flight (use a dict of lists).

Print a formatted report.

#### Examples:

- Bookings: [("Alice","AI101",5000), ("Bob","AI101",5000), ("Alice","AI202",7000), ("Charlie","AI303",3000), ("Bob","AI303",3000)].
- Total revenue: 23000. Revenue per flight: AI101=10000, AI202=7000, AI303=6000. Top flight: AI101. Multi-bookers: Alice (AI101, AI202), Bob (AI101, AI303).

#### Test Cases:

| # | Bookings                  | Total | Top Flight | Multi-Bookers | Notes                    |
|---|---------------------------|-------|------------|---------------|--------------------------|
| 1 | 5 bookings as above       | 23000 | AI101      | Alice, Bob    | Standard                 |
| 2 | [("Dan","FL01",1000)]     | 1000  | FL01       | (none)        | Single booking           |
| 3 | Eve books 3 flights       | 1800  | X3 (700)   | Eve           | One passenger, 3 flights |
| 4 | 3 passengers, same flight | 3000  | F1         | (none)        | All same flight          |
| 5 | B1=200, C1=200 (tie)      | 500   | B1 or C1   | (none)        | Revenue tie              |

### Program 2: Employee Performance Ranker (Medium)

#### Problem Statement:

A company evaluates employees. Given N employees, each with a name and scores in 4 categories (list of 4 integers), compute the weighted average using weights 30%, 25%, 25%, 20%. Rank employees by weighted average descending. If two employees have the same average, rank alphabetically by name. Print the ranked list with rank numbers.

#### Examples:

- Alice [80,70,90,85]:  $80 \cdot 0.3 + 70 \cdot 0.25 + 90 \cdot 0.25 + 85 \cdot 0.2 = 81.0$ .
- Bob [90,85,70,80]:  $27 + 21.25 + 17.5 + 16 = 81.75$ .
- Charlie [75,80,85,90]:  $22.5 + 20 + 21.25 + 18 = 81.75$ .
- Ranking: 1. Bob (81.75), 2. Charlie (81.75), 3. Alice (81.0). Bob and Charlie tie, Bob comes first alphabetically.



## Test Cases:

| # | Employees                                                      | Rank 1      | Notes                     |
|---|----------------------------------------------------------------|-------------|---------------------------|
| 1 | Alice [80,70,90,85], Bob [90,85,70,80], Charlie [75,80,85,90]  | Bob (81.75) | Tie broken alphabetically |
| 2 | Dan [100,100,100,100]                                          | Dan (100.0) | Single, perfect           |
| 3 | Eve, Frank both [50,50,50,50]                                  | Eve (50.0)  | Tied, alphabetical        |
| 4 | Zara [90,90,90,90], Amy [80,80,80,80]                          | Zara (90.0) | Clear ranking             |
| 5 | P1 [100,0,0,0], P2 [0,100,0,0], P3 [0,0,100,0], P4 [0,0,0,100] | P1 (30.0)   | Weights determine ranking |

## Program 3: Warehouse Inventory Reconciliation (Hard)

### Problem Statement:

A warehouse reconciles physical inventory against system records. Given:

- `system_records: dict {product_id: quantity}` (what the system says).
- `physical_count: dict {product_id: quantity}` (what was physically counted).
- `prices: dict {product_id: unit_price}`.

Find:

- Products in system but not physically present (missing).
- Products physically present but not in system (unregistered).
- Products with mismatched quantities (discrepancies) with monetary impact.
- Total monetary impact of all discrepancies.

Sort discrepancies by monetary impact descending. Print a reconciliation report.

### Examples:

- System: {P001:50, P002:30, P003:20, P004:10}. Physical: {P001:45, P002:30, P005:15, P003:25}.
- Missing: P004. Unregistered: P005. Discrepancies: P003 (sys 20, phys 25, diff 5, impact 1000), P001 (sys 50, phys 45, diff 5, impact 500). Total impact: 1500.

## Test Cases:

| # | Scenario                     | Missing | Unreg  | Disc Count | Total Impact | Notes       |
|---|------------------------------|---------|--------|------------|--------------|-------------|
| 1 | 4 sys, 4 phys, overlap       | P004    | P005   | 2          | 1500         | Standard    |
| 2 | Perfect match                | (none)  | (none) | 0          | 0            | No issues   |
| 3 | System has X, physical empty | X       | (none) | 0          | 0            | All missing |

|   |                              |        |        |   |      |                  |
|---|------------------------------|--------|--------|---|------|------------------|
| 4 | System empty, physical has Y | (none) | Y      | 0 | 0    | All unregistered |
| 5 | A:10vs5, B:20vs25, C:30vs10  | (none) | (none) | 3 | 4750 | All discrepant   |

#### Program 4: Group Anagrams, Timed Revisit (LeetCode 49, Hard)

##### Problem Statement:

Given a list of strings, group the anagrams together. Return a list of groups.

##### Test Cases:

| # | Input                                      | Output Groups                                    | Notes        |
|---|--------------------------------------------|--------------------------------------------------|--------------|
| 1 | ["eat", "tea", "tan", "ate", "nat", "bat"] | [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]] | Standard     |
| 2 | [""]                                       | [[""]]                                           | Empty string |
| 3 | ["a"]                                      | [["a"]]                                          | Single char  |
| 4 | ["abc", "bca", "cab", "xyz", "zyx"]        | [["abc", "bca", "cab"], ["xyz", "zyx"]]          | Two groups   |
| 5 | ["listen", "silent", "hello", "world"]     | [["listen", "silent"], ["hello"], ["world"]]     | Mixed        |

#### Program 5: Online Shopping Cart (Medium)

##### Problem Statement:

An e-commerce system processes a shopping cart. Given a list of items, each as a dict with keys "name" (str), "price" (float), "quantity" (int), "category" (str):

- Compute total cart value (sum of price \* quantity for all items).
- Apply category-wise discounts: Electronics 10%, Clothing 15%, Books 5%, other categories 0%.
- If total after discount exceeds Rs 2000, apply an additional 5% loyalty discount.
- Add 18% GST on the final amount.
- Print an itemized receipt.

##### Examples:

- Laptop (Electronics, 50000, qty 1), T-Shirt (Clothing, 500, qty 3), Python Book (Books, 600, qty 2).
- Subtotal: 52700. Category discounts: 5000+225+60=5285. After discount: 47415. Loyalty (5%): 2370.75. After loyalty: 45044.25. GST (18%): 8107.97. Final: 53152.22.

##### Test Cases:

| # | Items                       | Subtotal | After Disc | Loyalty?      | Final    | Notes                   |
|---|-----------------------------|----------|------------|---------------|----------|-------------------------|
| 1 | Laptop+Shirt+Book           | 52700    | 47415      | Yes (2370.75) | 53152.22 | Full pipeline           |
| 2 | Pen (Stationery, 50, qty 2) | 100      | 100        | No            | 118.00   | No discount, no loyalty |



|   |                     |       |       |                    |          |                               |
|---|---------------------|-------|-------|--------------------|----------|-------------------------------|
| 3 | Shirt (1000, qty 2) | 2000  | 1700  | No<br>(1700<=2000) | 2006.00  | Below<br>loyalty<br>threshold |
| 4 | Shirt (1000, qty 3) | 3000  | 2550  | Yes (127.5)        | 2858.55  | Above<br>threshold            |
| 5 | Phone+Novels        | 21500 | 19425 | Yes (971.25)       | 21775.43 | Two<br>categories             |

## CHALLENGE PROGRAMS (HOMEWORK)

### Challenge 1: Merge Intervals (LeetCode 56, Medium)

#### Problem Statement:

Given a list of intervals [start, end], merge all overlapping intervals and return the non-overlapping intervals.

#### Test Cases:

| # | Intervals                    | Output                 | Notes                    |
|---|------------------------------|------------------------|--------------------------|
| 1 | [[1,3],[2,6],[8,10],[15,18]] | [[1,6],[8,10],[15,18]] | Standard                 |
| 2 | [[1,4],[4,5]]                | [[1,5]]                | Touching                 |
| 3 | [[1,4],[2,3]]                | [[1,4]]                | Contained                |
| 4 | [[1,2]]                      | [[1,2]]                | Single                   |
| 5 | [[5,6],[1,2],[3,4]]          | [[1,2],[3,4],[5,6]]    | Non-overlapping unsorted |

### Challenge 2: Sort Colors (LeetCode 75, Medium)

#### Problem Statement:

Given a list with elements 0, 1, and 2 (representing red, white, blue), sort them in-place in one pass using the Dutch National Flag algorithm.

#### Test Cases:

| # | nums          | Output        | Notes          |
|---|---------------|---------------|----------------|
| 1 | [2,0,2,1,1,0] | [0,0,1,1,2,2] | Standard       |
| 2 | [2,0,1]       | [0,1,2]       | Three elements |
| 3 | [0]           | [0]           | Single         |
| 4 | [1,1,1]       | [1,1,1]       | All same       |
| 5 | [2,2,1,1,0,0] | [0,0,1,1,2,2] | Reverse sorted |

### Challenge 3: Sliding Window Maximum (LeetCode 239, Hard)

#### Problem Statement:

Given a list `nums` and a sliding window of size `k`, return the maximum value in each window as the window moves from left to right.

#### Examples:

- Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`. Output: `[3,3,5,5,6,7]`.

#### Test Cases:

| # | nums                | k | Output        | Notes          |
|---|---------------------|---|---------------|----------------|
| 1 | [1,3,-1,-3,5,3,6,7] | 3 | [3,3,5,5,6,7] | Standard LC    |
| 2 | [1]                 | 1 | [1]           | Single element |
| 3 | [1,2,3,4,5]         | 1 | [1,2,3,4,5]   | k=1            |
| 4 | [5,4,3,2,1]         | 5 | [5]           | k=n            |
| 5 | [7,7,7,7]           | 2 | [7,7,7]       | All same       |



## CDP 416 - Company Simulation 1, TCS NQT Pattern

### PROBLEM 1 (Medium): Courier Tracking Code Processor

#### Problem Statement

A courier company processes tracking codes. Each tracking code is a string. A valid tracking code must satisfy all of the following rules:

- Exactly 10 characters long.
- The first 2 characters are uppercase letters (the region code).
- The next 6 characters are digits (the shipment number).
- The last 2 characters are uppercase letters (the destination code).

Given N tracking codes, perform the following:

1. Separate valid and invalid codes. For each invalid code, report the specific rule that failed.
2. Among valid codes, group them by region code (first 2 letters) using a dict.
3. Within each region group, sort codes by the numeric portion (middle 6 digits) in ascending order.
4. Print: the count of valid and invalid codes, the grouped valid codes (sorted within each group), and the region with the most shipments.

#### Input:

- `n`: number of tracking codes.
- `codes`: list of n strings.

#### Output:

- Count of valid and invalid codes.
- For each invalid code: the code and the rule that failed.
- For each region: the region code and its sorted list of tracking codes.
- The region with the most shipments.

#### Example:

- Input: `codes = ["KA000123DL", "MH000456BM", "KA000089DL", "INVALID1", "KA000500MU"]`.
- Valid (4): KA000123DL, MH000456BM, KA000089DL, KA000500MU. Invalid (1): INVALID1 (length 8, not 10).
- Grouped: KA = [KA000089DL, KA000123DL, KA000500MU] (sorted by numeric), MH = [MH000456BM].
- Region with most: KA (3 shipments).

#### Test Cases:

| # | Codes                                                                  | Valid | Invalid | Top Region | Notes     |
|---|------------------------------------------------------------------------|-------|---------|------------|-----------|
| 1 | ["KA000123DL", "MH000456BM", "KA000089DL", "KA000500MU", "MH000100BM"] | 5     | 0       | KA (3)     | All valid |

|   |                                                                    |   |   |              |                       |
|---|--------------------------------------------------------------------|---|---|--------------|-----------------------|
| 2 | ["KA00012DL","1A000123DL","KA0001AB34","KAABCDEFGH","KA000123dl"]  | 0 | 5 | N/A          | All invalid           |
| 3 | ["AB123456CD"]                                                     | 1 | 0 | AB (1)       | Single valid          |
| 4 | ["KA000100DL","MH000200BM","TN000300CH","AP000400HY","GJ000500SU"] | 5 | 0 | Any (1 each) | All different regions |
| 5 | ["KA000999DL","KA000001DL","KA000500DL"]                           | 3 | 0 | KA (3)       | Sorted: 001, 500, 999 |

## PROBLEM 2 (Hard): Factory Production Analyzer

### Problem Statement

A factory has  $M$  production lines operating across  $N$  shifts. The production data is given as an  $M \times N$  matrix where `matrix[i][j]` represents the number of items produced by line  $i$  during shift  $j$ .

Compute the following:

1. Total production per line (row sums) and total production per shift (column sums).
2. The most productive line (highest row sum) and the most productive shift (highest column sum). If tie, the one with the smaller index.
3. A line is "underperforming" if its total production is below the factory average (average of all line totals). Report all underperforming lines.
4. A shift is "peak" if its total production exceeds 1.5 times the average shift production. Report all peak shifts.
5. Compute the "efficiency score" per line:  $(\text{line\_total} / \text{max\_possible}) * 100$  where  $\text{max\_possible} = \text{number\_of\_shifts} * \text{highest\_single\_cell\_value\_in\_the\_entire\_matrix}$ . Rank lines by efficiency score descending. If two lines have the same score, rank by line number ascending.

Print a formatted production report.

#### Input:

- $m, n$ : number of lines and shifts.
- $M$  rows of  $N$  integers each (the production matrix).

#### Output:

- Line totals and shift totals.
- Most productive line and shift.
- Underperforming lines.
- Peak shifts.



- Efficiency ranking.

#### Example:

- M=4, N=4. Matrix:

...

100 120 90 110

80 70 60 50

150 140 130 160

90 100 95 85

...

Line totals: [420, 260, 580, 370]. Shift totals: [420, 430, 375, 405].

Best line: 2 (580). Best shift: 1 (430). Factory avg: 407.5.

Underperforming: Line 1 (260), Line 3 (370).

Peak shifts: none (max shift 430, threshold 611.25).

Max cell: 160. Max possible: 640. Efficiency: L2=90.62%, L0=65.62%, L3=57.81%, L1=40.62%.

#### Test Cases:

| # | M x N                             | Best Line    | Best Shift    | Underperforming | Peak | Notes          |
|---|-----------------------------------|--------------|---------------|-----------------|------|----------------|
| 1 | 4x4 (as above)                    | Line 2 (580) | Shift 1 (430) | Lines 1, 3      | None | Standard       |
| 2 | 1x3 [[50,50,50]]                  | Line 0 (150) | Shift 0 (50)  | None            | None | Single line    |
| 3 | 3x2<br>[[10,10],[10,10],[10,10]]  | Line 0 (20)  | Shift 0 (30)  | None            | None | All identical  |
| 4 | 2x2 [[200,10],[10,200]]           | Line 0 (210) | Shift 0 (210) | None            | None | Diagonal heavy |
| 5 | 2x3<br>[[100,200,300],[10,20,30]] | Line 0 (600) | Shift 2 (330) | Line 1 (60)     | None | Imbalanced     |

## PROBLEM 3 (Hard): Telecom Call Record Analyzer

### Problem Statement

A telecom company analyzes call records. Each record is given as three values: caller name, receiver name, and call duration in minutes.

Given N call records and a threshold T, compute:

1. Total call duration per person (both incoming and outgoing calls count toward a person's total).
2. The person who spent the most total time on calls.
3. The most frequently called pair. A pair is undirected: Alice calling Bob and Bob calling Alice are the same pair. If multiple pairs tie, choose the pair whose names come first alphabetically (compare the sorted pair tuples).
4. All callers whose average outgoing call duration exceeds the threshold T. Report them alphabetically.



5. Print a summary report.

#### Input:

- $n$ : number of call records.
- $N$  lines, each with: caller\_name receiver\_name duration\_minutes.
- $t$ : the threshold for average outgoing duration.

#### Output:

- Total duration per person.
- Person with most total time.
- Most frequent pair and count.
- Callers above threshold with their average outgoing duration.

#### Example:

- Records: [("Alice","Bob",10), ("Bob","Alice",15), ("Alice","Charlie",20), ("Charlie","Alice",5), ("Bob","Charlie",8)]. Threshold: 12.
- Person totals: Alice=50, Bob=33, Charlie=33.
- Top person: Alice (50 min).
- Pairs: (Alice,Bob):2, (Alice,Charlie):2, (Bob,Charlie):1. Tie, alphabetical: (Alice,Bob).
- Outgoing averages: Alice=(10+20)/2=15, Bob=(15+8)/2=11.5, Charlie=5/1=5. Above 12: Alice (15.0).

#### Test Cases:

| # | Records                                                                      | Thresh old | Top Pers on  | Top Pair             | Above Thresh old    | Notes            |
|---|------------------------------------------------------------------------------|------------|--------------|----------------------|---------------------|------------------|
| 1 | 5 records as above                                                           | 12         | Alice (50)   | (Alice,B ob) count 2 | Alice (15.0)        | Stand ard        |
| 2 | [("X","Y",100)]                                                              | 50         | X or Y (100) | (X,Y) count 1        | X (100.0)           | Single record    |
| 3 | [("A","B",5) x3, ("C","D",10)]                                               | 6          | A or B (15)  | (A,B) count 3        | C (10.0)            | Repea ted pair   |
| 4 | [("P","Q",30),("Q","P",30),("R","S",30),("S","R",30)]                        | 25         | Any (60)     | (P,Q) count 2        | P,Q,R, S (all 30.0) | All tied         |
| 5 | [("Alice","Bob",2),("Alice","Charlie",3),("Alice","Dan",1),("Bob","Eve",50)] | 10         | Bob (52)     | (Alice,B ob) count 1 | Bob (50.0)          | One heavy caller |

# CDP 417 - Company Simulation 2, Infosys and Cognizant Pattern

## PROBLEM 1 (Medium): Social Media Hashtag Analyzer

### Problem Statement

A social media platform analyzes hashtags in user posts. Each post is a string that may contain regular words and hashtags (words beginning with #).

Given N posts and a trending threshold T:

1. Extract all hashtags from all posts. Hashtags are case-insensitive (convert to lowercase). A hashtag must start with # and have at least one character after it.
2. Count the frequency of each unique hashtag using a dict.
3. Identify trending hashtags: those with frequency  $\geq T$ . Print them sorted by frequency descending. If two hashtags have the same frequency, sort them alphabetically.
4. Find the post that contains the most hashtags and report its index (0-based) and count.
5. Print a summary: total unique hashtags, trending hashtags list, and the post with the most hashtags.

### Input:

- n: number of posts.
- N lines, each a post string.
- t: the trending threshold.

### Output:

- Total unique hashtags.
- Trending hashtags with frequencies.
- Post index with the most hashtags and count.

### Example:

- Posts: ["Loving the weather #sunny #beautiful", "Great day #sunny #happy #outdoor", "Nothing beats #sunny days #beautiful", "Feeling #happy today"]. Threshold: 2.
- Unique hashtags: 4 (#sunny, #beautiful, #happy, #outdoor).
- Trending (freq  $\geq 2$ ): #sunny (3), #beautiful (2), #happy (2).
- Post with most hashtags: Post 1 (3 hashtags).

### Test Cases:

| # | Posts                                             | Threshold | Unique | Trending                            | Most Hashtags Post | Notes       |
|---|---------------------------------------------------|-----------|--------|-------------------------------------|--------------------|-------------|
| 1 | 4 posts with #sunny, #beautiful, #happy, #outdoor | 2         | 4      | #sunny(3), #beautiful(2), #happy(2) | Post 1 (3)         | Standard    |
| 2 | ["No hashtags                                     | 1         | 0      | (none)                              | Post 0 (0)         | No hashtags |

|   |                                                                              |   |   |                                         |            |                           |
|---|------------------------------------------------------------------------------|---|---|-----------------------------------------|------------|---------------------------|
|   | here", "Just plain text"]                                                    |   |   |                                         |            |                           |
| 3 | ["#Python<br>#python<br>#PYTHON"]                                            | 1 | 1 | #python(3)                              | Post 0 (3) | Case insensitive          |
| 4 | ["#a #b #c",<br>"#d #e", "#f"]                                               | 2 | 6 | (none)                                  | Post 0 (3) | All unique, none trending |
| 5 | ["#sale #sale<br>#discount",<br>"#sale<br>#offer",<br>"#discount<br>#offer"] | 2 | 3 | #sale(3),<br>#discount(2),<br>#offer(2) | Post 0 (3) | Repeated in same post     |

## PROBLEM 2 (Hard): Gift Box Combination Finder

### Problem Statement

A gift shop prepares custom gift boxes. Each box has a target weight limit that must be matched exactly. Given a list of N gift items (each with a weight in grams), find ALL combinations of items whose total weight equals exactly the target weight.

Rules:

- Each item can be used at most once.
- Print all valid combinations. Sort them by number of items (fewest first). If two combinations have the same number of items, sort them lexicographically (by weight values).
- If no valid combination exists, print "NO VALID COMBINATION".

Constraint:  $N \leq 15$  (small enough for recursive exploration).

**Input:**

- `n`: number of items.
- `weights`: list of n positive integers (item weights in grams).
- `target`: the exact target weight.

**Output:**

- All valid combinations (sorted as specified), or "NO VALID COMBINATION".

**Example:**

- Items: [2, 3, 5, 7, 1], Target: 8.
- Sorted items: [1, 2, 3, 5, 7].
- Valid combinations: [1, 7], [3, 5], [1, 2, 5].
- Output (sorted by length, then lex): [1, 7], [3, 5], [1, 2, 5].

**Test Cases:**

| # | Weights     | Target | Valid Combinations    | Notes            |
|---|-------------|--------|-----------------------|------------------|
| 1 | [2,3,5,7,1] | 8      | [1,7], [3,5], [1,2,5] | 3 combos, sorted |
| 2 | [1,2,3]     | 100    | NO VALID COMBINATION  | Impossible       |



|   |           |   |              |                     |
|---|-----------|---|--------------|---------------------|
| 3 | [5]       | 5 | [5]          | Single item matches |
| 4 | [1,1,1,1] | 2 | [1,1] x 6    | Duplicate weights   |
| 5 | [3,4,5,6] | 9 | [3,6], [4,5] | Two pairs           |

## PROBLEM 3 (Hard): Library Management System

### Problem Statement

A library management system tracks books and processes borrow requests. Design the system with the following:

**Class `Book`:** attributes title, author, copies\_available, and a waitlist (initially empty list). Methods:

- `is_available()`: returns True if copies\_available > 0.
- `borrow(member_name)`: if available, decrement copies and return True. If not available, add member\_name to waitlist and return False.

**Class `Library`:** holds a dict of books (title as key, Book object as value). Methods:

- `add_book(title, author, copies)`: creates and stores a Book.
- `process_requests(requests)`: given a list of (member\_name, book\_title) tuples, process each request in order. Returns: list of fulfilled requests, list of waitlisted requests, and list of books with no remaining copies.

Given N books and M borrow requests, process all requests and print:

1. Fulfilled requests (member got the book).
2. Waitlisted requests (book unavailable or not found).
3. Books with zero copies remaining.

### Input:

- n: number of books. N lines with: title, author, copies.
- m: number of requests. M lines with: member\_name, book\_title.

### Output:

- Fulfilled requests.
- Waitlisted requests.
- Out-of-stock books.

### Example:

- Books: ("Python Basics", "Author A", 2), ("Data Science", "Author B", 1).
- Requests: Alice borrows Python Basics (OK), Bob borrows Python Basics (OK), Charlie borrows Python Basics (waitlisted, 0 copies), Dan borrows Data Science (OK), Eve borrows Data Science (waitlisted).
- Fulfilled: Alice-Python Basics, Bob-Python Basics, Dan-Data Science. Waitlisted: Charlie-Python Basics, Eve-Data Science. Out of stock: Data Science, Python Basics.

### Test Cases:

| # | Books                         | Requests   | Fulfilled | Waitlisted | Out of Stock | Notes    |
|---|-------------------------------|------------|-----------|------------|--------------|----------|
| 1 | 3 books<br>(copies:<br>2,1,3) | 6 requests | 4         | 2          | 2 books      | Standard |



|   |                                |                             |   |   |   |                     |
|---|--------------------------------|-----------------------------|---|---|---|---------------------|
| 2 | 1 book<br>(copies: 5)          | 2 requests                  | 2 | 0 | 0 | Plenty of<br>copies |
| 3 | 1 book<br>(copies: 0)          | 2 requests                  | 0 | 2 | 1 | Zero copies         |
| 4 | 1 book                         | Request for<br>missing book | 0 | 1 | 0 | Book not<br>found   |
| 5 | 3 books<br>(copies: 1<br>each) | 6 requests (2<br>per book)  | 3 | 3 | 3 | All depleted        |



# CDP 418 - Company Simulation 3, Capgemini and Accenture Pattern

## PROBLEM 1 (Medium): Hospital Records Processor

### Problem Statement

A hospital records system receives patient data as pipe-delimited strings in the format:

patient\_id|patient\_name|department|billing\_amount

For example: "P001|Ravi|Cardiology|4500"

Given N such records:

1. Parse each record and store as a list of dicts with keys: "id", "name", "dept", "amount".
2. Compute department-wise total billing and patient count using dicts.
3. Find the department with the highest total billing.
4. Find the department with the most patients.
5. Print a formatted summary with departments sorted alphabetically.

### Input:

- n: number of records.
- N lines, each a pipe-delimited string.

### Output:

- Department-wise billing and patient count (sorted alphabetically by department name).
- Department with highest billing.
- Department with most patients.

### Example:

- Records: ["P001|Ravi|Cardiology|4500", "P002|Priya|Neurology|6000", "P003|Amit|Cardiology|3500", "P004|Sita|Orthopedics|5000", "P005|Kiran|Neurology|7000"].
- Billing: Cardiology=8000, Neurology=13000, Orthopedics=5000.
- Count: Cardiology=2, Neurology=2, Orthopedics=1.
- Highest billing: Neurology (13000). Most patients: Cardiology (2, first with max count).

### Test Cases:

| # | Records                                                         | Highest Billing   | Most Patients  | Notes               |
|---|-----------------------------------------------------------------|-------------------|----------------|---------------------|
| 1 | 5 records:<br>Cardiology(2),<br>Neurology(2),<br>Orthopedics(1) | Neurology (13000) | Cardiology (2) | Standard            |
| 2 | 1 record: General                                               | General (1000)    | General (1)    | Single record       |
| 3 | 3 records, all dept X                                           | X (600)           | X (3)          | All same department |
| 4 | 3 records, 3 different depts                                    | Dept3 (3000)      | Any (1 each)   | One per department  |

|   |                                           |                        |         |                    |
|---|-------------------------------------------|------------------------|---------|--------------------|
| 5 | 3 records with float amounts, ENT and Eye | ENT or Eye (1000 each) | ENT (2) | Float billing, tie |
|---|-------------------------------------------|------------------------|---------|--------------------|

## PROBLEM 2 (Medium): Vehicle Insurance Premium Calculator

### Problem Statement

A vehicle insurance company calculates premiums based on multiple factors. Given:

- Vehicle type: 1=Two-wheeler, 2=Car, 3=Commercial.
- Vehicle age (years).
- Driver age (years).
- Accident history count (number of past accidents).
- Coverage level: 1=Basic, 2=Comprehensive.

Premium calculation rules:

1. **Base premium:** Two-wheeler=Rs 2000, Car=Rs 8000, Commercial=Rs 15000.
2. **Depreciation surcharge:** If vehicle age > 5 years, add 15% of base premium.
3. **Driver age surcharge:** If driver age < 25 or > 65, add 10% of base premium.
4. **Accident surcharge:** Each accident adds 10% of base premium.
5. **Coverage multiplier:** Basic=1.0x, Comprehensive=1.5x (applied on the total after all surcharges).

Validation: vehicle\_type must be 1, 2, or 3. All ages must be > 0. Accident count must be >= 0.

Coverage must be 1 or 2. Raise ValueError for invalid inputs.

Print an itemized premium breakdown.

#### Input:

- vehicle\_type, vehicle\_age, driver\_age, accidents, coverage (all integers, one per line).

#### Output:

- Itemized breakdown: base, each surcharge, subtotal, coverage multiplier, final premium.
- Or error message for invalid inputs.

#### Example:

- Vehicle type=2 (Car), age=3, driver age=30, accidents=1, coverage=1 (Basic).  
- Base: 8000. Depreciation: 0 (3 <= 5). Age surcharge: 0 (25 <= 30 <= 65). Accident: 800 (1 \* 10% \* 8000). Subtotal: 8800. Coverage: 1.0x. Final: 8800.

### Test Cases:

| # | Type   | V.Age | D.Age | Acc | Cov      | Final | Notes                             |
|---|--------|-------|-------|-----|----------|-------|-----------------------------------|
| 1 | Car(2) | 3     | 30    | 1   | Basic(1) | 8800  | 1 accident surcharge only         |
| 2 | TW(1)  | 8     | 22    | 0   | Comp(2)  | 3750  | Old vehicle + young driver + Comp |

|   |            |    |    |   |          |            |                         |
|---|------------|----|----|---|----------|------------|-------------------------|
| 3 | Comm(3)    | 10 | 70 | 3 | Comp(2)  | 34875      | All surcharges + Comp   |
| 4 | Car(2)     | 1  | 40 | 0 | Basic(1) | 8000       | Clean record, base only |
| 5 | Invalid(4) | 5  | 30 | 0 | Basic(1) | ValueError | Invalid vehicle type    |

## PROBLEM 3 (Hard): Food Delivery Platform Analytics

### Problem Statement

An online food delivery platform analyzes order data. Each order is given as a dict with keys:

- `customer_id` (string)
- `restaurant_name` (string)
- `items` (list of strings, the food items ordered)
- `total_amount` (float, the order total)

Given N orders, compute:

1. **Total revenue per restaurant** using a dict.
2. **Top 3 restaurants by revenue**, sorted by revenue descending. If fewer than 3 restaurants exist, show all.
3. **Most popular item** across all orders. Count frequency of each item across all orders (flatten all item lists, count with dict). Report the item with the highest frequency.
4. **Frequent customers**: customers who ordered from 3 or more different restaurants. Use a dict of sets to track unique restaurants per customer. Report them sorted alphabetically.
5. Print a comprehensive analytics report.

### Input:

- `n`: number of orders.
- For each order: `customer_id`, `restaurant_name`, `items` (comma-separated), `total_amount`.

### Output:

- Revenue per restaurant.
- Top 3 restaurants.
- Most popular item.
- Frequent customers.

### Example:

- 6 orders from 3 restaurants. Customer C1 orders from Pizza Palace, Burger Barn, and Sushi Spot (3 different restaurants). Item "Coke" appears in 3 orders.
- Top 3: Pizza Palace (1000), Sushi Spot (600), Burger Barn (530). Most popular: Coke (3). Frequent customers: C1.

## Test Cases:

| # | Orders                              | Top Restaurant      | Most Popular Item       | Frequent Customers | Notes                          |
|---|-------------------------------------|---------------------|-------------------------|--------------------|--------------------------------|
| 1 | 6 orders, 3 restaurants             | Pizza Palace (1000) | Coke (3)                | C1                 | Standard                       |
| 2 | 1 order                             | R1 (100)            | A (1)                   | (none)             | Single order                   |
| 3 | 4 orders, 1 customer, 4 restaurants | R4 (400)            | X (4)                   | C1                 | One customer, many restaurants |
| 4 | 3 orders, all same restaurant       | R1 (450)            | A or B or C (tied at 2) | (none)             | Item frequency tie             |
| 5 | 6 orders, all tied in revenue       | Any (1000 each)     | P or Q (tied at 3)      | C1, C2             | Everything tied                |